

**Diseño e Implementación de un Agente en Espacio de
Usuario**
**para la Gestión Dinámica de Frecuencia (DVFS) en
Sistemas**
**Heterogéneos mediante Modelos Ligeros de Machine
Learning**

Yeison Adrián Cáceres Torres

Ricardo Andrés Pérez Porras

Trabajo de Grado para Optar el Título de Ingeniero de Sistemas

Director

Gilberto Javier Díaz Toro

Doctor en Ingeniería

Universidad Industrial de Santander

Facultad de Ingenierías Físico-Mecánicas

Escuela de Ingeniería de Sistemas e Informática

Bucaramanga

2026

Agradecimientos

Resumen

Título: Diseño e Implementación de un Agente en Espacio de Usuario para la Gestión Dinámica de Frecuencia (DVFS) en Sistemas Heterogéneos mediante Modelos Ligeros de Machine Learning.

Autores: Yeison Adrián Cáceres Torres; Ricardo Andrés Pérez Porras.

Palabras Clave: DVFS; eficiencia energética; computación de alto rendimiento; modelo Roofline; telemetría microarquitectónica; aprendizaje automático supervisado.

Descripción:

El consumo energético constituye hoy una restricción estructural para el escalamiento de la computación de alto rendimiento (HPC), particularmente en nodos heterogéneos que integran CPU y aceleradores gráficos. El Escalado Dinámico de Voltaje y Frecuencia (DVFS) es el principal mecanismo de control disponible a nivel de hardware, pero su aprovechamiento efectivo depende de decidir, durante la ejecución, cuál punto operativo conviene según el comportamiento de la carga. Los gobernadores nativos del sistema operativo deciden a partir de la utilización agregada del procesador, una señal que no distingue si el rendimiento está limitado por la capacidad aritmética del núcleo o por la transferencia de datos desde memoria; en el segundo caso, sostener la frecuencia máxima incrementa el consumo sin una mejora proporcional del tiempo de ejecución.

Este trabajo propone el diseño, la implementación y la evaluación de un agente en espacio de usuario que infiere el régimen de ejecución de la aplicación a partir de telemetría microarquitectónica y traduce esa inferencia en decisiones discretas de DVFS sobre CPU y GPU, con el objetivo de optimizar el Producto Energía–Retardo (EDP) sin incurrir en una penalización severa del rendimiento ni en una sobrecarga de inferencia que anule el beneficio

energético.

El desarrollo se organiza en cuatro fases: caracterización de cargas y construcción del conjunto de datos; entrenamiento y selección de un clasificador supervisado ligero; implementación del servicio de control; y validación experimental frente a los gobernadores nativos de Linux. El presente documento reporta el instrumento de medición y el protocolo experimental contruidos durante la primera fase — cuyo aporte metodológico central es un procedimiento reproducible para etiquetar como *compute-bound* o *memory-bound* cada ventana de ejecución de CPU o corrida completa de GPU, mediante una calibración empírica del modelo Roofline sobre la plataforma de medición, con trazabilidad explícita de la calidad de cada observación — junto con un primer entrenamiento y análisis del clasificador de GPU, posible ya dentro de esta fase por disponer de su conjunto de entrenamiento definido.

Abstract

Title: Design and Implementation of a User-Space Agent for Dynamic Frequency Management (DVFS) in Heterogeneous Systems through Lightweight Machine Learning Models.

Authors: Yeison Adrián Cáceres Torres; Ricardo Andrés Pérez Porras.

Keywords: DVFS; energy efficiency; high performance computing; Roofline model; micro-architectural telemetry; supervised machine learning.

Description:

Energy consumption has become a structural constraint on the scaling of high performance computing (HPC), particularly on heterogeneous nodes that combine CPUs and graphics accelerators. Dynamic Voltage and Frequency Scaling (DVFS) is the main control mechanism available at the hardware level, but exploiting it effectively requires deciding, at run time, which operating point suits the current workload behaviour. Native operating system governors base that decision on aggregate processor utilisation, a signal that does not distinguish whether performance is limited by the arithmetic capability of the core or by data transfer from memory; in the latter case, sustaining maximum frequency raises power draw without a proportional improvement in execution time.

This work proposes the design, implementation and evaluation of a user-space agent that infers the execution regime of an application from microarchitectural telemetry and translates that inference into discrete DVFS decisions over CPU and GPU, aiming to optimise the Energy-Delay Product (EDP) without incurring a severe performance penalty or an inference overhead that cancels the energy benefit.

The work is organised in four phases: workload characterisation and dataset construction; training and selection of a lightweight supervised classifier; implementation of the control

service; and experimental validation against the native Linux governors. This document reports the measurement instrument and experimental protocol built during the first phase, whose central methodological contribution is a reproducible procedure for labelling execution windows as *compute-bound* or *memory-bound* through an empirical calibration of the Roofline model on the measurement platform, with explicit traceability of the quality of each observation.

Índice general

Agradecimientos	1
Resumen	2
Abstract	4
Introducción	13
Planteamiento y justificación del problema	14
Objetivos	15
1. Marco de referencia	17
1.1. Marco Conceptual	17
1.1.1. Termodinámica del silicio y potencia CMOS	18
1.1.2. Modelo Roofline y regímenes de limitación del rendimiento	18
1.1.3. Telemetría microarquitectónica	21
1.1.4. Espacios de ejecución: <i>user-space</i> y <i>kernel-space</i>	25
1.1.5. Interfaces estándar de observabilidad e instrumentación	26
1.1.6. Gestión dinámica de frecuencia en sistemas operativos: DVFS, P-states y <i>governors</i>	29
1.1.7. Cargas de trabajo: kernels, benchmarks y microbenchmarks	34
1.1.8. Aprendizaje automático supervisado ligero	34
1.1.9. Evaluación del desempeño de un clasificador	39
1.1.10. Hiperparámetros y selección multiobjetivo	41
1.1.11. Validación anidada y fuga de información	42
1.1.12. Comparaciones estadísticas pareadas	45
1.1.13. Producto Energía–Retardo (EDP)	45
1.1.14. Síntesis: alcance y límites del marco adoptado	46
1.2. Estado del Arte	47

2. Metodología	52
2.1. Enfoque metodológico	52
2.2. Diseño metodológico	53
2.2.1. Población y muestra	53
2.2.2. Plataforma experimental	53
2.2.3. Instrumentos de recolección	55
2.2.4. Variables observadas	57
2.3. Fase 1: Recolección de información y caracterización de cargas	60
2.3.1. Verificación previa de la plataforma	60
2.3.2. Catálogo de cargas de trabajo	61
2.3.3. Cribado de cobertura y congelación del catálogo definitivo	64
2.3.4. Calibración y criterio de etiquetado	64
2.3.5. Caracterización de intensidad operacional y calibración Roofline en GPU	65
2.3.6. Dimensionamiento de las cargas GPU y verificación de invarianza de la intensidad operacional	69
2.3.7. Observabilidad efectiva y latencia de conmutación de reloj en GPU	70
2.3.8. Matriz experimental y control de sesgos	72
2.3.9. Post-procesamiento y control de calidad de los datos	75
2.3.10. Contrato de granularidad del conjunto de entrenamiento	79
2.3.11. Medición directa de FLOPs por ventana y su validación empírica	81
2.3.12. Reproducibilidad	84
2.3.13. Selección de características y contrato de columnas	85
2.4. Fase 2. Desarrollo del modelo de aprendizaje automático	86
2.4.1. Construcción de las variables de entrada CPU	86
2.4.2. Auditoría de correlación y colinealidad	87
2.4.3. Submuestreo reproducible y ponderación	87
2.4.4. Validación anidada por familia	88
2.4.5. Congelación y auditoría del candidato CPU	89
2.5. Fase 3: Implementación del agente de control DVFS	90
2.5.1. Arquitectura de dos bucles desacoplados	90
2.5.2. Detección de fase en GPU	90
2.5.3. Diseño del bucle de CPU e integración con el clasificador	90
2.5.4. Derivación <i>offline</i> de la tabla de política	91
2.5.5. Actuación y validación en modo simulado	91
2.6. Fase 4: Validación experimental y análisis de resultados	91
2.6.1. Escenarios de comparación	91

2.6.2.	Conmutación de gobernadores y protocolo de medición	91
2.6.3.	Cálculo del EDP y prueba de significancia	92
2.6.4.	Generación del reporte comparativo	92
2.7.	Aspectos éticos	92
3.	Resultados	94
3.1.	Resultados de CPU	94
3.1.1.	Composición de fase por kernel	95
3.1.2.	Variación de la composición con la frecuencia	97
3.1.3.	Actuación DVFS, telemetría y techos Roofline de CPU	97
3.1.4.	Kernels CPU de patrón de acceso irregular	100
3.1.5.	Clasificador de fase en CPU	102
3.1.6.	Política de frecuencia por clase en CPU	128
3.2.	Resultados de GPU	129
3.2.1.	Validación de control y etiquetado	129
3.2.2.	Diagnóstico del primer conjunto GPU	129
3.2.3.	Reanálisis de campañas históricas con NVML	130
3.2.4.	Continuidad de la ruta CUPTI	133
4.	Discusión	134
5.	Conclusiones	140
5.1.	Limitaciones	141
5.2.	Trabajo Futuro	142
	Repositorio y recursos complementarios	150

Índice de tablas

2.1. Características de la plataforma experimental.	54
2.2. Señales de telemetría de CPU activas en el instrumento de producción. . . .	59
2.3. Catálogo de cargas de trabajo.	62
2.4. Rejilla fina de frecuencia de la campaña definitiva.	72
2.5. Taxonomía de clasificación de frecuencia por ventana de muestreo.	74
2.6. Taxonomía de anotaciones de calidad por ventana de muestreo.	79
2.7. FLOPs por instrucción retirada según ancho vectorial y tipo de operación, evento FP_ARITH_INST_RETIRED de Ice Lake-SP. La columna FMA ya está incorporada en el valor que reporta el contador de hardware.	82
3.1. Cobertura y composición del conjunto de intervalos CPU.	95
3.2. Parámetros finales calibrados de los cuatro kernels CPU de patrón de acceso irregular.	101
3.3. Intervalos de las familias CPU nuevas tras el filtro de calidad (campaña <code>pacca_cpu_new_families_</code>	
3.4. Efecto del esquema de partición sobre el resultado, con el mismo modelo y los mismos datos.	102
3.5. Flujo de observaciones desde la campaña CPU hasta la evaluación del modelo.	103
3.6. Intervalos elegibles y seleccionados por familia y clase.	104
3.7. Sensibilidad del resultado al tope de intervalos por celda de familia y clase. .	106
3.8. Métricas de evaluación empleadas y pregunta que responde cada una.	108
3.9. Ejemplo de cálculo con cuatro celdas reales del modelo final.	109
3.10. Valores de todas las métricas para el modelo final, sin abstención. Media de cinco semillas de muestreo, desviación entre semillas e intervalo de confianza del 95 % por remuestreo de las 30 familias.	110
3.11. Variables de entrada del candidato CPU congelado.	112
3.12. Pares marcados por la auditoría de correlación sobre el conjunto CPU elegible.	113
3.13. Factor de inflación de la varianza posterior al filtro de correlación.	113

3.14. Variantes del vector de entrada bajo el mismo modelo y el mismo protocolo. La diferencia se calcula frente al vector de 12 variables.	114
3.15. Comparación de modelos con hiperparámetros fijos. La latencia corresponde a una fila, un hilo, medida en el nodo de destino.	115
3.16. Espacio de búsqueda de hiperparámetros.	116
3.17. Búsqueda de hiperparámetros anidada frente a la configuración fija, con el mismo protocolo.	117
3.18. Configuración final del clasificador CPU.	118
3.19. Métricas por clase del modelo final sobre los 1 296 981 intervalos evaluados. .	118
3.20. Resultado del modelo final por familia retirada, sobre todos sus intervalos elegibles.	119
3.21. Resultado del modelo final por nivel de frecuencia.	122
3.22. Efecto de la decisión selectiva con umbral de confianza 0.85.	123
3.23. Celdas de familia y clase cuyo perfil de contadores más cercano pertenece a otra familia de la clase opuesta. La distancia es euclídea sobre las medianas estandarizadas de IPC, MPKI, tasa de fallos y fracción de ciclos detenidos. .	124
3.24. Resultados del candidato GPU congelado y de sus referencias.	132

Índice de figuras

1.1. Modelo Roofline conceptual	19
1.2. Unidad de Monitorización de Rendimiento (PMU)	21
1.3. Contadores de núcleo y contadores de <i>uncore</i>	23
1.4. Espacios de usuario y de kernel	25
1.5. Lectura de eventos mediante <code>perf_event</code>	27
1.6. Dominios energéticos de Intel RAPL	28
1.7. P-states y C-states del procesador	30
1.8. Jerarquía de los mecanismos DVFS	31
1.9. Solicitudes de frecuencia entre hermanos SMT	32
1.10. Dominios de procesamiento y memoria en GPU	33
1.11. Bosque aleatorio (<i>Random Forest</i>)	36
1.12. Árboles extremadamente aleatorizados (Extra-Trees)	37
1.13. Construcción aditiva de un modelo XGBoost	38
1.14. Representación conceptual del frente de Pareto para dos objetivos en conflicto: maximizar la calidad predictiva y minimizar la latencia de inferencia. Cada punto representa una configuración de hiperparámetros. Las configuraciones A–D no son dominadas, mientras que E es dominada por B al presentar simultáneamente mayor latencia y menor calidad predictiva. Fuente: elaboración propia con base en Deb et al. [18].	42
1.15. Validación cruzada anidada	44
2.1. Ruta de muestreo con estructura circular de un productor y un consumidor .	56
2.2. Recorrido de una campaña a través de las dos capas del instrumento	57
3.1. Composición de fase por kernel en CPU	96
3.2. Composición de fase por nivel de frecuencia en CPU	97
3.3. Muestreo estratificado del conjunto CPU	104
3.4. Acierto del modelo final en cada celda de familia y clase	107

3.5. Matriz de correlación de las entradas CPU	114
3.6. Comparación de modelos del clasificador CPU	116
3.7. Matrices de confusión del clasificador CPU	119
3.8. Resultado del clasificador CPU por familia	121
3.9. Calibración y compromiso de la abstención en CPU	123
3.10. Exactitud según la distancia al <i>ridge</i>	125
3.11. Curva de aprendizaje por número de familias	126
3.12. Balance de clases del conjunto GPU inicial	130
3.13. Separabilidad de las clases GPU con señales NVML	132

Introducción

En los sistemas de cómputo de alto rendimiento (HPC), el consumo energético se ha consolidado como una restricción crítica para el escalado sostenible, especialmente en arquitecturas heterogéneas CPU–GPU. En este contexto, el Escalado Dinámico de Voltaje y Frecuencia (DVFS) constituye el principal mecanismo de control a nivel de hardware, permitiendo ajustar los estados operativos de los procesadores para optimizar el compromiso entre consumo energético y tiempo de ejecución. Sin embargo, la efectividad de este mecanismo depende de la capacidad del sistema para adaptar dinámicamente la frecuencia al comportamiento cambiante de las aplicaciones, lo cual representa un desafío abierto en entornos HPC modernos.

En la literatura, este problema ha sido abordado principalmente mediante dos enfoques. Por un lado, los gobernadores de energía del sistema operativo, como `ondemand` o `schedutil`, emplean políticas reactivas basadas principalmente en la utilización del procesador, sin considerar la naturaleza microarquitectónica de la carga de trabajo, lo que limita su capacidad de adaptación a cargas de trabajo científicas dinámicas. Por otro lado, diversas propuestas han explorado estrategias heurísticas basadas en métricas microarquitectónicas, utilizando reglas deterministas para ajustar la frecuencia de operación. Aunque estos métodos presentan bajo *overhead*, su dependencia de umbrales estáticos y su limitada capacidad de generalización reducen su efectividad en escenarios donde las fases computacionales varían rápidamente.

A pesar de estos avances, persiste una limitación fundamental: la incapacidad de los enfoques actuales para capturar de forma robusta las transiciones dinámicas entre regímenes *compute-bound* y *memory-bound*, así como para adaptarse a la heterogeneidad CPU–GPU sin intervención manual o ajuste específico por carga de trabajo. Esta brecha evidencia la necesidad de mecanismos de control más flexibles que permitan inferir el régimen computacional de la aplicación en tiempo de ejecución y tomar decisiones de escalado de frecuencia fundamentadas en el perfil de ejecución.

En este contexto, el presente trabajo propone el diseño e implementación de un agente en

espacio de usuario basado en modelos ligeros de aprendizaje automático, capaz de clasificar en tiempo de ejecución las fases de una aplicación como compute-bound o memory-bound a partir de telemetría microarquitectónica, y de ajustar dinámicamente la frecuencia de CPU y GPU para optimizar el Producto Energía–Retardo (EDP). Se plantea demostrar que este enfoque mejora la eficiencia energética frente a los gobernadores nativos de Linux, manteniendo una degradación controlada del rendimiento y una sobrecarga de inferencia mínima y cuantificada mediante métricas experimentales.

Planteamiento y justificación del problema

Existe una ineficiencia energética estructural en la operación de los sistemas de alto rendimiento (HPC), especialmente en infraestructuras que integran aceleradores gráficos (GPU) para ejecutar aplicaciones científicas intensivas. Cuando una aplicación entra en una fase donde el procesamiento depende principalmente de la transferencia de datos desde memoria, mantener la CPU o la GPU a su frecuencia máxima no mejora el rendimiento de manera proporcional. En estas condiciones, los núcleos de cómputo permanecen parcialmente inactivos esperando datos, lo que genera un consumo energético elevado sin un beneficio equivalente en tiempo de ejecución. Esta desalineación entre demanda computacional y frecuencia operativa ha motivado el estudio del escalado de frecuencia como mecanismo para mejorar la eficiencia energética en aplicaciones científicas [53] y en sistemas de gran escala donde la gestión de potencia impacta el comportamiento global del sistema [16].

El problema se agrava por la naturaleza multifásica de las aplicaciones HPC, cuyo perfil de ejecución cambia a lo largo del tiempo de ejecución. En particular, estas aplicaciones alternan entre fases donde el rendimiento está limitado por la capacidad de cómputo del procesador (*compute-bound*) y fases donde está restringido por la transferencia de datos desde memoria (*memory-bound*). Esta distinción es crítica porque, en el régimen *memory-bound*, incrementos de frecuencia tienden a producir mejoras marginales en el tiempo de ejecución, mientras aumentan el consumo energético; en contraste, en fases *compute-bound* la frecuencia sí impacta el desempeño de forma más directa. Por ello, se han propuesto marcos de caracterización para identificar y clasificar estas fases en entornos HPC, con el fin de habilitar decisiones de control más informadas [4, 11].

A partir de este contexto, la gestión eficiente de frecuencia no depende únicamente de la disponibilidad del mecanismo DVFS en el hardware, sino de la capacidad del sistema para decidir, en tiempo de ejecución, cuál configuración resulta más conveniente según el comportamiento de la carga. Aunque se han propuesto mecanismos automatizados para seleccionar

frecuencias óptimas de GPU con el fin de mejorar la eficiencia energética sin comprometer significativamente el desempeño [2], persiste una brecha en la implementación de estrategias que integren de forma coordinada componentes CPU–GPU y que operen durante la ejecución de la aplicación, considerando además el costo computacional asociado al propio proceso de decisión.

Esta brecha resulta especialmente relevante en entornos académicos y científicos, donde el consumo eléctrico incide directamente en los costos operativos, la disponibilidad efectiva de recursos y la sostenibilidad institucional. En este contexto, contar con un mecanismo capaz de observar el comportamiento de la aplicación, inferir su fase de ejecución y ajustar la frecuencia de operación sin intervenir el código fuente constituye una alternativa con valor técnico y práctico.

Desde la perspectiva internacional, la literatura reciente ha mostrado avances en la optimización energética mediante escalado de frecuencia, gestión de potencia y caracterización de cargas HPC [53, 16, 4, 11, 2]. Sin embargo, a nivel nacional y regional, este tipo de soluciones aún no se encuentra ampliamente implementado en infraestructuras académicas de cómputo científico, particularmente bajo esquemas que combinen telemetría de bajo nivel, modelos ligeros de aprendizaje automático y control en espacio de usuario. En consecuencia, el problema no solo es técnicamente relevante, sino también pertinente en términos de aplicabilidad local.

A partir de esta necesidad de optimización energética en sistemas heterogéneos, se formula la siguiente pregunta de investigación que servirá como eje de desarrollo del proyecto:

¿En qué medida el diseño e implementación de un agente en espacio de usuario, guiado por modelos ligeros de aprendizaje automático, permite ajustar la frecuencia de operación (DVFS) en sistemas heterogéneos CPU–GPU para reducir el consumo energético, sin que el costo de la inferencia computacional degrade el rendimiento global de la aplicación, en comparación con los gobernadores nativos de Linux?

Objetivos

Objetivo General

Caracterizar, diseñar, implementar y evaluar un agente en espacio de usuario basado en modelos ligeros de Aprendizaje Automático para el ajuste dinámico de frecuencia (DVFS)

en sistemas heterogéneos CPU–GPU, con el propósito de maximizar la eficiencia energética en aplicaciones científicas sin inducir una penalización severa en su rendimiento global.

Objetivos Específicos

1. Caracterizar el comportamiento computacional y el consumo energético de cargas de trabajo representativas (intensivas en cómputo y en memoria) bajo distintos estados de frecuencia, recolectando telemetría de bajo nivel mediante contadores de rendimiento por hardware e interfaces de potencia estándar (Perf y RAPL para CPU y NVML para GPU).
2. Entrenar y validar modelos clásicos de Aprendizaje Automático (tales como Árboles de Decisión o Bosques Aleatorios) que sean capaces de clasificar, en tiempo de ejecución y con baja latencia de inferencia, las fases de ejecución de las aplicaciones basándose en la telemetría extraída.
3. Desarrollar un servicio de control (*daemon*) en el espacio de usuario que interactúe de forma asíncrona con el sistema operativo, leyendo el estado de los contadores de hardware, ejecutando la inferencia del modelo clasificador y aplicando políticas proactivas de DVFS a través de las interfaces estándar del sistema operativo, en función de la fase de ejecución inferida por el modelo.
4. Evaluar el impacto empírico del sistema propuesto mediante el cálculo del Producto Energía–Retardo (EDP), con el fin de determinar estadísticamente si el ahorro energético compensa la sobrecarga computacional (*overhead*) derivado de la inferencia del modelo en comparación con los gobernadores nativos de Linux.

Capítulo 1

Marco de referencia

1.1. Marco Conceptual

El ajuste dinámico de frecuencia y voltaje para la optimización energética se fundamenta en la relación no lineal entre los parámetros de operación del hardware y la disipación de calor. Sobre esa base física, este trabajo asume una cadena de dependencias que articula el resto del capítulo: el régimen que limita el rendimiento de una fase de ejecución, entendido en los términos del modelo Roofline, deja una traza observable en los contadores del procesador y en las interfaces de telemetría de la plataforma; esa traza puede resumirse en un vector de características y, si es suficientemente discriminante, permite inferir el régimen mediante un modelo de clasificación cuyo costo de inferencia sea compatible con un lazo de control; y esa inferencia habilita, a su vez, una política discreta de frecuencia cuyo beneficio neto debe juzgarse con una métrica que integre energía y tiempo. Cada eslabón de esa cadena es una suposición que puede fallar, y las secciones siguientes desarrollan tanto el concepto que lo sustenta como las condiciones bajo las cuales deja de sostenerse.

El capítulo sigue ese mismo orden. Las secciones 1.1.1 a 1.1.2 establecen el fundamento físico y el criterio de clasificación de regímenes; las secciones 1.1.3 a 1.1.5 describen qué puede observarse realmente de ese comportamiento y con qué limitaciones de alcance, privilegio y granularidad; la sección 1.1.6 presenta el mecanismo de actuación y sus restricciones temporales; la sección 1.1.7 delimita las cargas sobre las que se mide; las secciones 1.1.8 a 1.1.11 desarrollan el modelo de inferencia, su evaluación y las condiciones de validez de esa evaluación; y las secciones 1.1.12 y 1.1.13 fijan los criterios con los que se decidirá si el sistema propuesto aporta una mejora defendible.

1.1.1. Termodinámica del silicio y potencia CMOS

La disipación de potencia en un circuito CMOS moderno se descompone en potencia estática (corrientes de fuga) y potencia dinámica, originada por la conmutación de cargas capacitivas. Como DVFS actúa sobre frecuencia y voltaje, su efecto principal recae sobre la potencia dinámica, modelable para una compuerta como:

$$P_{\text{dyn}} = \alpha \cdot C \cdot V^2 \cdot f \quad (1.1)$$

donde α es el factor de actividad, C la capacitancia efectiva conmutada y V , f el voltaje y la frecuencia de operación [22]. Dado que operar a mayor frecuencia suele exigir mayor voltaje por integridad temporal, reducir frecuencia habitualmente permite también reducir voltaje, con una caída de potencia más que proporcional por la dependencia cuadrática en V . Este acoplamiento es la base física del compromiso energía–rendimiento en DVFS: cuánto se gana en potencia al bajar el punto operativo depende del comportamiento computacional predominante de la aplicación, no solo de la reducción de frecuencia en sí.

1.1.2. Modelo Roofline y regímenes de limitación del rendimiento

El modelo Roofline analiza el rendimiento de una aplicación en función de su intensidad operacional y de los límites físicos del hardware:

$$P = \min(P_{\text{pico}}, I \cdot BW_{\text{pico}}) \quad (1.2)$$

donde P_{pico} es el rendimiento aritmético pico, I la intensidad operacional (operaciones por byte transferido desde memoria) y BW_{pico} el ancho de banda pico [61]. Si el término activo es $I \cdot BW_{\text{pico}}$ el kernel es *memory-bound*; si es P_{pico} , es *compute-bound*.

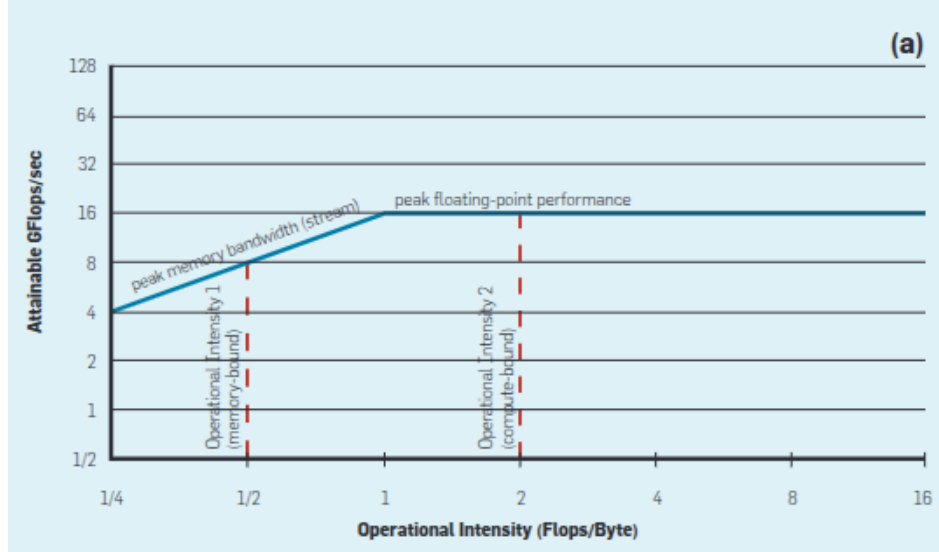


Figura 1.1: Modelo Roofline conceptual: relación entre el techo de ancho de banda de memoria y el techo de rendimiento de cómputo. Fuente: tomado de Williams et al. [61, Fig. 1(a)].

La intensidad operacional I relaciona el trabajo aritmético con los bytes transferidos desde el nivel de memoria modelado, no con el tamaño nominal del conjunto de trabajo [61, p. 66]. Por ello, que los datos excedan la caché de último nivel solo indica que parte de ellos deberá atravesar niveles inferiores de la jerarquía, pero no determina cuánto tráfico se genera ni cuánta reutilización ocurre. La multiplicación de matrices densas ilustra esta diferencia: mediante bloqueo o mosaico (*tiling*), los bloques pueden reutilizarse y su movimiento entre niveles de memoria se amortiza, sin requerir que las matrices completas residan simultáneamente en caché [23, Secs. 3–4]. Para una cantidad fija de operaciones, esa reducción del tráfico aumenta la intensidad operacional. El tamaño del conjunto de trabajo frente a la caché es, por tanto, un indicio útil, pero no un criterio suficiente para clasificar una carga como *memory-bound* o *compute-bound*.

Punto de inflexión y criterio de clasificación

La frontera entre regímenes es el punto de inflexión o *ridge point*, donde ambas ramas de (1.2) se igualan:

$$I_{\text{ridge}} = \frac{P_{\text{pico}}}{BW_{\text{pico}}} \quad (1.3)$$

I_{ridge} se convierte así en el umbral operativo de etiquetado: una intensidad observada inferior se clasifica como *memory-bound* y una superior como *compute-bound*, de acuerdo con la separación de regímenes que plantea el modelo Roofline [61]. La caracterización Roofline

puede determinar techos de cómputo por separado para distintos tipos de dato y precisiones aritméticas [3]. En consecuencia, cuando esos techos difieren, el valor de P_{pico} empleado en (1.3) debe corresponder a la precisión de las operaciones medidas; cada precisión requiere entonces su propio I_{ridge} . Una carga de precisión mixta no puede compararse directamente con un único techo sin definir previamente cómo se tratará esa mezcla.

Por la misma razón, I_{ridge} tampoco es una constante del dispositivo cuando el punto operativo varía. Ambos términos de (1.3) dependen de la frecuencia: el rendimiento aritmético pico escala con el reloj de las unidades de cómputo, mientras que el ancho de banda alcanzable hacia memoria principal responde a un dominio de reloj distinto y no necesariamente en la misma proporción [25, 2]. En consecuencia, un barrido de frecuencia desplaza el punto de inflexión, y comparar intensidades medidas en niveles distintos contra un único I_{ridge} atribuiría al régimen de la carga un efecto que en realidad procede del punto operativo. La calibración debe entonces resolverse por precisión y por nivel de frecuencia, no una sola vez por dispositivo.

Los valores de fábrica sirven como referencia inicial, pero no representan necesariamente los techos alcanzables bajo la configuración experimental. La caracterización Roofline recomienda estimarlos mediante microkernels y condiciones de ejecución representativas [48].

Cálculo de la intensidad operacional observada

Aplicar el criterio anterior exige calcular la intensidad sobre un mismo intervalo de observación:

$$I_i = \frac{\text{FLOPs}_i}{B_i^{\text{DRAM}}} \quad (1.4)$$

donde FLOPs_i representa el trabajo aritmético realizado y B_i^{DRAM} los bytes transferidos hacia o desde memoria principal durante el intervalo i [61]. Ambos términos deben corresponder al mismo intervalo temporal; dividir magnitudes obtenidas con granularidades distintas produciría una intensidad sin consistencia física (sección 1.1.3).

Los contadores de rendimiento no miden FLOPs como una magnitud abstracta, sino eventos definidos por una microarquitectura concreta. Incluso eventos aparentemente directos, como las instrucciones retiradas, pueden presentar diferencias de implementación y sensibilidad a las condiciones de medición [60]. Por ello, cualquier conversión desde eventos PMU hacia FLOPs requiere verificar la disponibilidad, la codificación y la semántica del evento utilizado.

El tráfico hacia memoria principal se observa desde un dominio distinto, mediante los contadores *uncore* del controlador de memoria integrado (IMC), descritos en la sección 1.1.3. Cuando su granularidad temporal es más gruesa que la de los contadores de núcleo, los FLOPs deben agregarse sobre el mismo intervalo antes de calcular la intensidad operacional.

1.1.3. Telemetría microarquitectónica

Los procesadores modernos incorporan Unidades de Monitorización de Rendimiento (*Performance Monitoring Units*, PMU), encargadas de registrar mediante hardware eventos asociados al comportamiento interno del procesador. Para ello, disponen de Contadores de Monitorización de Rendimiento (*Performance Monitoring Counters*, PMC), que acumulan ocurrencias de eventos como instrucciones retiradas, ciclos de ejecución, fallos de caché o accesos a determinados recursos microarquitectónicos. Los PMC pueden ser fijos o programables. Los primeros están asociados a eventos predefinidos por la arquitectura y, en la documentación de Intel, se identifican como registros de función fija independientes de los programables ($FC_0, FC_1, \dots$); los segundos permiten seleccionar el evento que se desea contabilizar mediante registros de configuración y se identifican como $PMC_0, PMC_1, \dots, PMC_n$ [33]. En arquitecturas Intel, esta selección se expresa principalmente mediante un código de evento (*Event Select*) y una máscara de unidad (*Unit Mask*), junto con calificadores adicionales cuando el evento lo requiere [33] (Figura 1.2).

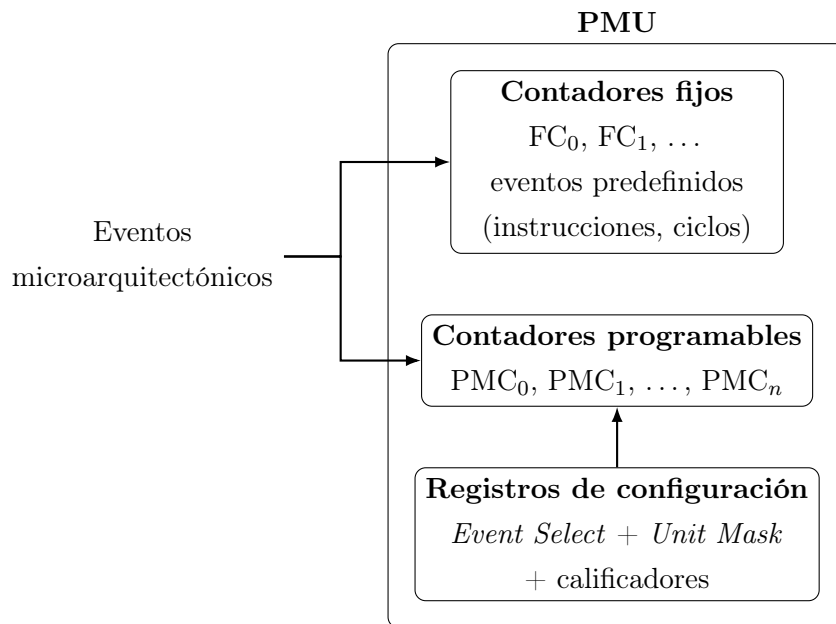


Figura 1.2: Organización conceptual de una Unidad de Monitorización de Rendimiento (PMU) con contadores fijos y programables. Fuente: elaboración propia.

La cantidad de contadores disponibles para realizar mediciones simultáneas es limitada y depende de la microarquitectura. En consecuencia, aunque el procesador pueda exponer un catálogo amplio de eventos, solo un subconjunto puede contabilizarse al mismo tiempo. Cuando la cantidad de eventos solicitados supera los contadores físicos disponibles, es necesario compartirlos mediante mecanismos de multiplexación [43], cuyo efecto sobre la medición se discute a continuación.

Multiplexación de contadores y escalado de medidas

Cuando los eventos solicitados exceden los contadores físicos disponibles, el kernel puede multiplexarlos, alternando periódicamente su asignación sobre los contadores de hardware. En este caso, el conteo observado se escala a partir de la relación entre el tiempo durante el cual el evento permaneció habilitado (*time enabled*) y el tiempo durante el cual contó efectivamente (*time running*). Este procedimiento constituye una extrapolación y supone que la tasa de ocurrencia del evento se mantiene aproximadamente estable durante los intervalos no observados, supuesto que puede perder validez cuando el comportamiento de la carga cambia dentro de la ventana [43].

La multiplexación introduce, por tanto, una fuente adicional de incertidumbre en la medición. Para reducirla, conviene limitar el conjunto de eventos a la capacidad física de la PMU y verificar la relación entre *time running* y *time enabled*, que permite identificar si una lectura estuvo sujeta a multiplexación [43, 41].

Eventos genéricos y eventos crudos

Los eventos genéricos son una abstracción portable que el kernel traduce a la codificación concreta de la microarquitectura; los eventos crudos se especifican mediante la codificación nativa del procesador (selector de evento, máscara de subevento y calificadores adicionales) [59, 33]. Cuando el kernel carece de una traducción para un evento disponible en el procesador, puede ser necesario usar su codificación cruda, lo que traslada al instrumento la responsabilidad de verificar que esa codificación corresponde a la microarquitectura evaluada. Una codificación incorrecta puede producir valores sin significado físico aun cuando la apertura del contador no genere un error explícito.

Contadores de núcleo y contadores de *uncore*

Los contadores de núcleo observan eventos atribuibles a un núcleo concreto y pueden asociarse a un proceso. Los contadores de *uncore* observan recursos compartidos del zócalo, como el controlador de memoria, la caché de último nivel distribuida y las interconexiones,

y por tanto son de alcance de sistema, no atribuibles a un proceso específico. Los contadores del controlador de memoria contabilizan el tráfico que alcanza la memoria principal, tanto por accesos de demanda como por precarga [30], por lo que constituyen la fuente estándar para estimar el denominador de la intensidad operacional (ecuación (1.4)). Su acceso, sin embargo, está sujeto a una categoría de privilegio distinta a la de los contadores de núcleo (sección 1.1.5).

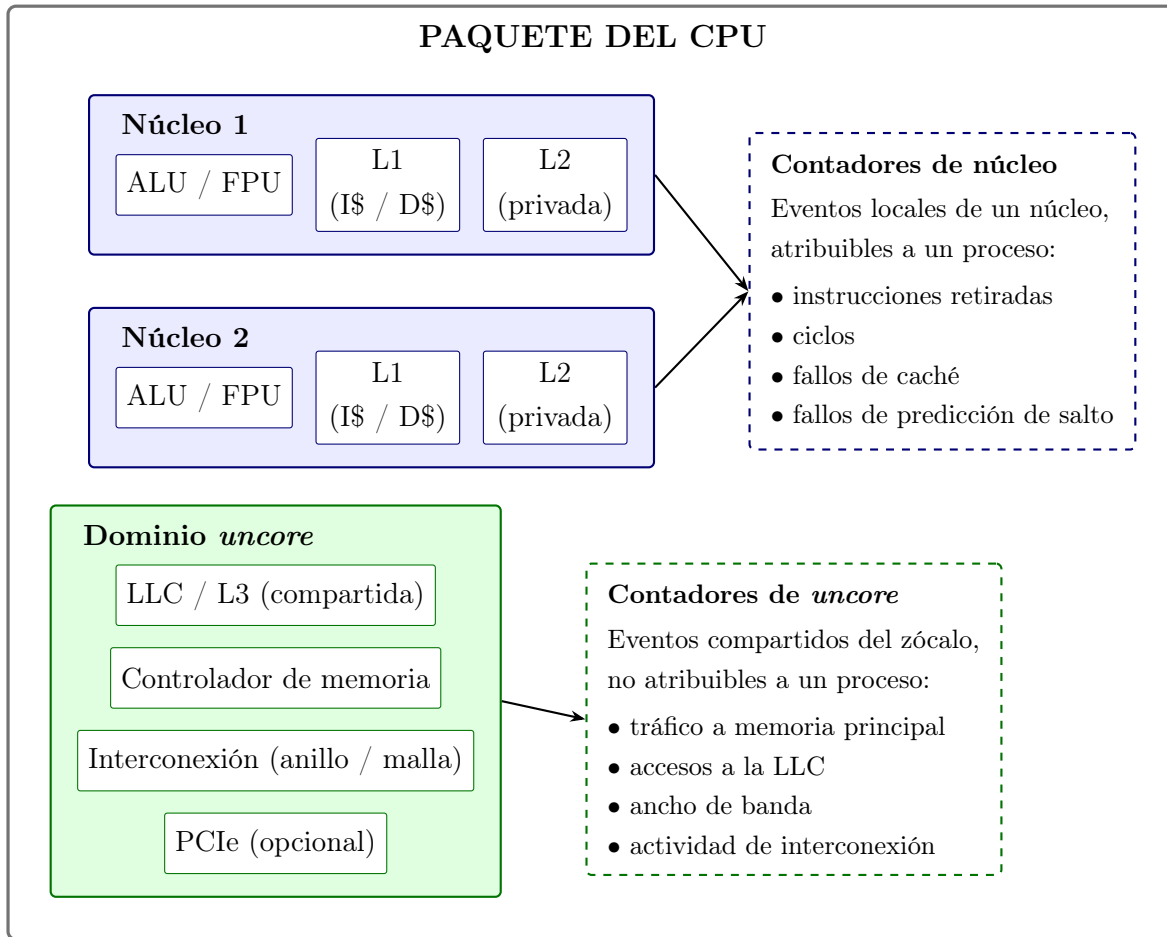


Figura 1.3: Ámbito de atribución de los contadores de núcleo y de *uncore* dentro del paquete del procesador. Los contadores de núcleo observan eventos locales de un núcleo concreto y pueden asociarse a un proceso; los de *uncore* observan recursos compartidos del zócalo y son de alcance de sistema. Fuente: elaboración propia.

Métricas microarquitectónicas de eficiencia

Las instrucciones por ciclo (IPC), las métricas de fallos de caché y los ciclos de estancamiento describen aspectos complementarios del comportamiento de la ejecución. El IPC relaciona las instrucciones retiradas con los ciclos transcurridos y constituye una medida del aprovechamiento efectivo del procesador [24, Ch. 6]. Por su parte, los eventos genéricos de

Linux `cache-references` y `cache-misses` permiten derivar métricas como la tasa de fallos de caché y los fallos por cada mil instrucciones (MPKI). La primera normaliza los fallos respecto al número de referencias, mientras que el MPKI los normaliza respecto al trabajo retirado, por lo que ambas métricas describen aspectos distintos de la presión ejercida sobre la jerarquía de memoria. No obstante, la abstracción genérica de estos eventos no garantiza un nivel específico de caché: aunque habitualmente se asocian al último nivel, su correspondencia puede variar entre procesadores e incluir accesos de precarga o tráfico de coherencia [41].

Los ciclos de estancamiento representan intervalos durante los cuales la ejecución no progresa al ritmo esperado. Sin embargo, un contador agregado de estancamiento del *backend* no permite identificar por sí solo la causa del bloqueo. Este puede originarse tanto en esperas relacionadas con memoria como en limitaciones internas del núcleo, entre ellas dependencias entre instrucciones o saturación de recursos de ejecución [29]. Para obtener una señal más directamente relacionada con presión de memoria pueden emplearse eventos que contabilizan ciclos con cargas pendientes hacia el subsistema de memoria, disponibles de forma específica para determinadas microarquitecturas [32]. Como ocurre con otros eventos crudos de la PMU (sección 1.1.3), su disponibilidad, codificación y semántica deben verificarse para la plataforma evaluada antes de interpretarlos [60, 33].

En conjunto, IPC, fallos de caché y ciclos de estancamiento constituyen señales microarquitectónicas complementarias para caracterizar el comportamiento de una carga y pueden utilizarse como características de entrada para un clasificador. Sin embargo, ninguna de estas métricas determina por sí sola que una ejecución sea *compute-bound* o *memory-bound*. En el modelo Roofline, esta distinción se establece a partir de la intensidad operacional de la carga y de su posición respecto al punto de inflexión de la plataforma [61] (sección 1.1.2).

Ventanas de muestreo y naturaleza temporal de la observación

En modo de conteo, las lecturas de los eventos de rendimiento representan valores acumulados desde su habilitación o último reinicio [41]. Para analizar el comportamiento dentro de una ventana temporal, interesa por tanto el incremento entre dos lecturas consecutivas y no únicamente el valor acumulado. En contraste, existen modos de reporte periódico que entregan directamente los conteos correspondientes a cada intervalo de observación, en lugar del valor acumulado [42].

El período de muestreo determina la resolución temporal de la observación. Intervalos más cortos permiten distinguir cambios de comportamiento con mayor detalle, pero requieren lecturas más frecuentes y pueden aumentar la sobrecarga de medición [42]. Intervalos más

largos reducen la frecuencia de observación, aunque pueden agrupar dentro de una misma ventana comportamientos distintos. Por esta razón, las tasas derivadas deben calcularse utilizando el tiempo realmente transcurrido entre lecturas, medido con una fuente temporal que no se vea afectada por ajustes del reloj del sistema [40].

1.1.4. Espacios de ejecución: *user-space* y *kernel-space*

Los sistemas operativos modernos emplean distintos niveles de privilegio para regular el acceso a los recursos del sistema. Los procesos en modo usuario operan con acceso restringido, mientras que el modo kernel permite acceder también a las regiones de memoria y recursos reservados al núcleo. Operaciones sensibles, como la gestión de memoria, la entrada/salida o la interacción directa con el hardware, requieren estos privilegios para proteger las estructuras críticas del sistema frente a procesos ordinarios [36, ch. 2]. En consecuencia, las aplicaciones acceden a dichos recursos mediante interfaces controladas por el sistema operativo, principalmente llamadas al sistema en entornos Unix y Linux [46]. Del mismo modo, cualquier mecanismo de monitoreo o control implementado en espacio de usuario debe apoyarse en interfaces que el kernel o la propia plataforma expongan explícitamente.

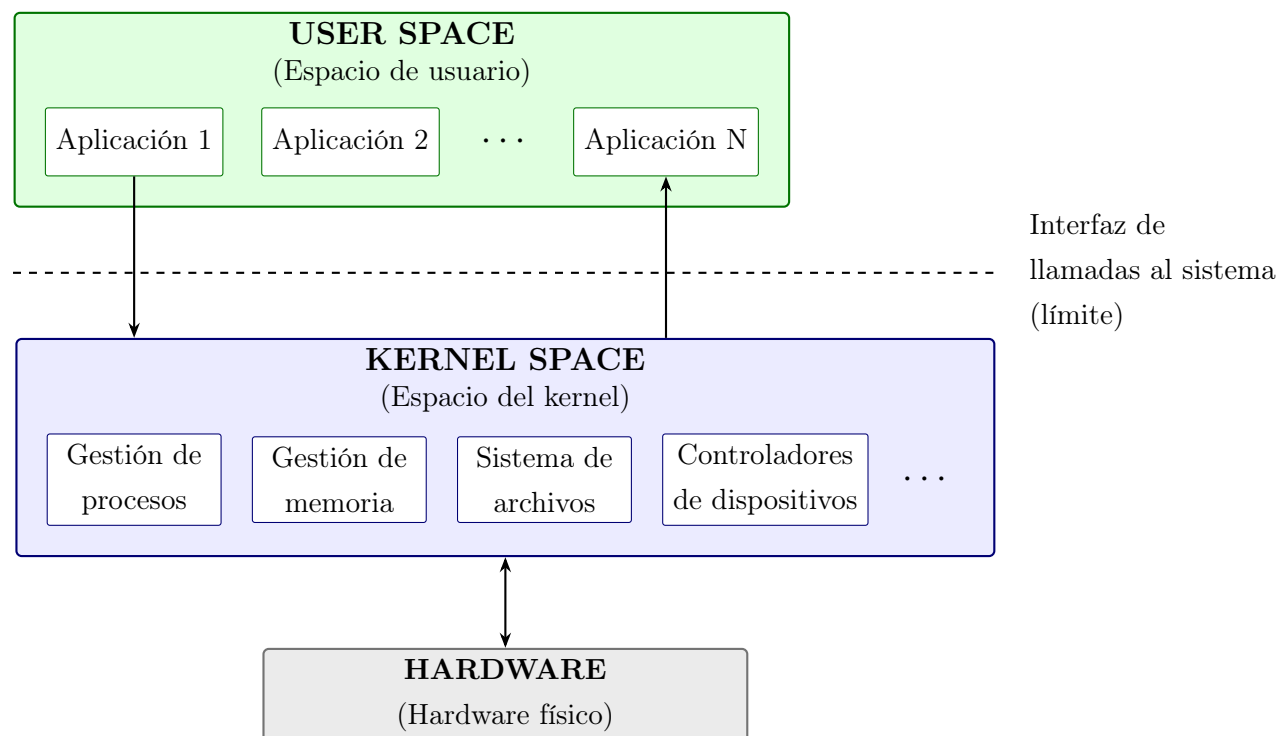


Figura 1.4: Relación conceptual entre *user-space*, *kernel-space* y el hardware físico, mediada por la interfaz de llamadas al sistema. Fuente: elaboración propia.

1.1.5. Interfaces estándar de observabilidad e instrumentación

La observabilidad de sistemas heterogéneos CPU–GPU depende de interfaces que expongan, desde *user-space*, métricas de rendimiento, utilización y consumo sin modificar el kernel ni los controladores. Las plataformas modernas exponen `perf_event` para contadores de CPU, Intel RAPL para energía de CPU, y NVIDIA Management Library (NVML) para telemetría de GPU [11, 59, 17, 50].

La interfaz `perf_event`

Linux proporciona la infraestructura `perf_event` para acceder desde espacio de usuario a los mecanismos de monitorización de rendimiento del procesador. A través de esta interfaz, el kernel configura los eventos de la PMU, administra su asignación sobre los contadores físicos disponibles y expone los conteos obtenidos a las herramientas de instrumentación [41]. De esta forma, herramientas como `perf` pueden observar eventos asociados a instrucciones, ciclos de ejecución, comportamiento de caché y otros aspectos microarquitectónicos sin acceder directamente a los registros de la PMU. En particular, el modo de reporte periódico introducido en la sección 1.1.3 corresponde al subcomando `perf stat`, que entrega los conteos de cada intervalo de observación en lugar del valor acumulado [42].

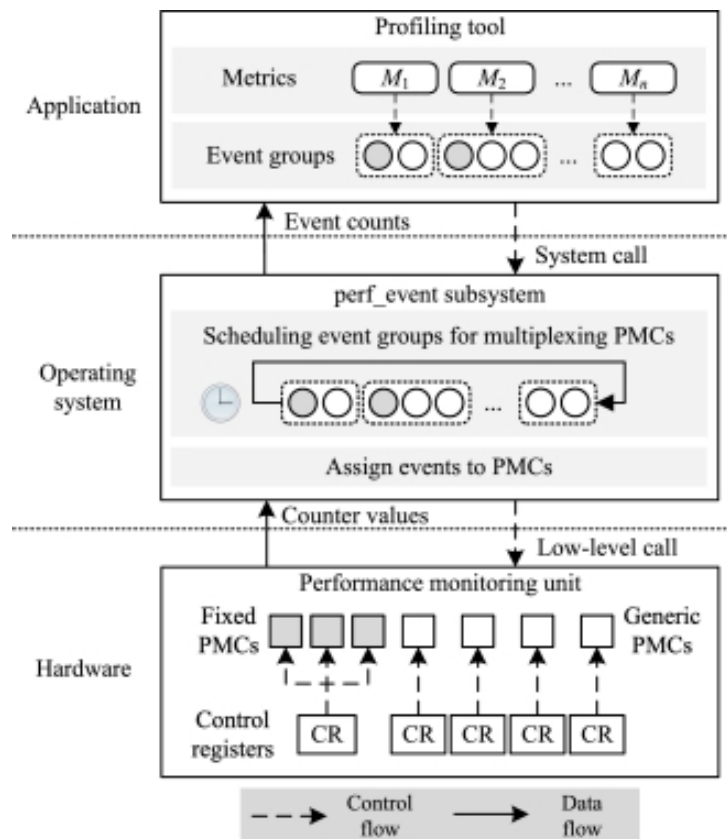


Figura 1.5: Flujo de configuración y lectura de eventos de rendimiento mediante el subsistema `perf_event` de Linux. Fuente: tomado de Liu et al. [43].

La interfaz permite además definir el alcance de una medición. Un evento puede asociarse a una tarea, siguiendo su ejecución aunque sea planificada sobre distintas CPU, o a una CPU determinada, contabilizando la actividad que ocurre en ese procesador durante el intervalo observado [41]. Esta distinción determina qué actividad contribuye al conteo y, por tanto, condiciona la interpretación de las métricas obtenidas.

Niveles de privilegio en el acceso a contadores

El acceso a los contadores de rendimiento desde espacio de usuario está regulado en Linux por el parámetro `perf_event_paranoid`, que establece restricciones para los procesos que no disponen de privilegios específicos de monitorización [41]. A medida que su valor aumenta, se restringe progresivamente el alcance de las mediciones: los niveles más restrictivos impiden la monitorización de alcance de CPU o sistema y pueden limitar la observación a la ejecución del propio proceso en espacio de usuario.

Esta restricción es especialmente relevante para las PMU de *uncore*, cuyos eventos representan actividad de componentes compartidos a nivel de paquete o socket y, por tanto, no

se atribuyen exclusivamente a un proceso individual (sección 1.1.3) [41]. En consecuencia, la disponibilidad efectiva de una métrica depende tanto de las capacidades de monitorización del hardware como de las políticas de acceso establecidas por el sistema operativo.

Intel RAPL

RAPL (*Running Average Power Limit*) es un mecanismo de gestión y monitorización energética incorporado en procesadores Intel que permite observar el consumo de energía mediante dominios asociados a distintos componentes del sistema. Entre los dominios disponibles pueden encontrarse el paquete completo del procesador (*Package*), los núcleos (*PP0/Core*) y la memoria DRAM, aunque su disponibilidad depende de la microarquitectura [17, 55].

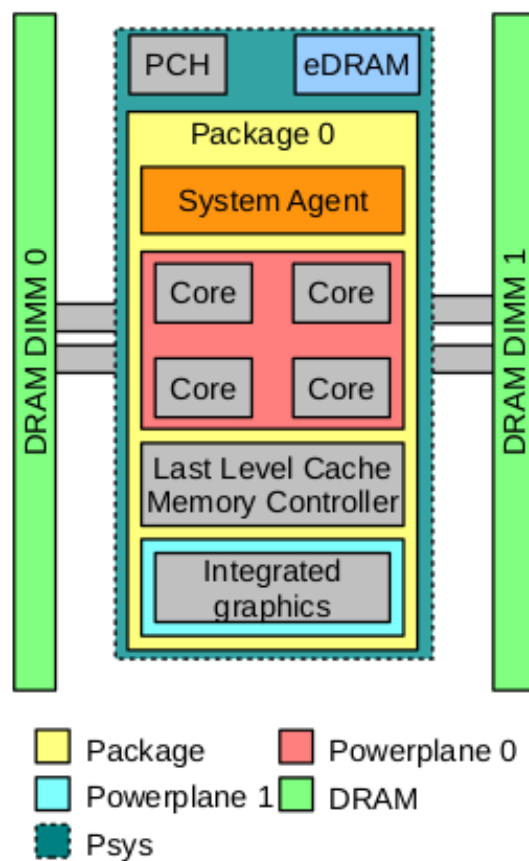


Figura 1.6: Dominios energéticos expuestos mediante Intel RAPL. Fuente: tomado de Thamm y Leser [55].

La información energética se mantiene mediante contadores acumulativos expuestos por registros específicos del modelo (MSR), cuyos valores deben interpretarse utilizando las unidades definidas por la plataforma [17, 55]. Debido a que estos contadores poseen un rango

finito, pueden presentar desbordamiento durante ejecuciones prolongadas, por lo que este comportamiento debe considerarse al calcular diferencias entre lecturas consecutivas. Asimismo, RAPL no representa necesariamente el consumo energético completo del nodo, sino únicamente los dominios que el procesador expone mediante esta infraestructura [55].

NVIDIA Management Library y Nsight Compute

NVIDIA Management Library (NVML) proporciona una interfaz para consultar el estado operativo de GPUs NVIDIA, incluyendo métricas de utilización, memoria, potencia y frecuencia [50]. Su información se obtiene a nivel del dispositivo y mediante períodos de muestreo definidos por el controlador. Por ejemplo, la utilización de GPU representa la fracción del intervalo durante la cual uno o más kernels estuvieron en ejecución, mientras que la utilización de memoria indica la fracción del intervalo durante la cual la memoria global fue leída o escrita [50]. Estas métricas describen actividad y estado operativo, pero no proporcionan directamente el número de operaciones aritméticas ni el volumen de datos transferidos necesarios para calcular la intensidad operacional del modelo Roofline.

Nsight Compute, en cambio, es un perfilador orientado al análisis detallado de kernels CUDA. La herramienta permite recolectar métricas de rendimiento asociadas al uso de unidades de cómputo, jerarquía de memoria, instrucciones y tráfico de datos mediante contadores de hardware e instrumentación del kernel [51]. Debido a que no todas las métricas pueden obtenerse simultáneamente, algunos perfiles requieren múltiples pasadas o repeticiones del lanzamiento analizado, lo que incrementa significativamente la sobrecarga de la medición [51].

NVML y Nsight Compute responden, por tanto, a niveles de observación distintos: la primera proporciona telemetría agregada del estado del dispositivo, mientras que el segundo permite caracterizar con mayor detalle el comportamiento de los kernels ejecutados. Esta diferencia determina el tipo de información que puede obtenerse de cada herramienta y su adecuación a distintos tipos de análisis.

1.1.6. Gestión dinámica de frecuencia en sistemas operativos: DVFS, P-states y *governors*

El escalado dinámico de voltaje y frecuencia (*Dynamic Voltage and Frequency Scaling*, DVFS) permite modificar el punto operativo del procesador para adaptar su capacidad de cómputo y consumo energético a las necesidades de la carga. Estas transiciones se apoyan en mecanismos hardware de gestión de potencia encargados de materializar los cambios solicitados por el sistema operativo [26]. Dentro del modelo ACPI, los estados de rendimiento

o P-states representan distintos niveles de capacidad mientras el procesador permanece activo en C0. P0 corresponde al mayor nivel de rendimiento, mientras que estados sucesivos reducen progresivamente dicha capacidad. Los C-states, en cambio, representan estados de inactividad con distintos compromisos entre ahorro energético y latencia de reactivación.

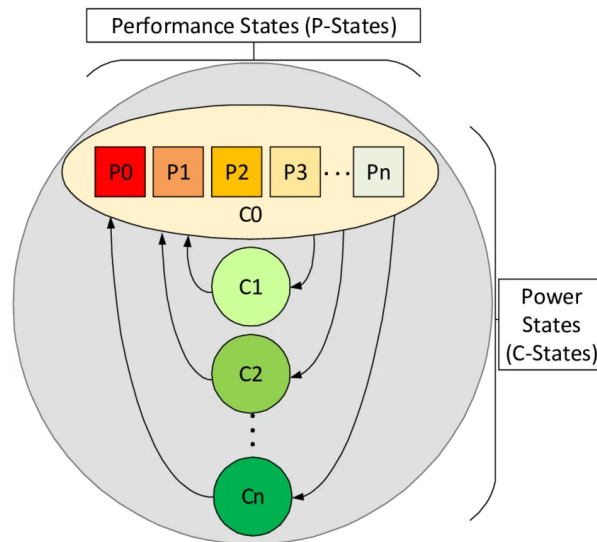


Figura 1.7: Relación conceptual entre estados de rendimiento (P-states) y estados de inactividad (C-states) del procesador. Fuente: tomado de Hebbar y Milenković [26].

Sobre estos mecanismos operan los *governors*, políticas software que determinan el nivel de rendimiento solicitado en función del estado del sistema. En el subsistema CPUFreq de Linux, los gobernadores genéricos **performance** y **powersave** solicitan respectivamente los límites superior e inferior de frecuencia disponibles, mientras que **ondemand**, **conservative** y **schedutil** modifican el nivel solicitado dinámicamente. El gobernador **userspace**, por su parte, permite que una aplicación en espacio de usuario solicite explícitamente la frecuencia de una política [26, 38].

La política de selección y el mecanismo encargado de actuar sobre el hardware son componentes distintos. Controladores como **acpi-cpufreq** operan junto con los gobernadores genéricos de CPUFreq, mientras que **intel_pstate** puede funcionar en modos activo y pasivo. En modo activo, **intel_pstate** utiliza sus propios algoritmos **performance** y **powersave**; este último realiza selección dinámica del nivel de rendimiento y no equivale al gobernador genérico **powersave**. En modo pasivo, el controlador se integra con los gobernadores genéricos de CPUFreq y se identifica como **intel_cpufreq** [39].

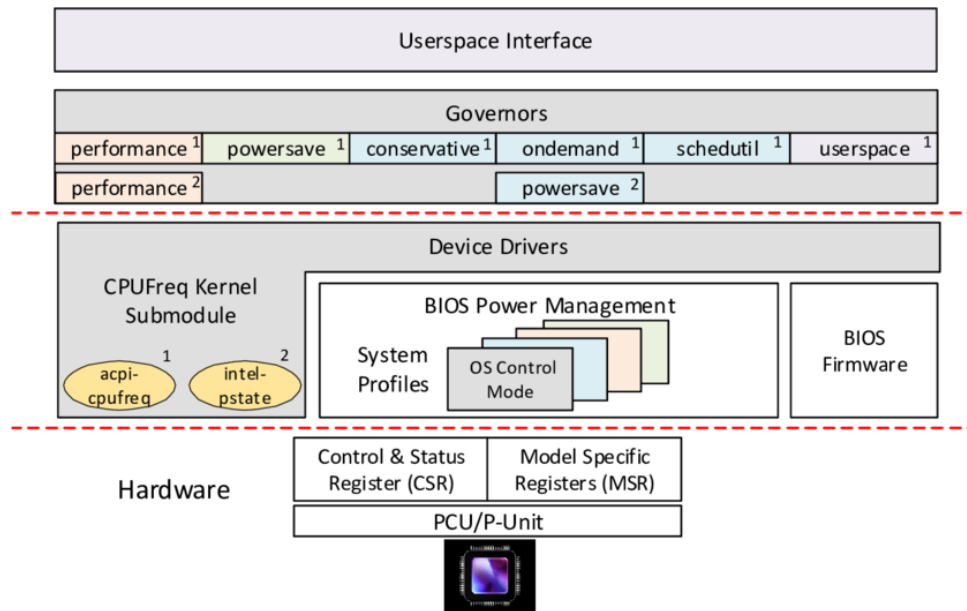


Figura 1.8: Jerarquía de los mecanismos de gestión de rendimiento desde la interfaz de espacio de usuario hasta el hardware. Fuente: tomado de Hebbar y Milenković [26].

Las transiciones entre niveles de rendimiento no son instantáneas y su tiempo de respuesta depende del procesador, del controlador utilizado y del grado de autonomía concedido al hardware. Mecanismos como los *Hardware-Managed P-states* (HWP) trasladan parte de la selección del estado operativo al propio procesador, reduciendo la intervención directa del sistema operativo [26]. En consecuencia, la granularidad temporal de una política DVFS debe considerar la capacidad efectiva de la plataforma para materializar sus decisiones.

Esa autonomía del hardware alcanza también al límite superior del rango. Los procesadores modernos incorporan mecanismos de incremento oportunista de frecuencia, conocidos en las plataformas Intel como Turbo Boost, que permiten superar la frecuencia base cuando las condiciones térmicas, de corriente y de número de núcleos activos lo admiten. Su activación depende del estado del procesador y no de una solicitud explícita del sistema operativo, por lo que el reloj efectivamente alcanzado puede no coincidir con el nivel solicitado a través de la interfaz de escalado [39, 26]. Esta distinción es determinante para cualquier experimento que compare niveles de frecuencia fijos: si el mecanismo permanece habilitado, distintos niveles solicitados pueden materializarse en relojes efectivos similares, y la diferencia de potencia atribuida al nivel quedaría contaminada. Por ello, el estado de este mecanismo debe fijarse explícitamente y la frecuencia efectivamente alcanzada debe verificarse por relectura durante la ejecución, en lugar de asumirse a partir del valor solicitado.

Dominios de frecuencia y topología SMT

El alcance de una solicitud de frecuencia no coincide necesariamente con la afinidad de ejecución de una carga. En Linux, el subsistema CPUFreq organiza el control mediante políticas asociadas a una o varias CPU lógicas, cuya composición depende del hardware y del controlador activo. Los atributos `affected_cpus` y `related_cpus` permiten identificar las CPU asociadas a cada política [38].

Esta distinción es especialmente relevante cuando está habilitado *simultaneous multithreading* (SMT). Dos procesadores lógicos hermanos comparten un mismo núcleo físico y, aunque puedan exponer límites de rendimiento independientes a nivel de software, su frecuencia efectiva no es necesariamente independiente. En particular, la documentación de `intel_pstate` establece que, con SMT habilitado, una solicitud de mayor rendimiento realizada por uno de los hermanos puede elevar el nivel efectivo alcanzado por el otro, incluso por encima del límite establecido en su política individual [39].

Por tanto, la colocación de la carga y el control de frecuencia deben tratarse como decisiones distintas. Restringir la ejecución a un solo procesador lógico por núcleo evita la contención entre hermanos SMT, pero no garantiza por sí solo el aislamiento del estado de frecuencia. Cuando se requiere controlar el punto operativo de un núcleo físico, deben considerarse también las solicitudes de rendimiento de sus procesadores lógicos hermanos y verificarse la frecuencia efectivamente alcanzada en la plataforma.

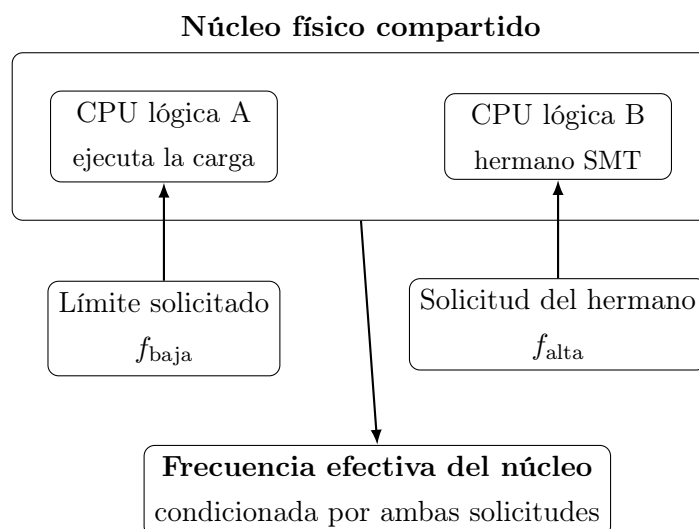


Figura 1.9: Relación conceptual entre las solicitudes de rendimiento de dos procesadores lógicos hermanos y la frecuencia efectiva de un núcleo físico con SMT habilitado. Fuente: elaboración propia con base en la documentación de `intel_pstate` [39].

Particularidades de DVFS en GPU

En las GPU, el control de frecuencia involucra dominios funcionales con sensibilidades distintas frente a la carga. De forma general, puede distinguirse un dominio de procesamiento, asociado principalmente a los multiprocesadores de *streaming* (SM), y un dominio de memoria, asociado a la memoria principal del dispositivo. La frecuencia de ambos subsistemas puede influir de manera diferente sobre el rendimiento y el consumo energético de una aplicación [25, 2].

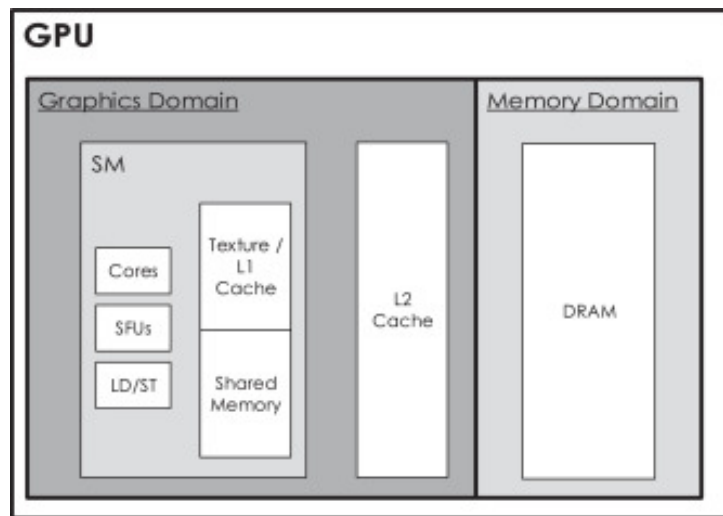


Figura 1.10: Representación conceptual de los dominios de procesamiento y memoria en una GPU. Fuente: tomado de Guerreiro et al. [25].

La sensibilidad a estos dominios depende del régimen de ejecución. En cargas limitadas por cómputo, cambios en la frecuencia del dominio de procesamiento tienden a afectar de forma más directa el tiempo de ejecución. En cargas limitadas por memoria, la influencia de esta frecuencia disminuye y adquiere mayor relevancia el comportamiento del subsistema de memoria [25, 2]. Esta relación también condiciona el consumo energético: una reducción de frecuencia puede disminuir la potencia instantánea, pero aumentar la energía total si el incremento del tiempo de ejecución compensa dicho ahorro [2].

A estas diferencias entre dominios se suma el costo temporal de modificar la frecuencia. En GPU, una transición requiere que la solicitud emitida desde el host sea recibida y aplicada por el acelerador, por lo que el cambio no es instantáneo. La latencia resultante depende de la arquitectura y del par de frecuencias involucrado, y puede alcanzar escalas de varios milisegundos [57]. Este costo limita la granularidad con la que puede aplicarse DVFS durante la ejecución, ya que cambios demasiado frecuentes pueden introducir una sobrecarga comparable al tiempo de ejecución de fases cortas.

1.1.7. Cargas de trabajo: kernels, benchmarks y microbenchmarks

Las aplicaciones de cómputo de alto rendimiento pueden combinar fases con demandas distintas sobre las unidades de cómputo, la jerarquía de memoria y otros recursos del sistema. Por esta razón, su comportamiento no siempre se describe adecuadamente mediante una única medida agregada de ejecución. El análisis puede realizarse sobre unidades más acotadas, denominadas kernels, que representan operaciones computacionales bien delimitadas, como una multiplicación de matrices, una transformada de Fourier o una actualización de *stencil*. Al concentrar un patrón definido de cómputo y acceso a datos, los kernels permiten estudiar con mayor claridad la relación entre intensidad operacional, ancho de banda de memoria y capacidad de cómputo [61].

Un benchmark es una carga utilizada para medir y comparar el comportamiento de un sistema bajo condiciones reproducibles. Dependiendo de su propósito, puede representar una aplicación completa, una parte significativa de ella o un conjunto de operaciones representativas. Su utilidad radica en proporcionar una base común para comparar arquitecturas, configuraciones o mecanismos de gestión del sistema [24, Ch. 12].

Un microbenchmark persigue un objetivo distinto: aislar una propiedad concreta del hardware o del software mediante una carga deliberadamente simple y controlada. Entre sus usos habituales se encuentran la medición del ancho de banda de memoria, la latencia de acceso o la capacidad de ejecución aritmética [24, Ch. 12]. En consecuencia, los benchmarks buscan representar comportamientos relevantes de una carga, mientras que los microbenchmarks permiten caracterizar de forma aislada mecanismos específicos de la plataforma.

1.1.8. Aprendizaje automático supervisado ligero

El aprendizaje supervisado busca estimar una función que relacione un conjunto de variables de entrada con una salida a partir de observaciones previamente etiquetadas. El modelo se ajusta utilizando datos de entrenamiento y su capacidad de generalización se evalúa sobre observaciones que no participaron en dicho ajuste [34, Ch. 2].

En el contexto de este trabajo, el término *ligero* se utiliza para describir modelos cuya inferencia puede ejecutarse con baja latencia y un costo computacional acotado, de modo que su incorporación a un bucle de control no introduzca una sobrecarga comparable al beneficio que se pretende obtener. Esta propiedad no depende únicamente de la familia del modelo, sino también de su complejidad efectiva, determinada por factores como la profundidad de los árboles y el número de estimadores del ensamble.

Familias de modelos candidatas

Regresión logística. La regresión logística modela la probabilidad de pertenencia a una clase binaria a partir de una combinación lineal de las características. Para un vector de entrada x , se define

$$z = \beta_0 + \beta_1 x_1 + \cdots + \beta_n x_n, \quad P(y = 1 \mid x) = \frac{1}{1 + e^{-z}}. \quad (1.5)$$

La transformación sigmoide restringe la salida al intervalo $[0, 1]$. Su estructura proporciona un modelo de baja complejidad e interpretación directa, aunque produce una frontera de decisión lineal en el espacio original de características, lo que limita su capacidad para representar separaciones fuertemente no lineales [34, Ch. 4].

Árbol de decisión. Un árbol de decisión particiona recursivamente el espacio de características mediante reglas asociadas a variables individuales. Cada observación recorre una secuencia de divisiones desde la raíz hasta una hoja, donde se determina la clase predicha. Esta estructura permite representar fronteras de decisión no lineales y facilita la inspección de las reglas aprendidas. Sin embargo, árboles de elevada profundidad pueden presentar alta varianza y ajustarse en exceso a los datos de entrenamiento [34, Ch. 8].

Bosques aleatorios (*Random Forest*). Random Forest combina múltiples árboles de decisión mediante un esquema de ensamble. Cada árbol se entrena sobre una muestra *bootstrap* del conjunto de entrenamiento y, durante la construcción de sus divisiones, solo se considera un subconjunto aleatorio de características. Estos mecanismos introducen diversidad entre los árboles y reducen su correlación. En clasificación, la predicción del bosque se obtiene agregando las decisiones individuales de los árboles, habitualmente mediante votación mayoritaria [9].

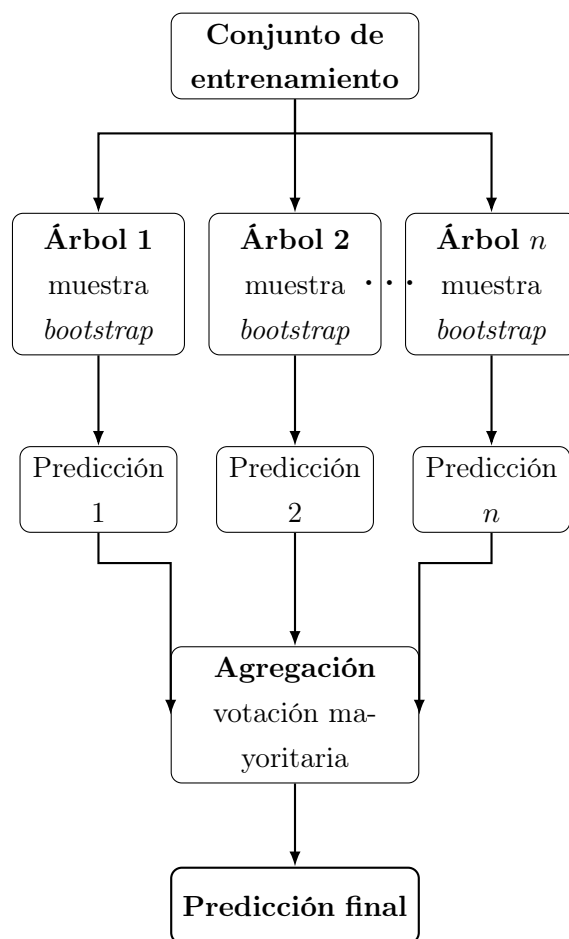


Figura 1.11: Esquema conceptual del funcionamiento de un bosque aleatorio (*Random Forest*). Los árboles se construyen a partir de muestras *bootstrap*, incorporando además selección aleatoria de características durante las divisiones, y sus predicciones se agregan para obtener la decisión final. Fuente: Adaptado de IBM [28].

Árboles extremadamente aleatorizados (*Extra-Trees*). Extra-Trees es un método de ensamble de árboles que introduce un grado adicional de aleatorización durante la construcción de las divisiones. En cada nodo se selecciona aleatoriamente un subconjunto de características y se genera un punto de corte aleatorio para cada característica candidata; posteriormente se elige la mejor división entre esos candidatos aleatorios. En su formulación original, los árboles se construyen utilizando el conjunto de entrenamiento completo, en lugar de muestras *bootstrap* independientes [21].

La aleatorización de los puntos de corte modifica el compromiso entre sesgo y varianza respecto a métodos que optimizan cada división y reduce el costo de construcción de los árboles. El efecto concreto depende del grado de aleatorización, de la configuración del ensamble y de las características de los datos [21].

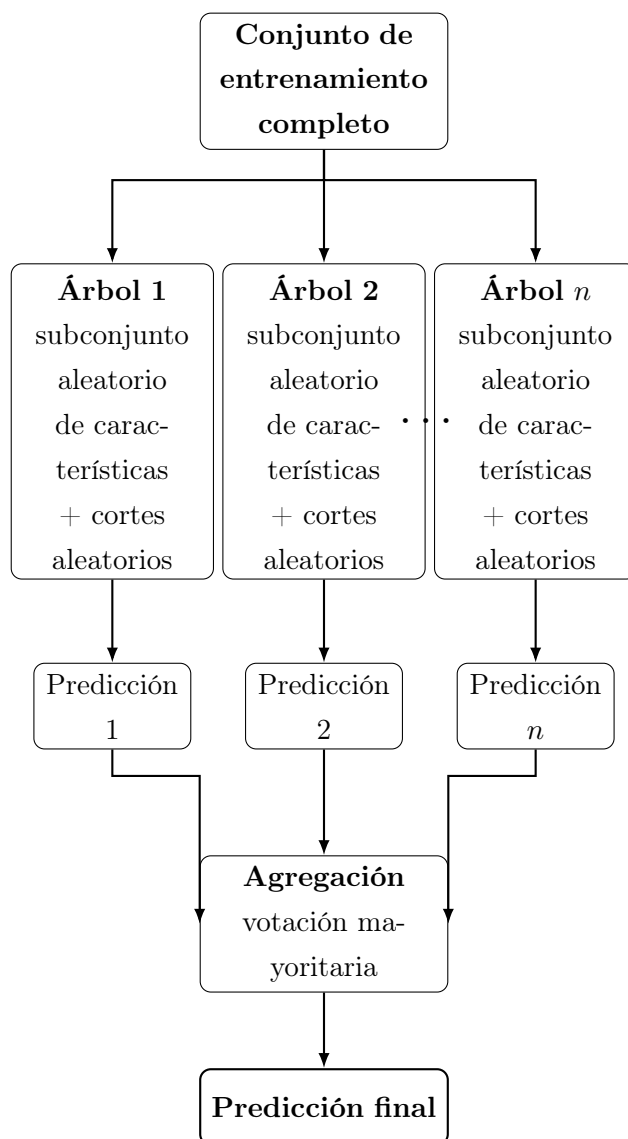


Figura 1.12: Esquema conceptual de un ensemble de árboles extremadamente aleatorizados (*Extra-Trees*). En su formulación original, los árboles utilizan el conjunto de entrenamiento completo e introducen aleatoriedad tanto en la selección de características como en los puntos de corte considerados durante las divisiones. Fuente: elaboración propia con base en Geurts et al. [21].

XGBoost. XGBoost implementa un ensemble aditivo de árboles construido de forma secuencial. En cada iteración se incorpora un nuevo árbol con el propósito de reducir la función objetivo del modelo acumulado, utilizando información de primer y segundo orden de la función de pérdida. La formulación incorpora además términos de regularización destinados a controlar la complejidad de los árboles [14]. Esta estructura permite representar relaciones no lineales e interacciones entre características, aunque su costo de inferencia depende del número y la profundidad de los árboles que conforman el ensemble.

Las cinco familias consideradas presentan compromisos diferentes entre capacidad de representación, interpretabilidad y costo de inferencia. La regresión logística proporciona una referencia lineal; el árbol de decisión introduce una frontera no lineal directamente inspeccionable; Random Forest y Extra-Trees combinan múltiples árboles mediante estrategias distintas de aleatorización y agregación; y XGBoost construye el ensamble de forma secuencial mediante potenciación. La complejidad efectiva de cada modelo depende además de sus hiperparámetros, por lo que su comparación debe considerar tanto el desempeño predictivo como el costo del modelo resultante.

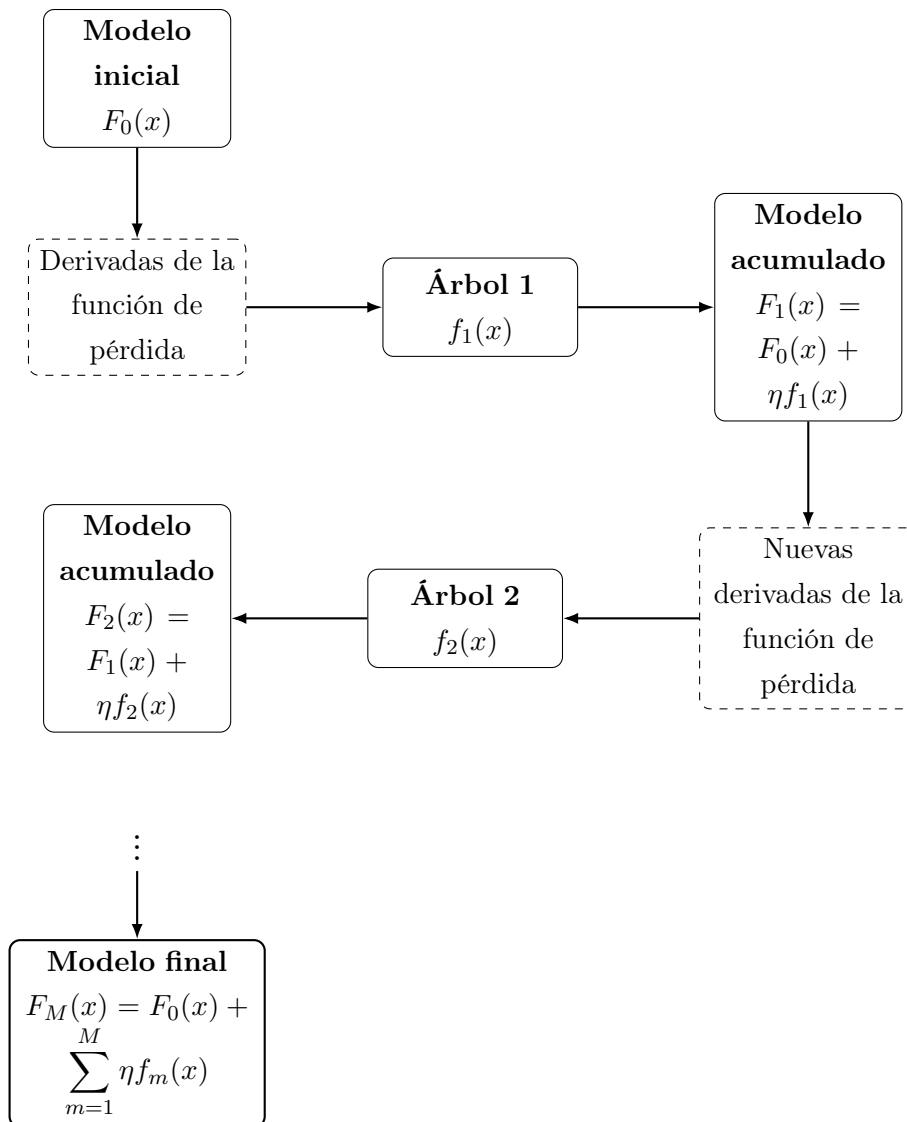


Figura 1.13: Esquema conceptual de la construcción secuencial y aditiva de un modelo XGBoost. Cada nuevo árbol se obtiene a partir de la información de la función de pérdida del modelo acumulado y su contribución se incorpora a la predicción existente. Fuente: elaboración propia con base en Chen y Guestrin [14].

1.1.9. Evaluación del desempeño de un clasificador

La matriz de confusión relaciona las clases reales con las clases predichas y permite distinguir, en clasificación binaria, verdaderos positivos (TP), verdaderos negativos (TN), falsos positivos (FP) y falsos negativos (FN). A partir de estos valores pueden derivarse distintas métricas, cada una sensible a aspectos diferentes del comportamiento del clasificador [54].

La exactitud (*accuracy*) expresa la proporción total de predicciones correctas:

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}. \quad (1.6)$$

Aunque proporciona una medida directa de la tasa global de aciertos, puede resultar poco informativa cuando existe un desequilibrio importante entre las clases, ya que una elevada proporción de ejemplos de la clase mayoritaria puede dominar el resultado [10]. Una alternativa es la exactitud balanceada (*balanced accuracy*), que en clasificación binaria corresponde al promedio entre la sensibilidad y la especificidad:

$$\text{BalancedAccuracy} = \frac{1}{2} \left(\frac{TP}{TP + FN} + \frac{TN}{TN + FP} \right). \quad (1.7)$$

La precisión y la sensibilidad se definen, respectivamente, como

$$\text{Precision} = \frac{TP}{TP + FP}, \quad \text{Recall} = \frac{TP}{TP + FN}. \quad (1.8)$$

A partir de ellas, el puntaje F_1 se obtiene mediante su media armónica:

$$F_1 = 2 \frac{\text{Precision} \text{ Recall}}{\text{Precision} + \text{Recall}}. \quad (1.9)$$

Esta métrica permite considerar conjuntamente ambos aspectos del desempeño sobre una clase de interés, sin sustituir por ello a las demás métricas de evaluación [54].

Precisión, sensibilidad y F_1 se definen respecto a una clase tomada como positiva, de modo que su valor cambia según cuál de las dos clases se elija. Reportar un único valor global oculta esa asimetría y puede enmascarar un desempeño pobre sobre la clase menos representada. Por ello conviene reportar las tres métricas por separado para cada clase y, cuando se requiera un resumen único, emplear el promedio macro, que promedia el valor obtenido en cada clase otorgándoles el mismo peso con independencia de su frecuencia [54].

Cuando las observaciones se agrupan en unidades experimentales heterogéneas, el mismo argumento se extiende a esas unidades: un promedio sobre todas las observaciones puede quedar dominado por los grupos más numerosos y ocultar un comportamiento distinto en el resto.

Otra medida derivada de los cuatro componentes de la matriz de confusión es el coeficiente de correlación de Matthews (MCC):

$$\text{MCC} = \frac{TP \cdot TN - FP \cdot FN}{\sqrt{(TP + FP)(TP + FN)(TN + FP)(TN + FN)}}. \quad (1.10)$$

El MCC toma valores entre -1 y 1 , donde 1 representa concordancia perfecta entre las clases reales y predichas, 0 indica ausencia de correlación y -1 corresponde a una discordancia perfecta. Al incorporar simultáneamente los cuatro componentes de la matriz de confusión, constituye una medida complementaria para evaluar clasificadores binarios, especialmente cuando la distribución de las clases no es uniforme [15].

Líneas base de referencia

Ninguna de las métricas anteriores es interpretable en términos absolutos sin una referencia con la cual compararla. Un clasificador trivial que predice siempre la clase mayoritaria alcanza una exactitud igual a la frecuencia de esa clase y puede incluso obtener un F_1 elevado sobre ella, sin haber aprendido ninguna relación entre las características y la etiqueta [15]. Por esta razón, el desempeño de un modelo se contrasta frente a líneas base sin capacidad predictiva, habitualmente la predicción constante de la clase mayoritaria y la asignación aleatoria que respeta las proporciones observadas.

La comparación debe realizarse bajo el mismo esquema de partición que el modelo evaluado, de modo que ambas estimaciones estén sujetas a las mismas condiciones. Solo una mejora apreciable y sostenida sobre esas referencias permite atribuir el resultado a información contenida en las características. Cuando un modelo no supera su línea base, la conclusión pertinente es que las señales disponibles no bastan para separar las clases en ese contexto de observación, lo que constituye un resultado informativo por sí mismo y no un fallo del procedimiento de evaluación.

Costo de inferencia como dimensión de evaluación

Cuando un clasificador se destina a operar dentro de un lazo de decisión sujeto a un presupuesto temporal, su costo de inferencia deja de ser un detalle de implementación y pasa

a ser una dimensión de evaluación equiparable a la calidad predictiva. La magnitud relevante es la latencia por predicción individual, ya que el lazo decide sobre una observación a la vez y no sobre lotes.

Esta latencia depende de la microarquitectura, de la implementación empleada y de la complejidad efectiva del modelo entrenado, entre otros factores el número y la profundidad de los árboles de un ensamble. En consecuencia, no puede estimarse analíticamente a partir de la familia del modelo ni extrapolarse desde otra plataforma: debe medirse sobre el mismo hardware en el que operará.

Reportar únicamente la latencia media es insuficiente para un componente sujeto a un plazo. Lo que condiciona la viabilidad es el peor caso acotado, porque una predicción ocasionalmente lenta puede exceder la ventana de decisión aunque el promedio se mantenga holgado. Por ello la caracterización se expresa mediante percentiles altos de la distribución, por ejemplo los percentiles 95 y 99, y no solo mediante su tendencia central, siguiendo el principio de evaluación estadísticamente rigurosa del desempeño de un sistema [20]. Un modelo cuya latencia en la cola de la distribución compromete el plazo del lazo de control no resulta admisible aunque su exactitud sea superior, de modo que la elección final se plantea como un compromiso entre dos criterios en conflicto (sección 1.1.10).

1.1.10. Hiperparámetros y selección multiobjetivo

Los parámetros internos de un modelo se estiman durante el ajuste a partir de los datos de entrenamiento. Los hiperparámetros, en cambio, controlan la estructura del modelo o el propio procedimiento de ajuste y deben fijarse antes de este, por lo que no se obtienen del mismo proceso de estimación. Son ejemplos la fuerza de regularización de la regresión logística, la profundidad máxima de un árbol, el número de estimadores de un ensamble o la tasa de aprendizaje de un modelo potenciado. Su elección modifica simultáneamente la capacidad predictiva y la complejidad efectiva del modelo resultante [8].

La configuración se selecciona evaluando distintas combinaciones sobre datos reservados para ese fin. Entre las estrategias habituales se encuentran la búsqueda exhaustiva sobre una rejilla de valores predefinidos y el muestreo aleatorio del espacio de configuración; esta última resulta preferible cuando solo una fracción de los hiperparámetros influye de forma apreciable sobre el resultado, ya que dedica el mismo presupuesto de evaluaciones a explorar más valores distintos de las dimensiones relevantes [8]. En todos los casos, la comparación exige que la estimación final del desempeño se realice sobre datos que no intervinieron en esta selección, condición que motiva el procedimiento descrito en la sección 1.1.11.

Cuando la elección debe atender simultáneamente a varios criterios en conflicto, como la calidad predictiva y el costo de inferencia de la sección 1.1.9, ninguna métrica escalar basta por sí sola para ordenar las alternativas. La comparación se formaliza entonces mediante la relación de dominancia: una configuración domina a otra si no es peor en ninguno de los criterios considerados y es estrictamente mejor en al menos uno. Las configuraciones que no son dominadas por ninguna otra conforman el frente de Pareto, que representa el conjunto de compromisos alcanzables entre los objetivos [18].

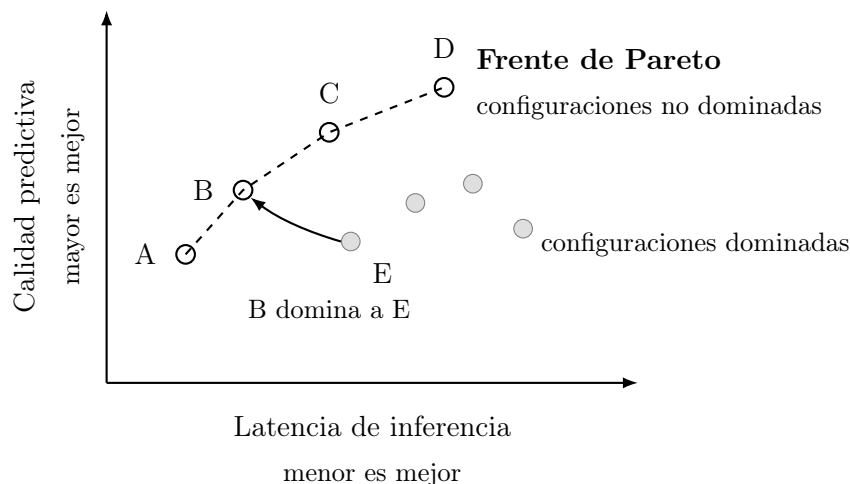


Figura 1.14: Representación conceptual del frente de Pareto para dos objetivos en conflicto: maximizar la calidad predictiva y minimizar la latencia de inferencia. Cada punto representa una configuración de hiperparámetros. Las configuraciones A–D no son dominadas, mientras que E es dominada por B al presentar simultáneamente mayor latencia y menor calidad predictiva. Fuente: elaboración propia con base en Deb et al. [18].

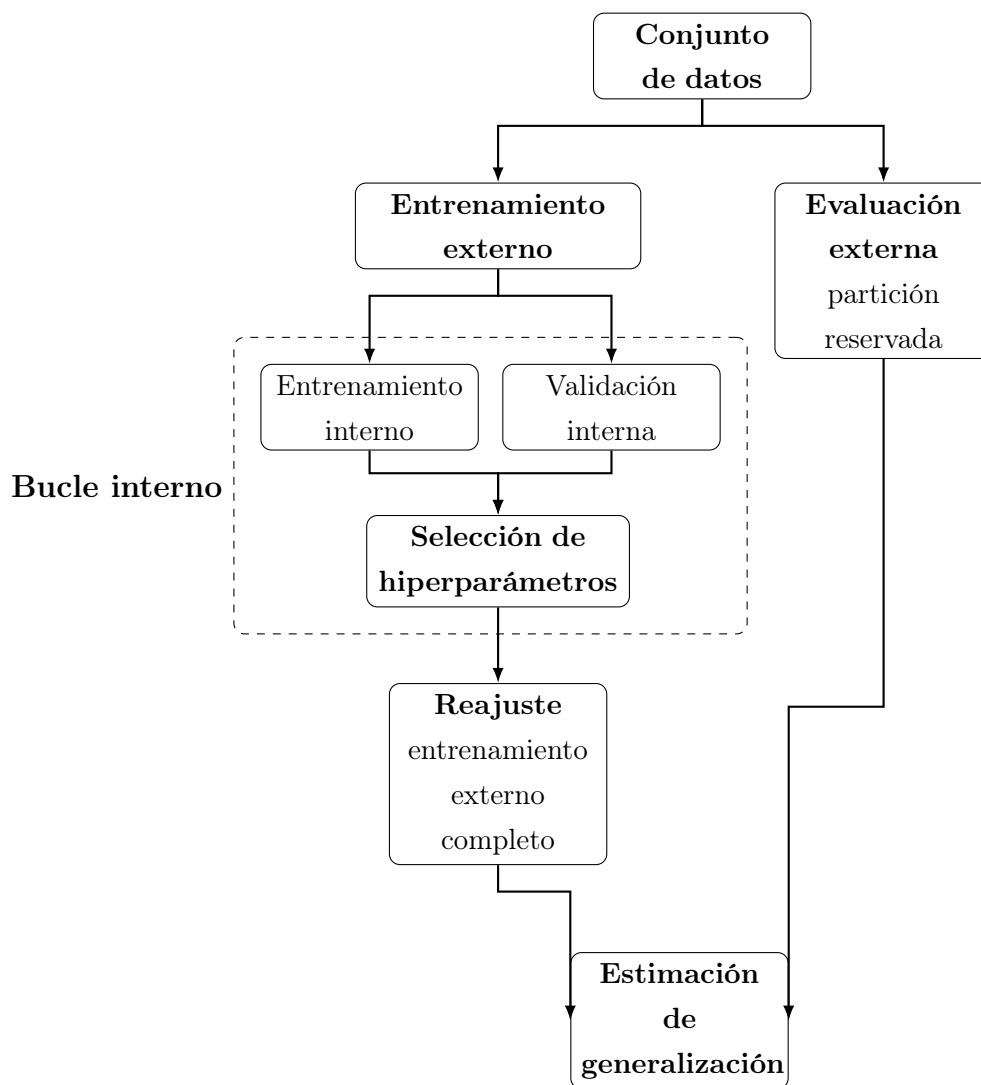
El frente delimita las opciones defendibles, pero no designa una entre ellas: seleccionar un único modelo requiere un criterio adicional, ya sea la combinación de los objetivos en una función escalar con pesos declarados explícitamente, o la imposición de una restricción sobre uno de ellos, por ejemplo un límite superior de latencia admisible por el lazo de control, para optimizar el restante dentro de esa frontera. El mismo razonamiento no se limita a las configuraciones de un modelo: permite también comparar familias distintas entre sí, al decidir si la ganancia predictiva de un ensamble más costoso justifica su mayor latencia frente a una alternativa más simple.

1.1.11. Validación anidada y fuga de información

La optimización de hiperparámetros introduce una etapa de selección adicional al entrenamiento del modelo. Si las configuraciones se seleccionan utilizando la misma información

sobre la que posteriormente se estima su desempeño, la evaluación puede presentar un sesgo optimista, ya que el proceso de selección se adapta indirectamente a esos datos [12].

La validación cruzada anidada separa ambos procesos mediante dos niveles. El bucle externo reserva observaciones para estimar la capacidad de generalización, mientras que el bucle interno utiliza únicamente los datos de entrenamiento correspondientes para seleccionar los hiperparámetros. Una vez elegida la configuración, el modelo se reajusta con los datos disponibles en el entrenamiento externo y se evalúa sobre la partición externa que no participó en su selección [12]. Esta separación se representa conceptualmente en la Figura 1.15.



El procedimiento se repite para cada partición externa

Figura 1.15: Esquema conceptual de la validación cruzada anidada. La selección de hiperparámetros se realiza únicamente dentro del conjunto de entrenamiento externo, mientras que la partición externa reservada se utiliza exclusivamente para estimar la capacidad de generalización. Fuente: elaboración propia con base en Cawley y Talbot [12].

La fuga de información constituye un problema diferente. Ocurre cuando las características utilizadas por el modelo contienen información sobre la variable objetivo que no estaría legítimamente disponible en el momento de realizar una predicción [35]. Por ejemplo, si una etiqueta se construye a partir de una determinada magnitud, emplear esa misma magnitud o una variable que la revele directamente como característica de entrada permitiría al clasificador acceder de forma indirecta al criterio que pretende inferir.

La independencia entre las particiones también debe considerar la estructura de los da-

tos. Cuando varias observaciones proceden de una misma unidad o de grupos estrechamente relacionados, distribuirlos entre entrenamiento y evaluación puede producir estimaciones excesivamente optimistas. En estos casos, la partición debe realizarse a nivel de grupo, de modo que las observaciones relacionadas permanezcan juntas y la evaluación refleje la generalización hacia grupos no observados [52]. Esta restricción debe preservarse también dentro del proceso de selección de hiperparámetros cuando se emplea validación anidada.

1.1.12. Comparaciones estadísticas pareadas

Cuando dos condiciones se evalúan sobre las mismas unidades experimentales, las observaciones correspondientes forman pares y no deben tratarse como muestras independientes. El análisis se realiza entonces sobre las diferencias observadas para cada unidad, permitiendo evaluar si existe un efecto sistemático asociado a la condición comparada [47, Ch. 2].

La prueba t pareada contrasta si la media de estas diferencias es estadísticamente distinguible de cero y supone una distribución aproximadamente normal de las diferencias. Cuando este supuesto no resulta adecuado, la prueba de rangos con signo de Wilcoxon constituye una alternativa no paramétrica que utiliza el signo y el orden de las diferencias en lugar de depender directamente de sus magnitudes [27, Ch. 3]. El valor p obtenido mediante una prueba de hipótesis permite cuantificar la evidencia contra la ausencia de una diferencia sistemática, pero no expresa por sí mismo la magnitud del efecto observado.

El remuestreo *bootstrap* permite complementar este análisis mediante la estimación de intervalos de confianza para el estadístico de interés. A partir de remuestreos con reemplazo de las observaciones disponibles se obtiene una distribución empírica que permite caracterizar la incertidumbre asociada al efecto estimado sin imponer una distribución paramétrica específica [19, Chs. 12–14].

La capacidad para detectar una diferencia depende del número de observaciones y de su variabilidad. Por esta razón, un resultado no significativo indica falta de evidencia suficiente para rechazar la hipótesis nula, pero no demuestra que las condiciones comparadas sean equivalentes.

1.1.13. Producto Energía–Retardo (EDP)

La evaluación de una estrategia de gestión energética debe considerar conjuntamente el consumo de energía y el tiempo de ejecución, ya que una reducción de potencia puede acompañarse de un incremento temporal suficiente para anular el beneficio energético. El Producto Energía–Retardo (*Energy–Delay Product*, EDP) combina ambas magnitudes en

una única métrica [37]:

$$EDP = E \cdot T, \quad (1.11)$$

donde E corresponde a la energía consumida y T al tiempo de ejecución. Un menor valor de EDP representa un compromiso más favorable entre ambas magnitudes bajo este criterio.

La formulación admite una generalización que pondera explícitamente el peso relativo del tiempo frente a la energía:

$$ED^nP = E \cdot T^n, \quad (1.12)$$

donde n expresa cuánto se penaliza la degradación temporal. Con $n = 1$ se recupera el EDP, mientras que $n = 2$ define el ED^2P , empleado cuando el rendimiento debe preservarse con mayor prioridad que el consumo [37, 2]. La elección de n es una decisión del evaluador y no una propiedad del sistema medido: dos valores distintos pueden ordenar de forma diferente las mismas configuraciones, por lo que debe declararse explícitamente antes de comparar resultados.

El EDP resulta especialmente útil en la evaluación de técnicas DVFS, donde la reducción de frecuencia puede disminuir la potencia instantánea pero aumentar el tiempo de ejecución. Por ello, evaluar únicamente potencia, energía o rendimiento de forma aislada puede conducir a conclusiones distintas de las obtenidas al considerar conjuntamente el costo energético y temporal [2, 37].

1.1.14. Síntesis: alcance y límites del marco adoptado

Los conceptos anteriores convergen en un compromiso metodológico concreto. La etiqueta de régimen procede del modelo Roofline y exige una medida directa de trabajo aritmético y de tráfico hacia memoria principal, disponible fuera de línea pero costosa o inaccesible durante la ejecución. Las señales que sí pueden leerse con baja intrusión describen actividad y no intensidad operacional, de modo que el clasificador se define precisamente como el sustituto que traduce las segundas en la primera. Esta separación entre la fuente de la verdad y la fuente de la predicción es la decisión de diseño central del trabajo, y de ella se desprenden las salvaguardas desarrolladas en las secciones previas: la exclusión de la magnitud que origina la etiqueta del vector de entrada, la partición por unidad experimental y no por observación individual, y la comparación contra líneas base sin capacidad predictiva.

Este marco delimita también lo que el trabajo no puede afirmar. La clasificación es binaria y estática respecto a un umbral calibrado, por lo que no describe fases intermedias ni transiciones continuas entre regímenes. La observabilidad disponible es asimétrica entre dispositivos, de manera que la misma pregunta no admite el mismo grado de evidencia en CPU y en GPU. Y el mecanismo de actuación tiene un costo temporal propio que acota la granularidad útil de cualquier política, con independencia de la calidad del clasificador. En consecuencia, un resultado que no muestre mejora sobre las líneas base o sobre los gobernadores nativos no invalidaría el marco: sería la respuesta que este marco permite dar sobre los límites de la telemetría y del mecanismo de control disponibles en la plataforma evaluada.

1.2. Estado del Arte

La gestión energética en sistemas de alto rendimiento ha evolucionado desde enfoques estáticos de reducción de potencia hacia estrategias cada vez más adaptativas, orientadas a preservar el rendimiento sin incurrir en costos energéticos desproporcionados. Dentro de este proceso, el escalado dinámico de voltaje y frecuencia (DVFS) se ha consolidado como uno de los mecanismos más estudiados para modificar el punto operativo de procesadores y aceleradores. Su aplicación, sin embargo, no conduce necesariamente al mismo resultado para todas las cargas de trabajo. La reducción de frecuencia puede disminuir la potencia instantánea, pero también prolongar el tiempo de ejecución y anular el beneficio energético esperado. Por esta razón, una parte importante de la investigación en este campo se ha desplazado desde la aplicación uniforme de DVFS hacia mecanismos capaces de reconocer el comportamiento de la carga y seleccionar configuraciones acordes con sus características.

Una de las primeras evidencias de esta dependencia fue presentada por Calore et al. [11], quienes estudiaron el efecto de DVFS sobre una CPU Intel Haswell y una GPU NVIDIA K80 mediante distintos kernels y una aplicación HPC completa. Los autores analizaron conjuntamente el tiempo y la energía de ejecución, y mostraron que una misma frecuencia no resulta igualmente conveniente para todas las regiones de una aplicación. En lugar de mantener un único estado operativo durante toda la ejecución, evaluaron cambios de frecuencia a nivel de función, lo que permitió observar que las oportunidades de ahorro dependen tanto de la naturaleza de la región ejecutada como de la arquitectura sobre la cual se realiza el cálculo. Este resultado introduce una idea central para los mecanismos de gestión adaptativa: la decisión de frecuencia debe responder al comportamiento de la carga y no únicamente a una política global fijada de antemano.

Aunque esta aproximación permite aprovechar diferencias entre regiones de ejecución,

requiere conocimiento previo sobre la estructura de la aplicación y puntos específicos en los cuales realizar la actuación. El problema se desplaza entonces hacia la posibilidad de caracterizar automáticamente una carga a partir de información observable durante su ejecución. En este sentido, Guerreiro et al. [25] estudiaron la relación entre el comportamiento de aplicaciones GPGPU y las variaciones de frecuencia de los dominios de cómputo y memoria. Su propuesta parte de métricas recolectadas en una configuración inicial y utiliza esa caracterización para anticipar la respuesta de la aplicación frente a otras configuraciones DVFS. De esta manera, los autores reducen la necesidad de recorrer exhaustivamente el espacio de frecuencias y muestran que es posible utilizar información de rendimiento para agrupar aplicaciones según su sensibilidad a DVFS y seleccionar configuraciones energéticamente favorables.

La propuesta de Guerreiro et al. [25] resulta particularmente significativa porque introduce la clasificación como un mecanismo intermedio entre la observación de la carga y la decisión energética. No obstante, las clases definidas en su trabajo responden principalmente a la sensibilidad de las aplicaciones frente a modificaciones en las frecuencias de cómputo y memoria, y no a una detección continua de fases *compute-bound* y *memory-bound* durante la ejecución. Su objetivo consiste en utilizar una caracterización limitada para inferir el comportamiento esperado bajo distintas configuraciones, manteniendo además el análisis concentrado en GPU. Aun así, el estudio demuestra que las métricas obtenidas del hardware contienen información suficiente para reducir el espacio de búsqueda de una configuración energética adecuada.

La misma problemática de evitar una exploración exhaustiva fue abordada posteriormente por Ali et al. [2]. Los autores propusieron un método automatizado y portable para seleccionar frecuencias de GPU a partir de métricas obtenidas inicialmente en la configuración predeterminada del dispositivo. Sobre esta caracterización construyen modelos de potencia y tiempo de ejecución, con los cuales estiman la respuesta de la aplicación en otros estados DVFS sin necesidad de ejecutarla en cada uno de ellos. La selección final se realiza considerando simultáneamente consumo energético y degradación del desempeño, de modo que una reducción de potencia no sea interpretada automáticamente como una mejora si viene acompañada de un incremento excesivo en el tiempo de ejecución. En las plataformas evaluadas, la propuesta alcanzó reducciones promedio de energía de 29.6 % con una pérdida de rendimiento de 5.2 % sobre NVIDIA GA100 y de 22.6 % con una pérdida de 4.7 % sobre NVIDIA GV100 [2].

En conjunto, los trabajos de Guerreiro et al. [25] y Ali et al. [2] muestran que el espacio DVFS puede reducirse mediante modelos contruidos a partir del comportamiento de la aplicación. Sin embargo, ambos enfoques parten principalmente de una caracterización utilizada

para predecir configuraciones posteriores. Esta estrategia es adecuada cuando la aplicación puede tratarse como una carga relativamente estable, pero resulta más limitada cuando su comportamiento cambia durante la ejecución. Las aplicaciones científicas suelen combinar regiones con demandas diferentes sobre las unidades de cómputo y el subsistema de memoria; por tanto, una política verdaderamente adaptativa requiere no solo estimar qué frecuencia puede resultar adecuada, sino también reconocer cuándo cambia el régimen de ejecución.

La clasificación de este comportamiento constituye precisamente el problema abordado por Antici et al. [4] mediante MCBound. Los autores plantean la distinción entre cargas *memory-bound* y *compute-bound* como un problema de clasificación basado en datos, utilizando información de ejecuciones HPC para caracterizar automáticamente nuevos trabajos. Su evaluación sobre datos del supercomputador Fugaku mostró que esta separación puede realizarse con una puntuación F1-macro de al menos 0.89 y con un costo reducido para el funcionamiento del sistema [4]. Este resultado aporta evidencia directa de que la naturaleza computacional de una carga puede inferirse automáticamente en lugar de depender exclusivamente de una clasificación manual basada en el conocimiento previo de la aplicación.

MCBound representa un antecedente especialmente cercano al componente de clasificación, aunque su propósito no es gobernar directamente los estados DVFS de CPU o GPU. La clasificación se utiliza para caracterizar trabajos HPC, pero no se integra con un mecanismo que traduzca de manera continua la clase inferida en una decisión de frecuencia durante las distintas fases de la ejecución. Esta separación resulta importante: demostrar que una carga puede clasificarse como limitada por memoria o por cómputo resuelve el problema de identificación, pero aún queda por determinar cómo utilizar esa información dentro de una política energética y si el costo de realizar dicha inferencia es suficientemente bajo para justificar su uso en tiempo de ejecución.

La actuación dinámica sobre la frecuencia dentro de una aplicación científica ha sido explorada por Simsek et al. [53] en el contexto de simulaciones astrofísicas ejecutadas sobre GPU. Los autores instrumentaron SPH-EXA para medir el comportamiento energético de diferentes regiones y modificar la frecuencia del acelerador en puntos determinados de la ejecución. Frente a una política de frecuencia uniforme, esta adaptación permitió reducir el consumo energético por GPU hasta en 7.82 %, con una degradación del rendimiento de aproximadamente 2.95 %. El estudio demuestra que existe un beneficio potencial al cambiar la frecuencia durante la ejecución y no únicamente antes de iniciar la aplicación. No obstante, la decisión depende de la instrumentación del código y del conocimiento de las regiones sobre las cuales se debe actuar, por lo que la política permanece vinculada a la estructura particular de la aplicación [53].

Un antecedente adicional, más cercano a un control *online* completo, es DRLCAP, un marco de *frequency capping* de GPU en tiempo de ejecución basado en aprendizaje por refuerzo profundo. DRLCAP monitoriza información a nivel de sistema para detectar cambios de fase y ajustar el límite de frecuencia sin intervención manual, reduciendo en promedio 22 % de la energía de GPU con menos de 3 % de degradación de rendimiento en arquitecturas NVIDIA [58]. Este resultado confirma que el control *online* de frecuencia es factible y efectivo, pero también expone una diferencia de diseño relevante para este trabajo: la política se aprende mediante refuerzo profundo, orientada exclusivamente a GPU, en lugar de formularse como una clasificación supervisada ligera de fases seguida de una decisión discreta e interpretable, coordinada sobre CPU y GPU.

Los antecedentes revisados permiten observar una evolución clara del problema. Calore et al. [11] mostraron que diferentes regiones de una aplicación pueden requerir configuraciones DVFS distintas; Guerreiro et al. [25] demostraron que la sensibilidad a la frecuencia puede caracterizarse a partir de métricas de ejecución; Ali et al. [2] utilizaron esta idea para estimar configuraciones energéticamente convenientes sin recorrer todo el espacio DVFS; Antici et al. [4] evidenciaron que la distinción entre cargas *compute-bound* y *memory-bound* puede plantearse como un problema de clasificación; Simsek et al. [53] comprobaron que modificar dinámicamente la frecuencia dentro de una aplicación científica puede producir ahorros energéticos medibles; y Wang et al. [58] demostraron con DRLCAP que un controlador *online* aprendido puede gobernar la frecuencia de GPU en tiempo real con resultados energéticos favorables. En conjunto, estos trabajos cubren las principales etapas necesarias para construir un mecanismo de gestión adaptativa, aunque dichas etapas aparecen generalmente resueltas de manera independiente.

A partir de esta revisión, el problema de investigación no radica en demostrar nuevamente que DVFS puede disminuir el consumo energético ni en establecer que las cargas pueden clasificarse de acuerdo con su comportamiento. La cuestión que permanece abierta para el presente trabajo es cómo integrar ambas capacidades dentro de un mecanismo de control de baja intrusión que pueda operar durante la ejecución. En particular, interesa determinar si la telemetría disponible desde espacio de usuario permite alimentar un clasificador supervisado ligero que identifique el régimen predominante de la carga y utilice esa información como señal para aplicar una política DVFS previamente caracterizada sobre el hardware.

Finalmente, la revisión evidencia una concentración importante de los enfoques recientes de predicción y selección de frecuencia en GPU. Aunque Calore et al. [11] consideran conjuntamente CPU y GPU, los trabajos de Guerreiro et al. [25], Ali et al. [2], Simsek et al. [53] y Wang et al. [58] profundizan principalmente en aceleradores gráficos. Esta diferencia

es relevante para un entorno heterogéneo, ya que CPU y GPU no ofrecen las mismas fuentes de telemetría, poseen mecanismos distintos de gestión de frecuencia y pueden responder de manera diferente ante una reducción del reloj. Por esta razón, una estrategia de control para sistemas CPU–GPU debe reconocer dichas diferencias en lugar de aplicar una única regla de observación y actuación a ambos dispositivos. El propósito del presente trabajo se sitúa precisamente en esta integración: utilizar mecanismos de clasificación de bajo costo como señal de contexto para decisiones DVFS y evaluar empíricamente si dicha estrategia mejora el compromiso entre energía y rendimiento frente a los mecanismos nativos de gestión de frecuencia.

Capítulo 2

Metodología

2.1. Enfoque metodológico

La presente investigación se desarrolla bajo un enfoque cuantitativo de tipo experimental aplicado, orientado al cumplimiento del objetivo general del proyecto. El propósito metodológico consiste en determinar en qué medida un agente de gestión DVFS permite maximizar la eficiencia energética sin degradar significativamente el rendimiento de la aplicación en ejecución. Dicho impacto se evalúa a través de métricas cuantificables y reproducibles, principalmente el tiempo de ejecución, la energía consumida y el Producto Energía–Retardo (EDP).

El carácter experimental se sustenta en la manipulación deliberada del estado de frecuencia impuesto al procesador como variable independiente, sobre un conjunto controlado de cargas de trabajo, observando su efecto sobre variables dependientes de rendimiento y consumo. El carácter aplicado responde a que el producto de la investigación no es únicamente conocimiento descriptivo sobre el comportamiento energético de las cargas, sino un artefacto software funcional cuya utilidad se evalúa empíricamente frente a las alternativas ya disponibles en el sistema operativo.

El desarrollo se estructura en cuatro fases secuenciales e interdependientes, que abarcan desde la caracterización de cargas y extracción de telemetría hasta la validación empírica del sistema.

2.2. Diseño metodológico

2.2.1. Población y muestra

La población de estudio está constituida por el conjunto de aplicaciones ejecutables en sistemas heterogéneos CPU–GPU. Debido a la imposibilidad de abarcar la totalidad de estas, se define una muestra no probabilística de tipo intencional, compuesta por benchmarks y microbenchmarks representativos de regímenes de ejecución contrastantes. Esta selección permite inducir comportamientos diferenciados y obtener evidencia experimental suficiente para construir un conjunto de datos útil para el entrenamiento del clasificador. El uso de benchmarks representativos y de una muestra intencional es coherente con la necesidad de trabajar con cargas controladas y reproducibles dentro del alcance de un proyecto de pregrado.

El criterio de inclusión en el catálogo experimental combina tres condiciones. En primer lugar, la carga debe disponer de una verificación interna de corrección que permita descartar ejecuciones inválidas, de modo que una corrida cuyo resultado numérico no sea correcto no contamine el conjunto de datos. En segundo lugar, el trabajo aritmético de la carga debe ser observable por el propio instrumento: en CPU, mediante los contadores de hardware de instrucciones de punto flotante retiradas descritos en la sección 2.3.11; en el acelerador, mediante el perfilado por contadores de hardware descrito en la sección 2.3.5. La carga no necesita reportar su propio conteo de operaciones, y cuando lo hace ese valor se emplea únicamente como contraste independiente de la medición directa, nunca como su fuente. En tercer lugar, ese trabajo debe consistir efectivamente en operaciones de punto flotante y no en otra clase de cómputo (generación de números aleatorios, ordenamiento de claves, recorrido de índices enteros): una carga de aritmética entera produce un conteo de operaciones de punto flotante cercano a cero y una intensidad operacional sin significado físico, no comparable con un techo Roofline calibrado en FLOPs. Las cargas cuyo trabajo no es genuinamente de punto flotante quedan excluidas del catálogo de caracterización, con independencia de su relevancia como benchmarks en otros contextos, según la regla de validez de FLOPs de la sección 2.3.2.

2.2.2. Plataforma experimental

Los experimentos se ejecutan sobre un nodo de cómputo heterogéneo cuyas características se resumen en la Tabla 2.1. La selección de la plataforma no fue arbitraria: además de la disponibilidad de un acelerador gráfico instrumentable mediante NVML, se estableció como condición de admisibilidad la presencia de una interfaz de medición energética interna accesible sin privilegios administrativos, dado que, como se argumentó en el marco conceptual,

dicha capacidad es una propiedad del procesador que no admite sustitución por software.

La identidad precisa de la microarquitectura no es un dato meramente descriptivo en este trabajo: varios de los eventos crudos de PMU que el instrumento utiliza (sección 1.1.3) carecen de mapeo genérico en el kernel de este nodo y se abren mediante una codificación específica del procesador, validada empíricamente. La restricción en el código para coincidir exactamente con el par `cpu family/model` de la Tabla 2.1, leído en tiempo de ejecución desde `/proc/cpuinfo`, actúa como un filtro de seguridad que impide, por diseño, aplicar una codificación cruda a un nodo distinto. Es el caso, por ejemplo, de los sub-eventos de `FP_ARITH_INST_RETIRED` que sostienen la medición directa de FLOPs (sección 2.3.11).

Tabla 2.1: Características de la plataforma experimental.

Componente		Descripción
Procesador		2 × Intel Xeon Gold 5315Y
Microarquitectura		Ice Lake-SP (10 nm, generación de servidor lanzada en 2021)
Identificador	CPUID	6 / 106
(cpu family/model)		
Extensiones SIMD	relevantes	AVX2, FMA3, AVX-512 (<code>avx512f</code> , <code>avx512bw</code> , <code>avx512cd</code> , <code>avx512dq</code> , <code>avx512vl</code> , <code>avx512_vnni</code> , entre otras) — el ancho de 512 bits es el que domina el conteo de FLOPs medido en las cargas de álgebra lineal densa del catálogo
Núcleos		8 núcleos físicos por zócalo, 2 hilos por núcleo (32 lógicos)
Nodos NUMA		2 (uno por zócalo)
Jerarquía de caché		L1d 48 KiB, L1i 32 KiB, L2 1.25 MiB por núcleo; L3 12 MiB por zócalo
Tamaño de línea de caché		64 bytes
Controlador de frecuencia		<code>intel_pstate</code>
Rango de frecuencia		0.8–3.6 GHz
Dominio de frecuencia		Por núcleo lógico
Medición energética		Intel RAPL (dominios de paquete y DRAM por zócalo)
Acelerador		NVIDIA A100 PCIe 40 GB
Gestor de recursos		Slurm

Las condiciones de ejecución se controlan mediante asignación exclusiva del nodo a través del gestor de recursos, de modo que ninguna carga ajena al experimento comparta el

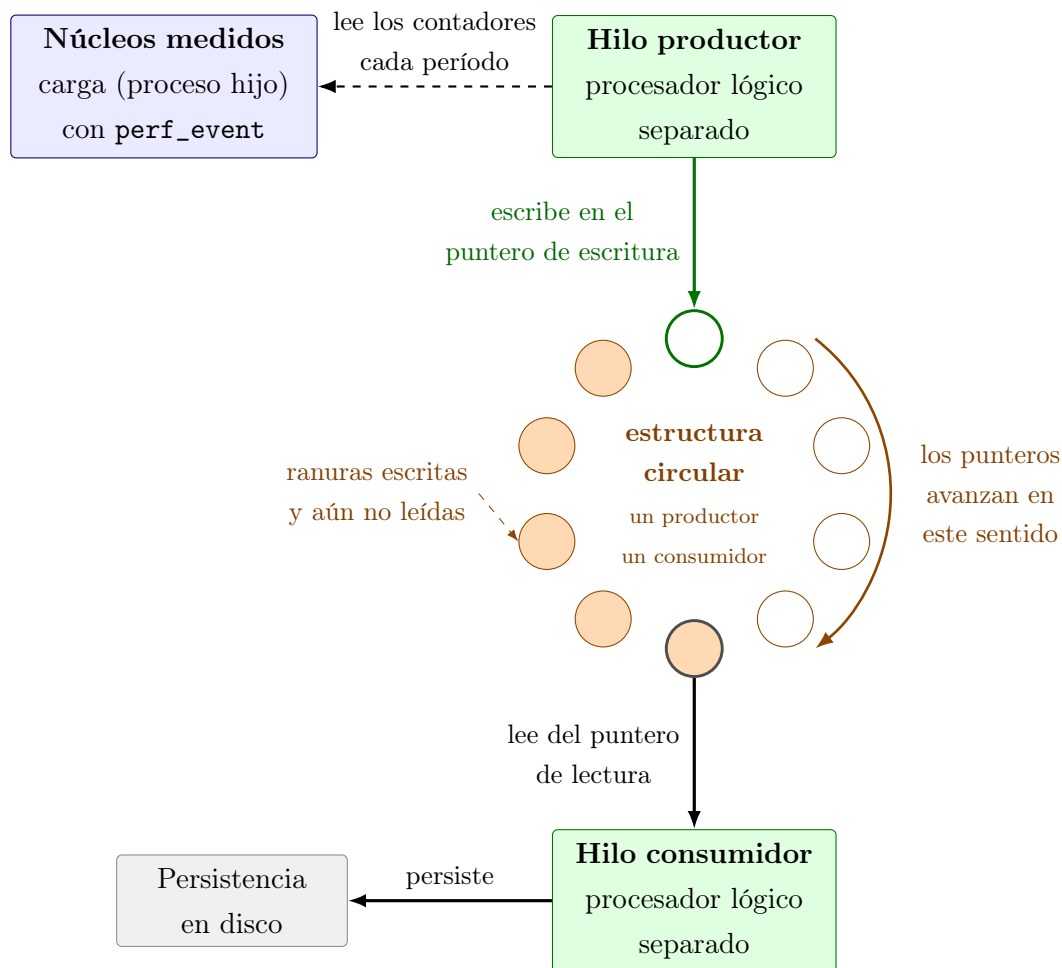
procesador durante las mediciones. Los procesadores lógicos empleados se restringen a hilos primarios de un mismo zócalo, evitando la asignación simultánea de hilos hermanos del mismo núcleo físico, de acuerdo con lo expuesto en la sección 1.1.6.

2.2.3. Instrumentos de recolección

La recolección de telemetría se implementa mediante un instrumento propio, desarrollado específicamente para este trabajo, compuesto por dos capas con responsabilidades separadas.

La capa inferior es un ejecutor escrito en C++ que lanza la carga de trabajo como proceso hijo y adjunta los contadores de hardware sobre ella. El diseño de esta capa responde directamente a los requisitos de atribución expuestos en la sección 1.1.5: el proceso hijo se detiene inmediatamente después de su creación y antes de ejecutar la primera instrucción de la carga, momento en el cual el proceso padre abre los contadores asociados a ese identificador de proceso con herencia habilitada; solo entonces se reanuda la ejecución. Este protocolo garantiza que ninguna instrucción de la carga medida se ejecute fuera del intervalo de observación y que la medida abarque el árbol completo de procesos e hilos que la carga genere, sin incluir actividad ajena.

El muestreo se realiza mediante un hilo productor dedicado, fijado a un procesador lógico distinto de los asignados a la carga, que a intervalos regulares lee los contadores y deposita las muestras en una estructura circular de productor y consumidor únicos. Un hilo consumidor, igualmente fijado a un procesador lógico separado, extrae las muestras y las persiste. Esta separación evita que la actividad de escritura a disco interfiera con la periodicidad del muestreo, y el aislamiento de ambos hilos respecto de los núcleos medidos reduce su interferencia sobre la carga observada. La ausencia de pérdidas en la estructura circular se registra como indicador de integridad de cada corrida. La Figura 2.1 resume esta ruta.



si el puntero de escritura alcanza al de lectura la muestra se pierde;
ese conteo de pérdidas es el indicador de integridad de la corrida

Figura 2.1: Ruta de una muestra de telemetría, desde la lectura de los contadores hasta su persistencia. El hilo productor escribe cada muestra en la ranura que señala el puntero de escritura (borde verde) y el hilo consumidor la retira por el puntero de lectura (borde negro); ambos punteros avanzan en el mismo sentido, de modo que las ranuras naranjas son las escritas que el consumidor aún no ha leído. Los dos hilos se fijan a procesadores lógicos distintos de los que ejecutan la carga, y la estructura circular desacopla la cadencia de muestreo de la escritura a disco. Fuente: elaboración propia.

La capa superior es un orquestador escrito en Python, responsable de la lógica experimental: interpreta el manifiesto que define la campaña, detecta las capacidades de la plataforma, ejecuta las verificaciones previas, realiza la calibración, planifica y lanza cada combinación experimental, aplica el estado de frecuencia correspondiente, valida el resultado de cada corrida y transforma las muestras crudas en el conjunto de datos final. La separación entre ambas

capas responde a un criterio de responsabilidad: la capa C++ resuelve la medición de baja latencia, donde el costo de la instrumentación es crítico, mientras que la capa Python resuelve la lógica experimental, donde prima la claridad y la trazabilidad sobre el rendimiento. La Figura 2.2 resume el recorrido de una campaña a través de ambas capas.

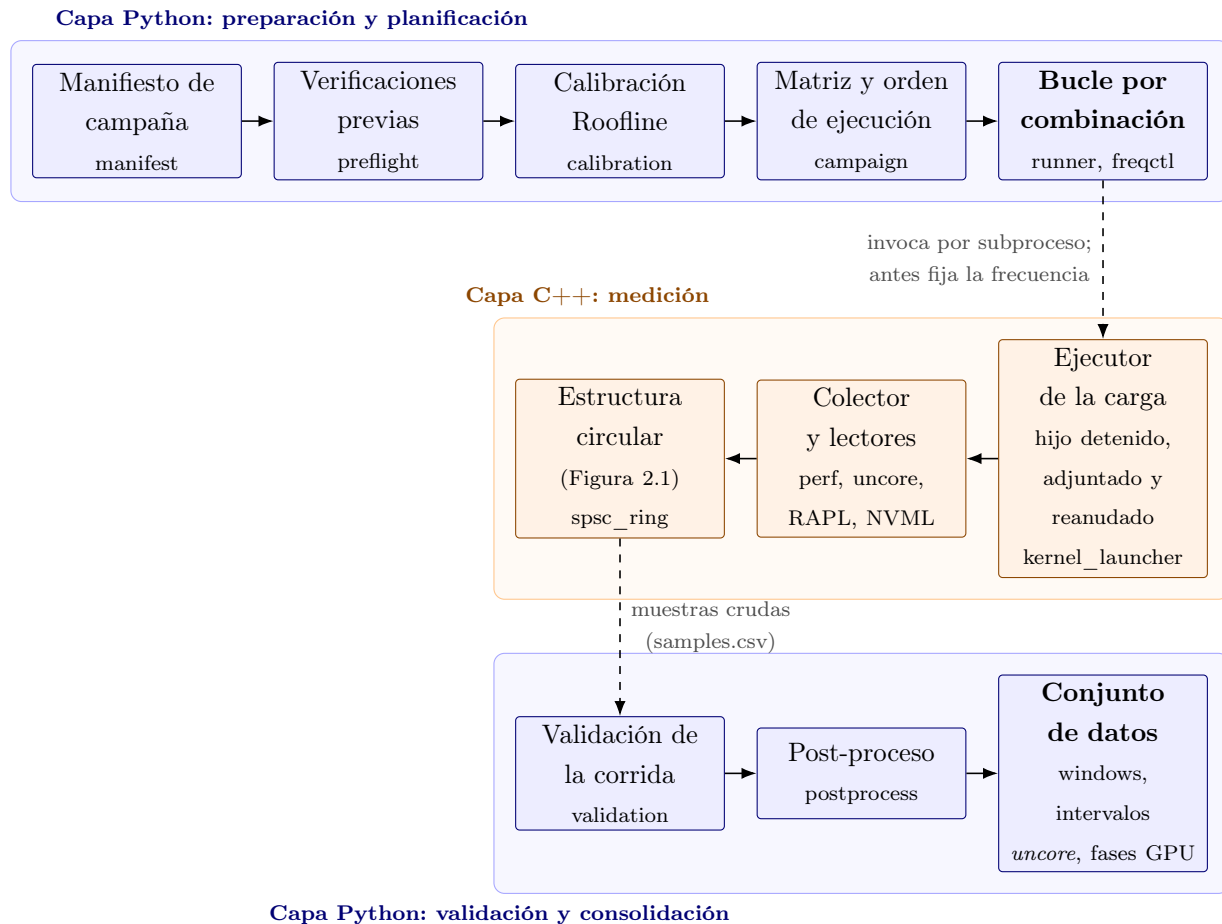


Figura 2.2: Recorrido de una campaña a través de las dos capas del instrumento. La capa Python prepara la campaña (banda superior), invoca por subprocesso el ejecutor de la capa C++ una vez por combinación de carga y frecuencia (banda intermedia), y consolida las muestras crudas que este produce en el conjunto de datos (banda inferior). La capa C++ concentra la medición sensible a la latencia; la capa Python, la lógica experimental, donde priman la claridad y la trazabilidad. Fuente: elaboración propia.

2.2.4. Variables observadas

Antes de describir las señales capturadas, conviene precisar qué se entiende por ventana de muestreo, término empleado de aquí en adelante. El hilo productor descrito en la sección anterior toma una lectura de los contadores cada cierto período configurado en la campaña; cada lectura individual es un valor acumulativo, y por sí sola no describe el comportamiento

de un intervalo concreto. Una ventana de muestreo se define como el intervalo delimitado por dos lecturas consecutivas, y su contenido se obtiene restando la lectura anterior de la lectura actual, conforme a lo argumentado en la sección 1.1.3. Por tanto, una ventana no es un conjunto o agregado de varias muestras, sino el resultado de un único par de lecturas consecutivas; se requieren dos lecturas para producir una ventana, de modo que la primera lectura de cada corrida no genera ventana alguna, al carecer de una lectura previa contra la cual diferenciarse (condición registrada en la Tabla 2.6 como “sin diferencia previa”). La duración real de una ventana corresponde al tiempo efectivamente transcurrido entre ambas lecturas, medido con un reloj monótono, y no al período nominal configurado.

La Tabla 2.2 enumera las señales activas del instrumento de producción y separa los contadores de núcleo, atribuibles a la carga, de los del controlador de memoria, que son de alcance de zócalo. El conjunto de eventos se dimensionó para permanecer dentro del presupuesto de contadores físicos simultáneos de la plataforma, verificado empíricamente sobre la máquina en lugar de asumirse a partir del modelo de procesador, con el fin de evitar el error de extrapolación asociado a la multiplexación descrito en la sección 1.1.3.

Tabla 2.2: Señales de telemetría de CPU activas en el instrumento de producción.

Señal exportada	Evento e interpretación
Contadores de núcleo (<i>atribuibles al proceso medido</i>)	
<code>instructions</code>	PERF_COUNT_HW_INSTRUCTIONS. Base de IPC e instrucciones por segundo.
<code>cycles</code>	PERF_COUNT_HW_CPU_CYCLES. Denominador de IPC y de las fracciones de <i>stall</i> .
<code>cache_references</code> , <code>cache_misses</code>	Eventos genéricos de <code>perf</code> que producen <code>cache_miss_rate</code> y MPKI, los cuales indican presión de memoria, no bytes movidos a DRAM.
<code>stalled_cycles_</code> <code>mem_any</code>	Evento crudo STALLS_MEM_ANY (0xA3/0x14, cmask=0x14). Ciclos de ejecución bloqueados mientras hay una carga de memoria pendiente; se normaliza como <code>stall_mem_ratio</code> . Señal predictiva, no la etiqueta Roofline.
<code>fp_*_double</code>	Subeventos de doble precisión de FP_ARITH_INST_RETIRED (0xC7). Numerador de FLOPs, ponderados por 1, 2, 4 u 8 elementos.
<code>time_enabled_ns</code> , <code>time_running_ns</code>	Metadatos de <code>perf</code> . Diagnostican la multiplexación y escalan los conteos; las ventanas degradadas se excluyen del entrenamiento.
Ámbito de zócalo o de sistema (<i>no atribuibles a un proceso</i>)	
Comandos CAS del IMC: <code>read/write_interval</code>	Número de comandos <i>Column Address Strobe</i> (CAS) de lectura y de escritura emitidos por el controlador de memoria integrado (IMC), contabilizados por su PMU de <i>uncore</i> . Cada comando transfiere una línea de 64 bytes hacia o desde la DRAM, de modo que la suma de ambos es el denominador DRAM del modelo Roofline. Su intervalo práctico (≥ 10 ms) es más grueso que la ventana de CPU, y su etiqueta se difunde a las ventanas que cubre.
RAPL: <code>pkg_uj</code> , <code>dram_uj</code>	Energía acumulada por zócalo. Sus diferencias, con corrección de desbordamiento, producen energía y potencia; no intervienen en la etiqueta <i>compute/memory-bound</i> .
<code>scaling_cur_freq</code> por CPU delegado	Lectura de <i>sysfs</i> por núcleo lógico. Verifica la frecuencia efectiva bajo carga y detecta divergencias entre los núcleos delegados (sección 1.1.6).

El evento histórico `L2_LINES_IN_ALL` (0xF1/0x1F) no integra el conjunto activo: se conservó únicamente su columna vacía en el CSV por compatibilidad de esquema. Se retiró porque el vigilante NMI del sistema operativo reserva un contador físico y, con diez eventos simultáneos, forzaba multiplexación que degradaba los contadores de FLOPs; el conjunto de producción usa nueve eventos (sección 3.1.3).

A partir de estas señales se derivan, para cada ventana, IPC, tasa de fallos de caché, MPKI, `stall_mem_ratio`, intensidad operacional y magnitudes energéticas. Las tasas de núcleo se calculan sobre el intervalo real entre lecturas consecutivas; la intensidad y la etiqueta Roofline conservan la granularidad del intervalo *uncore*, conforme a lo argumentado en la sección 1.1.3.

2.3. Fase 1: Recolección de información y caracterización de cargas

La primera fase tiene por objetivo construir el conjunto de datos etiquetado que sustentará el entrenamiento del clasificador, junto con el instrumento y el protocolo que garantizan su validez. Se describe a continuación el procedimiento completo.

2.3.1. Verificación previa de la plataforma

Antes de ejecutar cualquier medición se aplica una batería de verificaciones automáticas sobre el nodo, orientada a confirmar que las condiciones supuestas por el diseño experimental se cumplen efectivamente. Estas verificaciones se agrupan en cinco familias: capacidades del entorno (estado de las tecnologías de aceleración de frecuencia, afinidad NUMA de los núcleos asignados, política de multihilo declarada, dominio real de frecuencia, número de contadores simultáneos disponibles); integridad del catálogo (existencia y permisos de ejecución de cada binario, correspondencia de su suma de verificación, validez del criterio de éxito declarado, suficiencia de memoria); disponibilidad de instrumentación (dominios energéticos solicitados frente a los presentes, corrección del manejo de desbordamiento); condiciones de almacenamiento y presupuesto; y, cuando la campaña contempla control de frecuencia, permisos efectivos de escritura y cobertura del dominio.

Las verificaciones se clasifican en bloqueantes y no bloqueantes. Una verificación bloqueante que no se satisface detiene la campaña antes de tocar el sistema, bajo el principio de que es preferible no producir datos a producir datos cuya validez no puede sostenerse. Este mecanismo se ejecuta de forma automática como parte del comando que lanza la campaña, y

no como un paso manual previo, de modo que no exista la posibilidad de iniciar una medición sin haberlo aplicado.

2.3.2. Catálogo de cargas de trabajo

El catálogo se compone de cargas de dos papeles. Las cargas de **calibración** determinan empíricamente los techos del modelo Roofline sobre la plataforma: un microbenchmark de ancho de banda de memoria, con un conjunto de trabajo dimensionado para exceder la caché de último nivel y hacer marginal la reutilización en caché, estima BW_{pico} , y un microbenchmark de densidad aritmética sobre datos en caché estima P_{pico} . Ambos valores son cotas empíricas de lo alcanzable en la plataforma, no las tasas exactas de transferencia a memoria ni de ejecución aritmética, cuya relación con el tráfico realmente observado se discute en la sección 1.1.2. Las cargas de **conjunto de datos** son los benchmarks cuyo comportamiento se pretende caracterizar. Todas provienen de suites de amplia adopción, se emplean sin modificar su código fuente, se compilan sobre la plataforma con las opciones por defecto de cada suite, y se registran por su suma de verificación: ese registro, asociado a la plataforma de compilación, permite confirmar antes de cada corrida que el binario ejecutado es exactamente el caracterizado, y descarta que una recompilación silenciosa altere las condiciones a mitad de campaña. La Tabla 2.3 enumera las cargas por nombre y suite de origen.

El bloque de calibración y las primeras filas de conjunto de datos de la Tabla 2.3 son las cargas con las que se construyó y validó el instrumento de medición. La campaña de datos definitiva amplía ese conjunto con cargas adicionales, y esa ampliación obliga a precisar qué cuenta como una observación independiente.

Una misma suite ofrece con frecuencia el mismo algoritmo en varios tamaños de problema: las NAS Parallel Benchmarks lo hacen mediante clases de tamaño, y los kernels de Rodinia mediante parámetros de dimensión de la entrada. Tratar cada tamaño como un ejemplo distinto inflaría el conjunto de datos sin añadir diversidad real, y dejar fuera de la evaluación un tamaño de un algoritmo no prueba generalización hacia un patrón computacional nuevo, sino solo hacia un tamaño nuevo del mismo patrón. Por esta razón, la unidad de diversidad que gobierna tanto la selección de cargas como la validación posterior es la **familia algorítmica**, entendida como el patrón de cómputo y de acceso a memoria compartido por todas las variantes de tamaño de una misma operación, y no la entrada individual del catálogo.

De esta definición se derivan tres reglas de admisión, aplicadas antes de comprometer tiempo de cómputo en el barrido completo:

Tabla 2.3: Catálogo de cargas de trabajo.

Carga	Suite u origen	Papel y régimen esperado
<i>Calibración de los techos Roofline</i>		
STREAM	McCalpin [45]	BW_{pico} de CPU.
ERT	Lo et al. [44]	P_{pico} de CPU.
Microbenchmarks propios (BW, FP32, FP64)	Elaboración propia	Techos de ancho de banda y de cómputo del acelerador.
DGEMM	NVIDIA cuBLAS	Referencia independiente del techo de cómputo del acelerador.
<i>Conjunto de datos — CPU</i>		
BT, MG, CG, SP, FT, LU	NAS Parallel Benchmarks [5]	Métodos implícitos, multigrid, gradiente conjugado y FFT; mayoría <i>memory-bound</i> .
DGEMM	Biblioteca BLAS optimizada [23]	Multiplicación densa de matrices; ancla <i>compute-bound</i> .
LavaMD (OpenMP)	Rodinia [13]	Dinámica molecular de N-cuerpos; régimen intermedio.
3MM (OpenMP)	RAJA Performance Suite [7]	Triple multiplicación densa, con verificación numérica propia; régimen intermedio.
<i>Conjunto de datos — acelerador</i>		
gaussian, backprop, heartwall, LUD, myocyte, DWT2D	Rodinia [13]	Eliminación gaussiana, retropropagación, seguimiento de rasgos, factorización LU, electrofisiología y transformada wavelet; mayoría <i>memory-bound</i> .
LavaMD	Rodinia [13]	Principal ancla <i>compute-bound</i> del acelerador.
DGEMM	NVIDIA cuBLAS	Referencia <i>compute-bound</i> del acelerador.
<i>Candidatos de ampliación del catálogo (sección 2.3.3)</i>		
EP	NAS Parallel Benchmarks [5]	Ancla <i>compute-bound</i> pura en CPU.
kmeans, SRAD, NW, particlefilter	Rodinia [13]	Patrones de acceso irregular en CPU: agrupamiento iterativo, difusión anisotrópica, programación dinámica en diagonal y filtro de partículas.
kmeans, SRAD, NW, b+tree, CFD	Rodinia [13]	Los mismos patrones en el acelerador, más recorrido de árbol y volúmenes finitos; kmeans , SRAD y NW forman pares CPU/acelerador.

1. **Objetivo mínimo por clase y por dispositivo.** Tras el etiquetado empírico, cada clase (*compute-bound* y *memory-bound*) debe quedar representada por al menos cinco o seis familias algorítmicas distintas en cada dispositivo. Un catálogo numeroso pero concentrado en una sola clase no satisface este criterio, y el número de variantes de tamaño de una familia no le concede mayor peso estadístico.
2. **Separación entre controles y cargas de entrenamiento.** Los microbenchmarks sintéticos propios sirven para verificar extremos del plano Roofline, resolución temporal del instrumento y presencia de transiciones, pero no pueblan el conjunto de datos de entrenamiento: una carga construida por el propio proyecto para exhibir un comportamiento no constituye evidencia de que el clasificador generalice hacia cargas científicas reales. Se declaran explícitamente como controles y quedan excluidos del cómputo del mínimo de familias por clase.
3. **Validez de FLOPs como medida de trabajo.** Toda familia cuya naturaleza aritmética invalide la intensidad operacional, en particular los recorridos sobre estructuras de datos y los kernels de aritmética entera sin operaciones de punto flotante, debe demostrar que el cociente FLOPs por byte describe trabajo útil antes de ser elegible; de lo contrario queda fuera del conjunto de entrenamiento, con su motivo registrado.

El hueco de cobertura del catálogo no es de volumen sino de **diversidad de patrón de acceso a memoria**: las familias disponibles son intensivas en ancho de banda de forma bastante uniforme, y está poco representado el acceso irregular (recorridos de estructuras, agrupamiento iterativo, programación dinámica sobre diagonales, recorrido de árboles), que es precisamente donde se concentran los casos límite más informativos para un clasificador de régimen. Se identificaron diez candidatos adicionales para cerrar ese hueco, cinco de CPU y cinco de acelerador, escogidos además de modo que tres de ellos formen pares del mismo algoritmo en ambos dispositivos, lo que permite contrastar si un mismo patrón de acceso recibe la misma clase en uno y otro. Estos candidatos no se incorporan al catálogo como referencias declarativas: cada uno exige binario reproducible, suma de verificación, validación numérica y, en el caso del acelerador, intensidad operacional medida conforme a la sección 2.3.5 antes de considerarse disponible.

El punto que exige más atención al aplicar este criterio es la clase *compute-bound* en el acelerador: varias de las cargas de Rodinia disponibles en el acelerador se eligieron en su momento por su presión sobre la memoria del dispositivo, de modo que alcanzar el mínimo de familias *compute-bound* en el acelerador depende de incorporar los candidatos de refuerzo de esa clase, y ese requisito se satisface añadiendo familias, no relajando el umbral.

2.3.3. Cribado de cobertura y congelación del catálogo definitivo

Antes del barrido completo, las cargas candidatas pasan por una etapa de **cribado** más barata: una rejilla de frecuencia reducida a tres niveles por dispositivo y el número mínimo de repeticiones, con el único fin de descartar candidatos, comprobar que su señal es utilizable y estimar su cobertura del plano Roofline. El cribado no deriva ningún óptimo energético. Se ejecuta por separado para cada dispositivo, y en el acelerador la caracterización de intensidad operacional de la sección 2.3.5 debe completarse antes, para no estimar la cobertura sobre etiquetas sin fundamento.

Para cada familia y dispositivo se mide la fracción de observaciones en cada clase, la distribución de la intensidad operacional respecto al punto de inflexión, la cobertura y las causas de rechazo por calidad. En el acelerador estos criterios se interpretan distinto: como la etiqueta es una constante por kernel medida fuera de línea, la fracción por clase dentro de una corrida es siempre del cien por cien en una sola clase, y lo que el cribado aporta allí es qué familias quedan lo bastante cerca del punto de inflexión como para cambiar de clase entre niveles de frecuencia, y si las señales de baja intrusión disponibles en línea separan las clases. El resultado es una **lista versionada y congelada de familias elegibles**, que distingue candidatos externos, controles y familias con semántica de FLOPs dudosa; solo esas familias pasan al barrido completo, y si la cobertura no alcanza el mínimo por clase se añaden candidatos en lugar de rebajar el umbral.

2.3.4. Calibración y criterio de etiquetado

La calibración se ejecuta al inicio de cada campaña, sobre la misma asignación de núcleos y con la misma política de afinidad que se emplea después en las corridas de caracterización, de modo que los techos obtenidos correspondan a la configuración realmente utilizada. A partir de los valores medidos se calcula el punto de inflexión de la ecuación (1.3) para cada dispositivo y cada precisión aritmética y, en la campaña que barre varios niveles de frecuencia, también para cada nivel, conforme al criterio de la sección 1.1.2; no se calcula una sola vez por dispositivo. Como control de validez, los valores obtenidos se contrastan contra una referencia empírica declarada en el manifiesto de la campaña, y una discrepancia superior a una tolerancia establecida detiene la ejecución: esta verificación protege frente a calibraciones realizadas bajo condiciones anómalas que pasarían inadvertidas si el valor resultante se aceptara sin contraste.

Sobre esta base se define el etiquetado. La etiqueta de cada unidad de observación se deriva exclusivamente de la comparación entre su intensidad operacional y el punto de inflexión

calibrado, sin intervención de juicio manual; la unidad sobre la que se define esa intensidad, y no siempre la ventana individual, se precisa en el contrato de granularidad de la sección 2.3.10. Adicionalmente, cada carga del catálogo declara una expectativa cualitativa de régimen, proveniente de la literatura sobre la suite correspondiente. Es importante subrayar que ambas anotaciones cumplen funciones distintas y no deben confundirse: la etiqueta derivada del modelo constituye la variable objetivo del problema de clasificación, mientras que la expectativa declarada actúa únicamente como control de coherencia. La correspondencia entre ambas, evaluada de forma agregada por carga, permite detectar errores sistemáticos del instrumento, pero la expectativa nunca sustituye ni corrige la etiqueta medida.

Como control adicional de estabilidad, cada campaña incluye repeticiones de una carga de referencia, sobre las cuales se calcula la dispersión relativa de las métricas de eficiencia. Una dispersión elevada indica que las condiciones de medición no son suficientemente estables como para sostener comparaciones finas entre corridas, y se registra como advertencia asociada a la campaña.

2.3.5. Caracterización de intensidad operacional y calibración Roofline en GPU

A diferencia de CPU, en GPU **no existe una medición de FLOPs por ventana**. La telemetría muestreada durante la corrida en el dominio GPU proviene exclusivamente de NVIDIA Management Library (NVML), y por cada muestra produce únicamente potencia, utilización (de cómputo y de memoria), reloj del multiprocesador de streaming, temperatura y energía acumulada del dispositivo, y ninguna de estas señales cuantifica cuántas operaciones de punto flotante realizó el kernel ni cuántos bytes movió. La intensidad operacional de cada carga GPU se obtiene, en cambio, mediante un procedimiento distinto y separado, más cercano a la calibración de los techos del modelo Roofline que a un muestreo continuo: una caracterización dinámica *fuera de línea* del kernel, ejecutada con un perfilador de instrucciones (NVIDIA Nsight Compute) [51] antes, y de forma independiente, de la campaña de adquisición de telemetría NVML en tiempo real. Nsight Compute expone contadores de hardware capaces de contabilizar, por cada lanzamiento de kernel, tanto el tráfico total hacia memoria del dispositivo como el número de instrucciones aritméticas de punto flotante efectivamente ejecutadas, discriminadas por tipo de operación (suma, multiplicación y multiplicación-suma fusionada) y por precisión (simple o doble). A partir de estos conteos, y para una precisión aritmética dada, la intensidad operacional de la carga se calcula como

$$I^{\text{GPU}} = \frac{\sum_{\ell=1}^L (N_{\ell}^{\text{add}} + N_{\ell}^{\text{mul}} + 2 N_{\ell}^{\text{fma}})}{\sum_{\ell=1}^L B_{\ell}}, \quad (2.1)$$

donde ℓ recorre L lanzamientos consecutivos del kernel; N_{ℓ}^{add} , N_{ℓ}^{mul} y N_{ℓ}^{fma} son los conteos de operaciones de suma, multiplicación y multiplicación–suma fusionada de punto flotante que el perfilador reporta para el lanzamiento ℓ , y B_{ℓ} es el total de bytes transferidos hacia o desde la memoria del dispositivo en ese lanzamiento. Cada multiplicación–suma fusionada contribuye con dos operaciones, según la misma convención adoptada para CPU en la ecuación (2.2).

A diferencia de CPU, donde ambos términos de la intensidad operacional se leen en vivo durante cada corrida y por intervalo del controlador de memoria, en el acelerador esa lectura no está disponible. La intensidad se caracteriza entonces una sola vez, fuera de línea, y se declara como una propiedad fija del catálogo, análoga a la suma de verificación del binario; el post-procesamiento la copia sin variación a todas las muestras NVML de la corrida. Este tratamiento es admisible por dos motivos. El primero es que el número de operaciones de punto flotante y el tráfico hacia la memoria del dispositivo que ejecuta un kernel quedan fijados por su algoritmo, por el tamaño del problema y por la precisión aritmética, no por el estado de frecuencia bajo el cual corre ni por el instante en que se observa; agregados sobre un número suficiente de lanzamientos consecutivos, su cociente se estabiliza, y comprobar esa estabilidad es precisamente el objeto del criterio de convergencia que se describe a continuación. El segundo es que el costo de intrusión del perfilado detallado sobre el tiempo de ejecución hace inviable repetirlo dentro de cada corrida de la campaña principal. La ecuación (2.1) es, por tanto, una medición directa de ambos términos de la intensidad operacional, análoga a la ecuación (1.4) en CPU, pero agregada por lote de lanzamientos en lugar de por intervalo de observación y calculada una única vez fuera de línea.

Que ese valor sea único por carga no lo vuelve, sin embargo, automáticamente admisible: un cociente calculado sobre un número insuficiente de lanzamientos puede no haberse estabilizado todavía, y aceptarlo equivaldría a asignar una etiqueta Roofline sin evidencia de que representa el comportamiento sostenido del kernel. Por ello la caracterización no produce una sola medición, sino una serie: el kernel se perfila con cantidades crecientes de trabajo y, para cada punto, se registran los lanzamientos solicitados frente a los efectivamente observados, los FLOPs discriminados por precisión y los bytes de memoria del dispositivo coherentes con

ella. Sobre esa serie se aplica un criterio de convergencia **declarado antes de medir**, y no ajustado a la vista del resultado: la intensidad operacional se acepta únicamente si su cambio relativo entre los dos puntos de mayor trabajo permanece por debajo del uno por ciento, y si el número de lanzamientos observados corresponde al solicitado. Cuando la serie no satisface esa condición, la carga queda registrada como no convergente y no elegible para aportar verdad Roofline, en lugar de admitirse con el último valor disponible.

El mismo procedimiento resuelve una segunda decisión que no puede tomarse por inspección del código: si el kernel es apto en absoluto para este modelo. La caracterización clasifica cada carga según la precisión efectivamente observada, distinguiendo las homogéneas en simple o doble precisión de aquellas con mezcla no trivial, y separa explícitamente las que no ejecutan operaciones de punto flotante en cantidad significativa, por ser de aritmética entera o por concentrar su trabajo en movimiento de datos. Estas últimas se marcan como no aptas para servir de verdad Roofline y se excluyen del conjunto de datos, con el motivo registrado, en lugar de recibir una intensidad operacional formalmente calculable pero carente de significado físico. Se conservan, junto con el veredicto, la salida cruda de la herramienta, los comandos ejecutados y las versiones del perfilador, del controlador y del entorno de ejecución, de modo que el juicio pueda auditarse sin repetir la medición.

Esta discriminación por precisión, sin embargo, presupone que Nsight Compute se invoque solicitando los contadores de **ambas** precisiones simultáneamente para cada carga, y no únicamente los de la precisión que el catálogo declara para ella por diseño de kernel: una auditoría posterior (ARC-110) encontró que las herramientas de caracterización usadas en este trabajo inicialmente seleccionaban un único juego de contadores según esa declaración, de modo que una carga con operaciones reales en la precisión no solicitada las habría subestimado sin ninguna señal de que la mezcla existía. Corregido para solicitar siempre ambas precisiones y contrastar la declaración del catálogo contra lo observado, el método de un único techo Roofline por precisión — el empleado en este trabajo — solo es válido para cargas empíricamente homogéneas en una precisión; una carga con proporciones no triviales de ambas precisiones no admite compararse contra un único ridge sin antes decidir explícitamente cómo tratar esa mezcla (excluirla, tratarla como una categoría aparte, o extender el modelo a varios techos), decisión que este trabajo no adopta unilateralmente para ninguna carga de su catálogo.

Conviene señalar explícitamente por qué el mecanismo de perfilado en GPU no hereda la restricción de presupuesto de contadores físicos simultáneos discutida para CPU en la sección 1.1.3, ni el riesgo de multiplexación asociado a ella. Un contador de PMU de CPU, abierto mediante `perf_event_open`, se muestrea en vivo mientras la carga corre una sola vez; cuando

el número de eventos solicitados excede el número de contadores físicos disponibles, el kernel reparte el tiempo de esa única ejecución entre ellos y extrapola cada lectura mediante el factor de escalado tiempo-habilitado/tiempo-contando (sección 1.1.3), un procedimiento que asume una tasa de evento uniforme durante todo el intervalo y que resulta especialmente riesgoso cuando la ventana de observación es corta frente al período de rotación del kernel. Nsight Compute, en cambio, no muestrea en vivo una única ejecución: cuando las métricas solicitadas exceden lo que el hardware de contadores de la GPU puede capturar en una sola pasada, la herramienta *reproduce* el lanzamiento del kernel completo tantas veces como sea necesario, una pasada por cada subconjunto de métricas, bajo la condición de que el kernel sea determinista frente a sus mismas entradas. Cada métrica se obtiene así de una ejecución íntegra del kernel, no de una fracción de tiempo compartida con otras métricas, por lo que no existe una extrapolación lineal equivalente a la de CPU ni una ventana de observación corta que pueda quedar a mitad de una rotación. La precisión aritmética (simple o doble) se discrimina, en consecuencia, sin ningún compromiso de presupuesto: da igual cuántas métricas o precisiones se soliciten, el costo se traduce en más pasadas de perfilado, no en una restricción de cuántas pueden coexistir — un costo asumible precisamente porque, como se explicó arriba, esta medición ocurre una única vez por carga, fuera de la campaña de adquisición de telemetría en tiempo real.

La calibración de los techos del modelo Roofline en GPU sigue el mismo principio general descrito para CPU — microbenchmarks dedicados a saturar por separado cada límite, bajo las condiciones reales de la plataforma —, con dos particularidades propias del dispositivo. Primera, dado que las GPU consideradas ejecutan operaciones de simple y doble precisión a tasas pico sustancialmente distintas, P_{pico} se calibra por separado para cada precisión, y el punto de inflexión de la ecuación (1.3) se calcula igualmente por separado, de modo que una carga expresada en una precisión se compara exclusivamente contra el techo correspondiente a esa misma precisión. Segunda, la carga empleada para calibrar P_{pico} se implementó como un microbenchmark propio de multiplicación–suma fusionada sobre datos residentes en registros, en lugar de recurrir a una biblioteca de álgebra lineal optimizada de terceros: una biblioteca de este tipo puede seleccionar internamente, sin que el usuario lo solicite explícitamente, una ruta de cómputo acelerada por hardware especializado no representativo de la aritmética general de punto flotante del dispositivo, lo que produciría un techo de cómputo inflado frente al alcanzable por las cargas del catálogo que no emplean dicha ruta, y en consecuencia clasificaría erróneamente como *memory-bound* cargas que, medidas contra el techo de aritmética vainilla correspondiente a su precisión, resultan *compute-bound*. Este riesgo es formalmente equivalente al ya discutido en la sección 2.2.1 para el caso de una carga que cuantifica su trabajo en una unidad distinta de FLOPs: en ambos casos, comparar magnitudes que no

están expresadas en el mismo referente invalida la clasificación resultante, aunque el error no se manifieste como una falla explícita del instrumento.

2.3.6. Dimensionamiento de las cargas GPU y verificación de invarianza de la intensidad operacional

Puesto que la intensidad operacional de cada carga GPU se fija mediante la medición directa descrita en la sección 2.3.5, cualquier ajuste posterior de los parámetros de ejecución de una carga — por ejemplo, para alcanzar una duración de corrida suficiente para los fines descritos en la sección 2.3.9 — exige distinguir explícitamente entre dos categorías de parámetros según su efecto sobre dicha medición.

La primera categoría comprende los parámetros que repiten el mismo patrón de cómputo un mayor número de veces sin alterar el tamaño del problema resuelto en cada repetición — un número de iteraciones de un mismo esquema numérico, o la duración simulada de un proceso cuyo paso de integración permanece fijo —. Un ajuste de esta naturaleza no puede alterar la intensidad operacional del kernel, dado que cada repetición ejecuta exactamente el mismo balance entre trabajo aritmético y tráfico de memoria que la anterior; en consecuencia, este tipo de ajuste puede aplicarse sin necesidad de repetir la medición de la sección 2.3.5. La segunda categoría comprende los parámetros que alteran el tamaño real del problema — la geometría de una malla, el número de elementos de una simulación de N-cuerpos, o la resolución de una imagen —, para los cuales dicha invarianza no puede asumirse: un problema de mayor tamaño puede modificar la proporción entre trabajo aritmético y tráfico de memoria, entre otras razones porque el conjunto de datos activo deja de residir en un nivel de la jerarquía de caché que sí alcanzaba a contener al problema original, incrementando el tráfico efectivo hacia memoria principal de forma no proporcional al aumento del trabajo aritmético. Por ello, todo ajuste de la segunda categoría se trata como una modificación que invalida la caracterización previa de la carga, y se somete obligatoriamente a una nueva medición mediante el procedimiento de la sección 2.3.5 antes de aceptar cualquier dato adquirido bajo la nueva configuración; cuando dicha remediación confirma un cambio material en la intensidad operacional, se verifica adicionalmente que la carga conserve su clasificación cualitativa frente al punto de inflexión vigente, dado que un cambio de magnitud no implica necesariamente un cambio de régimen.

Cuando una carga no dispone de un parámetro de la primera categoría — por ejemplo, porque su implementación no contempla una noción de repetición interna del mismo cómputo —, la única vía disponible para extender su duración es un parámetro de la segunda categoría, con el costo de verificación que ello implica; y cuando, adicionalmente, el modelo de ejecución

del dispositivo impone un límite estructural sobre dicho parámetro — por ejemplo, un límite documentado sobre el número de unidades de ejecución concurrentes direccionables por un único lanzamiento de kernel —, la duración máxima alcanzable para esa carga queda acotada por dicho límite de hardware, con independencia de cualquier criterio estadístico de suficiencia de muestreo. En tales casos, la imposibilidad de extender la corrida más allá del límite estructural del dispositivo se declara como una limitación del instrumento específica de esa carga, y no se fuerza una extensión que comprometería la validez de la caracterización ya obtenida.

2.3.7. Observabilidad efectiva y latencia de conmutación de reloj en GPU

Dos propiedades de la plataforma condicionan por completo el diseño de la campaña del acelerador y deben medirse antes de ejecutarla, no verificarse después: con qué frecuencia la interfaz de gestión del dispositivo actualiza realmente sus señales, y cuánto tarda el reloj en asentarse tras solicitar un cambio. Ninguna de las dos admite un valor tomado de la documentación del fabricante.

Cadencia efectiva frente a cadencia solicitada. El período configurado en el colector es la cadencia con que se invoca la interfaz, no la frecuencia con que esta publica un valor nuevo. Un barrido previo sobre la plataforma experimental, con distintos períodos de sondeo y varias repeticiones por punto, mostró que ambas magnitudes difieren de forma sustancial: un período grueso resultó frágil para las cargas cortas, que quedaban con muy pocas lecturas útiles a lo largo de toda su ejecución, mientras que un período fino no convirtió esas lecturas en observaciones independientes, sino que reveló escalones sostenidos de aproximadamente cien milisegundos durante los cuales la potencia y la utilización devolvían valores consecutivos idénticos. Ambas afirmaciones son simultáneamente ciertas y no se contradicen: sondear más fino reduce la incertidumbre sobre *cuándo* ocurrió un cambio y evita perder cobertura en corridas breves, pero no añade información física entre dos actualizaciones del sensor.

De esto se sigue una precaución que gobierna el resto del tratamiento de los datos del acelerador: el número de valores distintos observados en una corrida constituye una **cota inferior** del número de actualizaciones físicas, nunca una medida de ellas, puesto que dos actualizaciones consecutivas pueden devolver el mismo número. La cadencia de campaña se elige, en consecuencia, como el período más grueso que conserva los bordes observables y la cobertura de la carga más corta frente a un período fino de referencia, y la decisión se versiona junto con la versión del controlador sobre la que se tomó, sin extrapolarse a otro

dispositivo.

Latencia de conmutación bajo carga. La segunda medición determina el tiempo transcurrido entre solicitar un nivel de reloj y verificar que el dispositivo se sostiene en él, bajo carga de cómputo real y no en reposo. El resultado no es un número único sino una **matriz dirigida** de pares de niveles, porque no hay razón para suponer que la latencia sea simétrica entre subir y bajar, ni constante a lo largo del rango, ni independiente de la distancia del salto; cada par se repite un mínimo de tres veces, y el nivel de origen se fija y confirma bajo carga antes de solicitar el destino. Se registra por separado el tiempo que tarda en retornar el comando de actuación, para no confundir el costo de invocarlo con la respuesta posterior del controlador.

El destino se declara alcanzado únicamente cuando varias lecturas consecutivas permanecen dentro de una tolerancia declarada, con el dispositivo activo y sin condiciones de limitación que invaliden la interpretación, y no en la primera lectura que toca el valor objetivo, criterio que confundiría un sobreimpulso transitorio con la convergencia. La tolerancia no se toma de la ficha técnica: se fija por debajo de la mitad del salto al reloj soportado vecino y se documenta junto con el resultado. De la matriz se deriva una **cota conservadora** igual al máximo observado sobre todos los pares y repeticiones, nunca su promedio, dado que el propósito de la magnitud es acotar el peor caso que la política podrá encontrar. Si la primera lectura estable aparece en el límite de la resolución temporal del muestreo, el valor se declara explícitamente como cota superior observable y no como latencia física exacta; esa distinción no lo invalida, porque una cota superior sigue siendo segura para el uso que se le da.

Consecuencias sobre el resto del diseño. Esta cota no es un dato descriptivo. Determina, primero, qué observaciones del barrido pueden entrar en el análisis energético: toda fase cuya duración sea inferior a la cota se excluye del cálculo agregado por nivel, con su motivo registrado, porque su medición está contaminada por una transición que no llegó a completarse. Determina, segundo, el valor mínimo de permanencia que el agente de control podrá respetar entre dos decisiones consecutivas, que no puede fijarse por conveniencia ni heredarse de un valor por defecto. Y determina, tercero, si actuar sobre la frecuencia del acelerador es siquiera un mecanismo viable para la distribución real de duraciones de fase del catálogo: si la cota resulta comparable o superior a esas duraciones, la respuesta metodológicamente correcta es declarar que en ese dispositivo no se actúa, y reportarlo como un hallazgo sobre los límites del mecanismo y no como una carencia del sistema propuesto.

2.3.8. Matriz experimental y control de sesgos

La matriz experimental se define como el producto cartesiano entre las cargas del catálogo, los estados de frecuencia y las repeticiones independientes. Los estados de frecuencia no forman una única rejilla. El cribado de cobertura (sección 2.3.3) emplea una **rejilla reducida** de tres niveles por dispositivo: el nativo (**REF**) y dos niveles fijos, hacia la mitad y hacia el extremo inferior del rango soportado en CPU, y en la mitad inferior del rango en el acelerador; esta rejilla solo sirve para descartar candidatos y estimar cobertura, y no alimenta ningún análisis energético. La campaña definitiva, sobre las familias ya congeladas, barre la **rejilla fina** de la Tabla 2.4, que es la que alimenta el conjunto de datos y la derivación de la política de frecuencia. El punto nativo (**REF**), común a ambas rejillas, se obtiene sin intervenir el gobernador del sistema y actúa como línea base de comparación.

Tabla 2.4: Rejilla fina de frecuencia de la campaña definitiva.

Dispositivo	Niveles de reloj (MHz; REF es el punto nativo)
CPU	REF, F0 (3200), F1 (2900), F2 (2600), F3 (2300), F4 (2000), F5 (1700), F6 (1400), F7 (1100), F8 (800)
GPU	REF, 1410, 1350, 1290, 1230, 1170, 1110, 810, 510, 210

La resolución de la rejilla fina no es uniforme, y la razón es metodológica. Una rejilla equiespaciada, como la de las campañas históricas de validación del instrumento (el nivel nativo y cinco niveles a pasos iguales del rango), tiene un salto superior que deja sin observar precisamente la región en la que una reducción pequeña de frecuencia puede conservar el presupuesto de rendimiento y reducir el consumo; un análisis sobre esa rejilla no puede concluir que no existe un óptimo interior, porque el experimento no midió esa zona. El mismo efecto aparece en el acelerador, donde el salto entre el reloj máximo y el siguiente nivel es demasiado grande para estudiar presupuestos pequeños de degradación. Por eso la rejilla fina concentra puntos en la región alta del rango y los espacia en la región baja, ya suficientemente caracterizada.

Los niveles de la rejilla fina se declaran en unidades absolutas de reloj y se resuelven y verifican contra los relojes que el nodo reporta como realmente soportados en la sesión de campaña; no se aceptan por figurar en la definición de la campaña. Todo nivel solicitado que el nodo no soporte se sustituye por el soportado más próximo o se excluye, con el resultado registrado. La calibración Roofline, en consecuencia, se ejecuta por dispositivo, por precisión y por nivel de esa rejilla, conforme al criterio de la sección 1.1.2, y no una sola vez por dispositivo.

Para reducir el sesgo experimental se adoptan cinco medidas. Primera, cada combinación se ejecuta en múltiples repeticiones independientes, entendidas como relanzamientos completos del binario y no como iteraciones internas, de modo que cada repetición parta de un estado de caché y de predictores equivalente. Segunda, el orden de ejecución de las combinaciones se aleatoriza a partir de una semilla registrada en el manifiesto, con el fin de evitar que una deriva temporal — por ejemplo, térmica — se confunda con el efecto de la carga o de la frecuencia; el registro de la semilla preserva la reproducibilidad del orden. Tercera, las condiciones de afinidad, número de hilos y asignación de núcleos se fijan explícitamente y de forma idéntica para todas las corridas, en lugar de heredarse del entorno. Cuarta, al concluir la campaña se verifica que el estado de frecuencia del sistema haya sido restaurado a su configuración original, y este mismo mecanismo de restauración se registra para ejecutarse también ante una terminación anómala.

Quinta, la matriz de frecuencia fija exige deshabilitar, además de acotar el rango por núcleo, el mecanismo de aceleración dinámica del procesador (*Turbo Boost*). Escribir un rango estrecho de frecuencia por núcleo no basta por sí solo para fijar el reloj físico bajo carga: mientras la aceleración dinámica permanece activa a nivel de plataforma, el gestor de energía del procesador puede seleccionar autónomamente un reloj superior al rango declarado, de modo que un nivel nominalmente fijo de la Tabla 2.4 no garantiza por sí solo que el núcleo haya operado efectivamente a ese reloj durante la carga. Por esta razón, la aceleración dinámica se deshabilita a nivel de plataforma antes de aplicar cualquier nivel fijo de la matriz, su desactivación efectiva se verifica por relectura antes de iniciar la medición, se comprueba que no haya cambiado de estado durante toda la ejecución de la carga, y se restaura a su configuración original al finalizar — bajo el mismo principio de verificación por relectura y restauración garantizada, incluso ante terminación anómala, que ya rige la cuarta medida. Una consecuencia directa es que el nivel fijo más alto de la rejilla es la frecuencia base del procesador, 3200 MHz: el reloj de 3600 MHz que figura en la Tabla 2.1 es la frecuencia máxima de aceleración dinámica (*Turbo Boost*), un estado oportunista que no constituye un punto operativo fijo, y solo se ejerce bajo el nivel REF, con el gobernador nativo y la aceleración dinámica activa. La verificación de que el reloj efectivamente observado coincide con el nivel solicitado, dentro de una tolerancia declarada, se aplica sobre la lectura de cada núcleo delegado por separado (Tabla 2.2) y no únicamente sobre un núcleo representativo. Esta verificación distingue dos niveles de exigencia, de acuerdo con lo que cada tipo de anomalía realmente indica. La integridad estructural de la traza — ausencia de una lectura, un número de núcleos leídos distinto del delegado, o un núcleo representativo inconsistente entre lecturas consecutivas — sigue rechazando la corrida completa: esa condición evidencia que el propio mecanismo de lectura falló, no que el reloj se haya desviado, y ninguna ventana

individual puede compensar esa pérdida de trazabilidad. La desviación respecto a la tolerancia declarada, en cambio, se evalúa y anota por ventana individual, según la taxonomía de la Tabla 2.5, en lugar de rechazar la corrida completa ante la primera lectura fuera de tolerancia. Esta distinción responde a un defecto identificado empíricamente en el criterio anterior: cuando los núcleos delegados se desvían por igual entre sí — por ejemplo, ante una dilución del reloj estimado durante un tramo de inactividad sincronizada de todos los hilos, y no ante una falla de actuación genuina —, ningún criterio basado en la dispersión entre núcleos detecta la anomalía, mientras que un criterio agregado que rechaza la corrida entera ante una sola lectura fuera de rango sí la penaliza, aun cuando el resto de sus ventanas cumplió el objetivo sin incidente. La anotación por ventana evita ambos extremos: ninguna desviación queda oculta, y ninguna corrida se pierde por completo a causa de una fracción ínfima de sus ventanas.

Tabla 2.5: Taxonomía de clasificación de frecuencia por ventana de muestreo.

Anotación	Condición
Válida	Todos los núcleos delegados leídos en la ventana caen dentro de la tolerancia declarada respecto al reloj efectivamente aplicado.
No confiable	Al menos un núcleo delegado se aparta de la tolerancia, fuera del margen de gracia posterior a una transición de nivel.
No verificable (margen de gracia)	La ventana cae dentro del margen de exclusión inmediatamente posterior a una transición de nivel, donde el transitorio físico de conmutación aún no se ha resuelto; la desviación, si la hay, no se atribuye a una falla de actuación.
No aplicable (gobernador nativo)	La corrida se ejecuta bajo el gobernador nativo de la plataforma (nivel de referencia), sin un objetivo de frecuencia fija contra el cual evaluar la tolerancia.

Esta anotación se conserva como columna independiente de la calidad general de la ventana descrita en la Tabla 2.6: una ventana puede ser válida en calidad general y, simultáneamente, no confiable en frecuencia, o viceversa. Solo las ventanas anotadas como válidas o no aplicables cuentan hacia el mínimo de ventanas exigido para aceptar la corrida; las anotadas como no confiables o no verificables se conservan íntegras en el conjunto de datos, con su anotación, pero no contribuyen a ese mínimo. Adicionalmente, cada corrida produce

un resumen de cobertura de frecuencia — fracción de ventanas válidas y longitud de la racha más larga de ventanas no confiables consecutivas — que no condiciona por sí solo la aceptación, pero permite auditar después, corrida por corrida, cuánta de su duración total quedó efectivamente verificada en frecuencia.

2.3.9. Post-procesamiento y control de calidad de los datos

Las muestras crudas se transforman en la traza auditable de la campaña mediante un procedimiento de diferenciación entre lecturas consecutivas, conforme a lo descrito en la sección 1.1.3. Cada fila de esa traza representa una ventana de muestreo del instrumento e incorpora, además de las métricas derivadas, un conjunto de campos de trazabilidad que permiten reconstruir su procedencia: identificador de corrida, carga, nodo, estado de frecuencia solicitado y observado, referencias a los archivos de calibración empleados y suma de verificación del binario ejecutado.

Un elemento central de este procedimiento es que ninguna observación se descarta silenciosamente. Cada ventana recibe una anotación de calidad que describe su condición, según la taxonomía de la Tabla 2.6, y se conserva en el conjunto de datos con dicha anotación. Esta decisión responde a un criterio de transparencia: el filtrado posterior es una decisión del análisis, no del instrumento, y la proporción de ventanas en cada categoría constituye en sí misma un indicador de la calidad de la campaña.

Entre estas categorías, la exclusión por calentamiento delimita explícitamente, para cada carga del catálogo, un intervalo inicial de la corrida declarado como transitorio de arranque, correspondiente al período en que la ejecución aún no alcanza un comportamiento estacionario — por ejemplo, mientras se completan la asignación de memoria, la creación de hilos y la sincronización inicial de la carga. Las ventanas cuya marca temporal cae dentro de ese intervalo se anotan como excluidas por calentamiento y no se emplean en el análisis, de modo que la región efectivamente caracterizada corresponda a la fase repetitiva del kernel y no a su arranque. A diferencia de otras anotaciones de la Tabla 2.6, que se derivan de una condición verificable en la propia ventana, la duración de este intervalo no se fija arbitrariamente por carga ni se estima a partir de una expectativa general de arranque: se determina mediante un procedimiento estadístico aplicado sobre la serie de telemetría de cada corrida, descrito a continuación.

Un aspecto favorable para la aplicación de este procedimiento es que la duración del calentamiento nunca condiciona la adquisición: el intervalo declarado se emplea exclusivamente como criterio de filtrado durante el post-procesamiento, y no se comunica en ningún momen-

to al ejecutor de la capa C++, de modo que toda corrida ya adquirida contiene, desde el primer instante, la totalidad del transitorio de arranque real, con independencia del valor de calentamiento vigente al momento de adquirirla. Esta propiedad permite recalcular y ajustar el criterio de calentamiento de forma retroactiva sobre corridas ya realizadas, sin necesidad de una nueva adquisición.

El procedimiento se desarrolla en dos etapas, aplicadas sobre una señal representativa del estado de carga del dispositivo — instrucciones por ciclo en el caso de CPU, utilización del dispositivo reportada por la interfaz de gestión de GPU en el caso de GPU —, calculada sobre ventanas móviles consecutivas de tamaño fijo.

En la primera etapa se aplica un criterio de estabilidad basado en el coeficiente de variación (CV %) de dicha señal, siguiendo el principio de evaluación estadísticamente rigurosa de desempeño propuesto por Georges et al. [20]: se declara alcanzado el fin del transitorio de arranque en el primer instante a partir del cual el CV % se mantiene por debajo de un umbral en dos ventanas consecutivas — exigencia de dos ventanas, y no de una sola, para no confundir el fin del arranque con una fluctuación transitoria dentro de una fase todavía inestable, ni con un cambio de fase legítimo del propio kernel que ocurra más adelante en la corrida —, y al punto detectado se le aplica un margen de seguridad multiplicativo antes de declararlo como intervalo de calentamiento definitivo. Una limitación de este criterio, identificada durante su verificación empírica sobre la señal de utilización de GPU, merece señalarse explícitamente: dado que el coeficiente de variación se define como el cociente entre la desviación estándar y la media, una ventana cuya señal permanece en cero satisface trivialmente el umbral, de modo que el criterio, aplicado sin más, puede declarar estable el reposo inicial del dispositivo — previo a que exista actividad real — en lugar de la fase de carga sostenida que se pretende delimitar, precisamente el resultado opuesto al buscado. Este modo de falla se corrige exigiendo, adicionalmente al umbral de CV %, que la media de la ventana supere un piso mínimo de actividad antes de aceptar la ventana como estable.

En la segunda etapa, reservada para las cargas donde el criterio de CV % no logra identificar ningún punto que satisfaga la condición anterior — un modo de falla conocido de este tipo de heurística de umbral fijo —, se aplica como respaldo un método de detección de puntos de cambio (*change point detection*) sobre la misma señal: en lugar de asumir que el estado estable es plano dentro de un umbral, el método busca recursivamente la partición de la serie que más reduce la suma de errores cuadráticos dentro de cada segmento resultante, en la línea de los métodos empleados para caracterizar estadísticamente el comportamiento de arranque de máquinas virtuales [6]. Cuando la serie presenta más de un punto de cambio — por ejemplo, una fluctuación breve y espuria previa al inicio real de la carga sostenida —,

se descarta el criterio de aceptar sin más el primer punto detectado, y se selecciona en su lugar el primer segmento cuya media alcanza una fracción declarada de la meseta de mayor carga observada en toda la corrida, de modo que una fluctuación que no llega a sostenerse no se confunda con el fin genuino del arranque.

Las dos etapas anteriores operan sobre una única señal por dispositivo y, por construcción, no pueden distinguir por sí solas una transición genuina de una fluctuación que coincide por casualidad con el criterio de aceptación. Por esta razón, el valor de calentamiento adoptado en el catálogo para cada carga de GPU se sometió, durante su calibración, a una verificación adicional aplicada por inspección directa de la traza cruda de potencia reportada por la interfaz de gestión del dispositivo: se confirmó que el intervalo propuesto por alguna de las dos etapas coincidiera con un salto identificable de potencia, del régimen de reposo al régimen de carga sostenida, y no únicamente con la señal primaria empleada para la detección. Esta verificación se aplicó, sin excepción, a las cargas de GPU recalibradas conforme a este criterio; no constituye una etapa adicional codificada dentro del procedimiento estadístico descrito arriba — que sí se ejecuta de forma sistemática, sin intervención, sobre toda carga del catálogo — sino un contraste puntual realizado durante la calibración, cuyo resultado se fijó como valor final una sola vez. Para tres cargas de GPU con unidades individuales de trabajo demasiado breves para que la cadencia de muestreo del dispositivo resolviera su actividad, ninguna transición de potencia resultó identificable a lo largo de la corrida, con independencia de cuánto se alargó su duración total conforme al criterio de la sección 2.3.6; en esos casos se concluyó que la señal disponible no sostenía una medición confiable del intervalo de calentamiento, el valor se conservó en su configuración por defecto, y la imposibilidad de medirlo se documentó explícitamente como evidencia negativa en el catálogo, en lugar de forzar un valor no respaldado por la medición. Ninguna carga de CPU fue sometida a este contraste: la señal de potencia de paquete del procesador no se empleó con este propósito en el desarrollo de este trabajo, de modo que el intervalo de calentamiento de las cargas de CPU descansa únicamente en las dos etapas estadísticas descritas arriba.

La propiedad señalada más arriba, según la cual el intervalo de calentamiento nunca condiciona la adquisición, permite además prescindir de una campaña de calibración separada. Puesto que la recolección captura siempre la traza completa desde el primer instante, una campaña de calibración y una campaña de datos que compartan manifiesto miden exactamente lo mismo, y ejecutarlas por separado duplicaría tiempo de cómputo sin necesidad técnica. El procedimiento adoptado pliega por tanto la calibración dentro de la propia campaña, en cuatro pasos: la campaña se ejecuta declarando explícitamente un intervalo de calentamiento nulo para todas sus cargas, de modo que ninguna ventana quede excluida y se conserve

íntegro el transitorio; la calibración se calcula sobre los mismos datos que esa campaña ya produjo, con las repeticiones y la cobertura de niveles de frecuencia garantizadas por ser la matriz real y no una reserva aparte; el valor resultante se aplica al catálogo, con copia de respaldo y verificación de suma de comprobación, sin sustitución silenciosa; y finalmente se re-procesan los datos crudos ya adquiridos con el catálogo corregido, sin relanzar ningún kernel, localizando cada corrida por su identificador real y no por una heurística sobre el nombre de su directorio.

Esta declaración de calentamiento nulo es un instrumento del primer paso y nunca debe figurar en el manifiesto que produce el conjunto de datos definitivo; el cuarto paso existe precisamente para devolver el criterio al valor calibrado. Para que esa distinción no pueda perderse en una reanudación, la declaración forma parte de la huella del protocolo experimental descrita en la sección 2.3.12: dos manifiestos que difirieran únicamente en ese campo dejan de compartir huella y no pueden mezclar sus corridas bajo un mismo identificador.

El cuarto paso arrastra una consecuencia que conviene hacer explícita, porque afecta la aceptación de corridas y no solo su procesamiento. El veredicto de aceptación de cada corrida se decide, durante la campaña en vivo, sobre el conjunto de ventanas provisional en el que nada ha sido excluido todavía, que es el único disponible en ese momento. Una corrida cuyo número de ventanas útiles apenas superaba el mínimo exigido puede quedar por debajo de él una vez descontado el calentamiento real, y seguiría figurando como aceptada si nadie reevaluara su veredicto. Por ello el re-procesamiento vuelve a aplicar los criterios de aceptación sobre los datos ya corregidos y sobrescribe el veredicto, sin eliminar ni desplazar ninguna corrida, registrando explícitamente cada cambio respecto al veredicto previo para que se revise antes de dar la campaña por cerrada.

Tabla 2.6: Taxonomía de anotaciones de calidad por ventana de muestreo.

Anotación	Condición
Válida	La ventana satisface todos los criterios de calidad.
Sin diferencia previa	Primera muestra de la corrida; carece de lectura anterior contra la cual diferenciar.
Excluida por calentamiento	La ventana pertenece al intervalo inicial declarado como transitorio de arranque.
Contadores degradados	Diferencia negativa, valor ausente, o fracción de tiempo contando inferior al umbral admitido.
Intensidad indefinida	No fue posible calcular la intensidad operacional, por ausencia de trabajo reportado o de tráfico observable.
Energía inválida	La lectura energética correspondiente no es válida en una campaña que la requiere.
Sin lectura de frecuencia	No se dispone de la frecuencia efectivamente observada durante la ventana.

De forma complementaria, cada corrida completa se somete a una validación posterior que examina su integridad global: ausencia de pérdidas en la estructura de muestreo, correspondencia de la suma de verificación del binario, cumplimiento del criterio de éxito propio de la carga y unicidad del identificador de corrida. Una corrida que no supere esta validación se registra como rechazada, conservando sus artefactos para inspección, en lugar de eliminarse.

2.3.10. Contrato de granularidad del conjunto de entrenamiento

El conjunto de ventanas descrito arriba es el artefacto crudo y auditable de la campaña, y se conserva como tal. No es, sin embargo, el conjunto que consume el entrenamiento del clasificador: la unidad de observación del modelo se declara por separado para cada dispositivo, porque en ambos casos la ventana de muestreo del instrumento y la unidad que posee una etiqueta con respaldo físico independiente no coinciden. Tratar cada ventana como un ejemplo produciría un conjunto de datos que aparenta un tamaño muy superior al de su contenido informativo real, y una estimación de desempeño correspondientemente optimista.

CPU: el intervalo del controlador de memoria. Los contadores de núcleo se leen con una resolución del orden del milisegundo, mientras que los contadores del controlador de memoria que aportan el denominador de la intensidad operacional entregan sus valores en

intervalos prácticos un orden de magnitud más gruesos (sección 1.1.3). Los bytes observados en uno de esos intervalos no pueden atribuirse físicamente a una sola ventana de núcleo, de modo que el procedimiento agrega los FLOPs de todas las ventanas cubiertas por un mismo intervalo, calcula la intensidad sobre ambos términos así alineados, y difunde el valor y su etiqueta a las ventanas cubiertas. Ese es el cálculo correcto de la verdad, pero implica que varias ventanas consecutivas comparten una única observación física de bytes y una única etiqueta.

En consecuencia, la fila del conjunto de entrenamiento de CPU corresponde a **un intervalo del controlador de memoria**, y no a una ventana de núcleo. Las características de cada fila se recalculan sobre los deltas crudos agregados de todo el intervalo, y no promediando los cocientes ya calculados de las ventanas que lo componen: promediar directamente magnitudes como las instrucciones por ciclo o los fallos por millar de instrucciones pondría por igual ventanas con cantidades de trabajo muy distintas, introduciendo un sesgo que ningún filtrado posterior corrige. Cada intervalo declara además sus límites temporales explícitos, y se rechaza aquel que carezca de lecturas válidas del controlador de memoria, que no alcance la cobertura utilizable exigida, que cruce la frontera del calentamiento o que contenga ventanas con contadores o lecturas de frecuencia degradadas; el motivo del rechazo se registra y en ningún caso se imputa un valor ni se recurre a un sustituto derivado de los contadores de caché. La granularidad declarada de la clasificación en CPU pasa a ser, por tanto, la del intervalo del controlador de memoria, y así se documenta en los metadatos del modelo entrenado.

GPU: la corrida completa. En el acelerador la restricción es distinta y más severa. Las señales disponibles en línea se actualizan en escalones de aproximadamente cien milisegundos (sección 2.3.7), de modo que muestras consecutivas no constituyen observaciones independientes; y la etiqueta procede de una intensidad operacional medida fuera de línea que es constante para todo el kernel, de modo que todas las muestras de una corrida comparten etiqueta por construcción. A esto se añade que las cargas del catálogo son binarios de terceros sin marcas de fase, por lo que no existe una frontera interna verificable contra la cual delimitar una unidad menor.

La fila del conjunto de entrenamiento del acelerador corresponde por tanto a **una corrida completa**, entendida como la combinación de carga, nivel de frecuencia y repetición, y nunca a una muestra periódica de la interfaz de gestión. Sus características son agregados robustos calculados sobre las muestras válidas y posteriores al calentamiento de esa corrida: mediana, media recortada, dispersión, rango intercuartílico, extremos, número de valores

distintos como cota inferior de la frescura de la señal, fracción de muestras válidas, duración cubierta y número de muestras. Las magnitudes que sostienen la verdad, es decir la intensidad operacional medida fuera de línea, el punto de inflexión empleado y la etiqueta misma, se conservan en la fila únicamente con fines de trazabilidad y quedan explícitamente prohibidas como características de entrada. Los microbenchmarks propios con fases programadas no son elegibles para entrenamiento bajo este contrato, puesto que su etiqueta constante de catálogo no describe las fases que fueron contruidos para exhibir; permanecen como controles diagnósticos, con esa condición registrada en la propia fila.

2.3.11. Medición directa de FLOPs por ventana y su validación empírica

El diseño original de este instrumento estimaba los FLOPs de cada ventana mediante un prorrateo del total reportado por el propio programa, en proporción a las instrucciones retiradas de cada ventana, bajo el supuesto de que la densidad de operaciones de punto flotante por instrucción retirada se mantiene aproximadamente constante dentro de una misma corrida. Dicho supuesto no se dio por válido sin verificación. Antes de adoptarlo como definitivo, se evaluó si la plataforma experimental permitía sustituirlo por una medición directa, siguiendo el mismo criterio de no asumir capacidades del hardware sin confirmarlas empíricamente que rige el resto de este instrumento (secciones 1.1.5 y 1.1.3). Esta sección documenta ese procedimiento de verificación y su resultado; el prorrateo original, descartado tras esta validación, queda documentado como antecedente metodológico en el registro de cambios del proyecto (ARC-27.1), no en el instrumento.

Encoding y ponderación. Para la plataforma experimental (Intel Xeon Gold 5315Y, Ice Lake-SP), la familia de eventos `FP_ARITH_INST_RETIRED` utiliza el *EventSelect* `0xC7`; cada modalidad aritmética se distingue mediante su *Unit Mask* (`UMask`), de modo que el encoding completo de cada sub-evento no se reduce a `0xC7` por sí solo. Los cuatro sub-eventos de doble precisión empleados se distinguen por el ancho del registro vectorial — escalar, y empaquetado de 128, 256 y 512 bits — mediante los `UMask` `0x01`, `0x04`, `0x10` y `0x40` respectivamente, sin alias simbólico expuesto por el kernel de Linux en este nodo, por lo que se programan como evento crudo mediante `perf_event_open`. Estos eventos contabilizan instrucciones retiradas, no FLOPs de forma abstracta: convertir su conteo requiere el número de elementos de doble precisión por instrucción (1, 2, 4 y 8 para escalar/128/256/512 bits) y el tipo de operación. Las instrucciones *fused multiply-add* (FMA) no exigen una ponderación adicional: la propia definición del evento hace que una FMA incremente el contador en 2 por elemento en vez

de 1, precisamente para que la cuenta ya represente FLOPs y no instrucciones en bruto [32] (Tabla 2.7). El total de FLOPs medidos en una ventana se obtiene entonces ponderando cada sub-evento únicamente por su número de elementos:

$$\text{FLOPs}_i^{\text{HW}} = 1 \cdot \Delta N_{\text{escalar},i} + 2 \cdot \Delta N_{128,i} + 4 \cdot \Delta N_{256,i} + 8 \cdot \Delta N_{512,i} \quad (2.2)$$

Tabla 2.7: FLOPs por instrucción retirada según ancho vectorial y tipo de operación, evento FP_ARITH_INST_RETIRED de Ice Lake-SP. La columna FMA ya está incorporada en el valor que reporta el contador de hardware.

Ancho vectorial	Elementos FP64	FLOPs (ADD/MUL)	FLOPs (FMA)
Escalar	1	1	2
128 bits	2	2	4
256 bits	4	4	8
512 bits	8	8	16

El encoding se determinó a partir de la definición oficial de Intel para esta microarquitectura [32], corroborada contra la tabla de eventos de LIKWID, que documenta el grupo FLOPS_DP con esta misma ponderación para Ice Lake-SP [56], en lugar de inferirse a partir de documentación genérica de otras microarquitecturas.

Diseño de la verificación. Se construyó un instrumento de prueba independiente del harness de producción, con el mismo protocolo de atribución por PID e `inherit=1` descrito en la sección 1.1.5, capaz de abrir simultáneamente los diez contadores que componían el presupuesto inicial: los seis entonces establecidos (sección 1.1.3) más los cuatro sub-eventos de FP_ARITH_INST_RETIRED de la ecuación (2.2). Este instrumento se ejecutó contra dos kernels de régimen opuesto del catálogo experimental: uno intensivo en cómputo (multiplicación de matrices densas, con un volumen de trabajo conocido analíticamente) y uno intensivo en memoria (un kernel multi-hilo de la suite NAS Parallel Benchmarks), en ambos casos bajo la afinidad de núcleos real de la campaña.

Resultados. Para el kernel de cómputo denso, cuyo total de FLOPs se conoce exactamente por construcción ($2 \cdot \text{iteraciones} \cdot n^3$), la ecuación (2.2) reprodujo ese total con un error relativo de 0.29 % sin afinidad fijada y de 0.30 % bajo la afinidad real de campaña. Para el kernel intensivo en memoria, el contraste se hizo contra el total de FLOPs que el propio kernel reporta al finalizar, arrojando un error relativo de 7.48 %; este valor es mayor que el del primer kernel, pero es consistente con una causa identificada y ajena al contador: el tiempo que dicho kernel cronometra internamente cubre únicamente su ciclo de cómputo principal

y excluye la fase final de verificación del resultado, la cual también ejecuta operaciones de punto flotante que el contador de hardware sí registra por abarcar el proceso completo. En ambos casos, los diez contadores abrieron simultáneamente sin evidencia de multiplexación (razón tiempo-contando sobre tiempo-habilitado igual a 1 en los diez), confirmando que esa combinación era sostenible bajo el estado del nodo observado en aquella verificación. Una prueba de humo posterior encontró que la activación del vigilante NMI del sistema operativo había reducido el presupuesto efectivo; la configuración de producción se ajustó entonces a nueve eventos retirando `L2_LINES_IN_ALL`, sin eliminar ninguno de los cuatro sub-eventos que sostienen la medición de FLOPs (sección 3.1.3).

Confirmación a escala de campaña. Con la verificación anterior favorable, la medición directa se integró al harness de producción como única fuente de FLOPs por ventana — sin conservar el prorrateo original como respaldo, una vez confirmado que no aportaba nada en la plataforma y el alcance de este trabajo (sección 1.1.2) —, y se ejecutó una campaña real completa: siete kernels del catálogo, matriz declarada a seis niveles de frecuencia (el nivel nativo y cinco niveles equiespaciados sobre el rango, la rejilla histórica descrita en la sección 2.3.8), tres repeticiones por combinación, 66 corridas aceptadas o rechazadas por los criterios de la sección 2.3.9 y siguientes. Esta campaña se orientó a validar la medición de FLOPs por ventana, que es independiente del estado de frecuencia, y no a ejercitar el control de frecuencia en sí. Sobre aproximadamente 1.29 millones de ventanas con delta válido en esa campaña, el 100 % obtuvo su valor de FLOPs mediante la ecuación (2.2), y ninguna ventana resultó con los contadores de punto flotante degradados. Este resultado, a una escala varios órdenes de magnitud mayor que la verificación inicial de dos kernels, respalda que la disponibilidad y estabilidad del encoding no es un caso aislado favorable, sino una propiedad sostenida de la combinación plataforma-instrumento descrita en este capítulo.

Alcance de esta validación. Los resultados anteriores se verificaron específicamente para la microarquitectura Ice Lake-SP de la plataforma experimental (Tabla 2.1), restringidos a ella por familia y modelo de procesador mediante el mismo mecanismo que protege los demás eventos crudos de este instrumento (sección 1.1.3); no se extrapolan a otro hardware sin repetir este procedimiento. Tampoco se reclama que el error relativo observado (0.29–7.48 %) sea una cota universal: ambos valores dependen del kernel evaluado y, en el segundo caso, de una particularidad de cómo ese kernel específico delimita su propio tiempo de ejecución. Dado que este trabajo no reclama portabilidad más allá de la plataforma descrita, y que la medición directa estuvo disponible sin excepción en las 66 corridas de la campaña real, el instrumento no conserva ningún mecanismo de respaldo para un nodo hipotético donde este

encoding no se haya validado: una ventana en esa situación queda con intensidad operacional indefinida, no con un valor estimado.

2.3.12. Reproducibilidad

Toda campaña queda definida por un manifiesto declarativo bajo control de versiones, que especifica las cargas, los estados de frecuencia, el número de repeticiones, el período de muestreo, la asignación de núcleos, la semilla de aleatorización y las referencias de calibración. Junto con los datos, cada campaña persiste el perfil de capacidades detectado en el nodo, los resultados de calibración y los metadatos de cada corrida. El instrumento de medición, el orquestador y las definiciones de campaña se mantienen en un repositorio público, de modo que la campaña sea reproducible a partir de los artefactos publicados.

Dado que la duración total de la matriz completa excede lo que una única asignación del gestor de recursos puede sostener de forma continua, el protocolo contempla explícitamente la posibilidad de reanudar una campaña interrumpida sin comprometer su validez. Dos garantías sostienen esta reanudación. Primera, una huella del protocolo experimental — que cubre, entre otros campos, las cargas declaradas, los estados de frecuencia, la semilla, la política de muestreo y la suma de verificación del propio instrumento — se persiste junto con los primeros resultados y se contrasta contra la del manifiesto vigente en cada reanudación; una discrepancia detiene la campaña en lugar de mezclar en el mismo conjunto de datos corridas producidas bajo protocolos distintos. Segunda, ante una reanudación genuina — reconocida por la presencia de corridas o de artefactos de calibración previos, incluso si la interrupción ocurrió antes de completar la primera combinación — la calibración no se repite: se recupera del disco la ya obtenida en la sesión anterior, verificando su presencia y validez para cada estado de frecuencia declarado antes de continuar, en lugar de remedirla en silencio y desplazar con ello el punto de inflexión que clasifica las corridas ya aceptadas.

Como verificación adicional de consistencia, toda vez que se modifica un parámetro de ejecución de una carga del catálogo — por ejemplo, a raíz de un ajuste realizado conforme a lo descrito en la sección 2.3.6 — se ejecuta una corrida completa de la campaña que incluya dicha carga, contrastando que la etiqueta derivada, la intensidad operacional y el punto de inflexión empleado resulten idénticos entre las distintas repeticiones independientes de esa misma carga. Esta comparación no sustituye la verificación de calidad por ventana descrita en la Tabla 2.6, sino que actúa como una verificación de nivel superior: una discrepancia entre repeticiones nominalmente idénticas indicaría una fuente de no determinismo en el instrumento o en la propia carga, y no únicamente un artefacto puntual de una corrida concreta.

2.3.13. Selección de características y contrato de columnas

El conjunto de columnas que alimenta el clasificador no se fija por criterio de autor sobre el conjunto de datos real, sino mediante un análisis previo cuyo resultado se congela antes de entrenar. El procedimiento opera sobre las filas elegibles del conjunto de entrenamiento de cada dispositivo, por separado, dado que ambos difieren en fuente de verdad y en columnas de calidad.

Se calculan primero las correlaciones lineal y de rangos entre todas las columnas candidatas, y se reportan los pares cuya correlación absoluta supera un umbral declarado. De cada par se conserva una sola columna, priorizando la medición física más directa sobre el indicador derivado; el criterio no es abstracto, sino que se aplica sobre casos concretos ya identificados, como la coexistencia de dos normalizaciones de la misma señal de fallos de caché por caminos distintos. Sobre el conjunto resultante se calcula además el factor de inflación de la varianza, que detecta la colinealidad entre tres o más columnas que un análisis por pares no revela. Las columnas ausentes, constantes o con valores no finitos se tratan explícitamente y no se arrastran en silencio.

Una restricción es innegociable y se implementa como comprobación dura, no como recomendación: ninguna columna que participe en la construcción de la etiqueta puede figurar entre las características de entrada. La intensidad operacional, el punto de inflexión empleado, los FLOPs y los bytes que la componen y la expectativa cualitativa declarada en el catálogo forman una lista explícita de columnas prohibidas, y su presencia en el vector de entrada aborta el procedimiento en lugar de emitir una advertencia. El fundamento es el descrito en la sección 1.1.11: un modelo que reciba como entrada la magnitud de la que se derivó su objetivo puede reproducir la regla de etiquetado sin haber aprendido ningún patrón inferible a partir de las señales realmente disponibles en producción.

El resultado del análisis es una propuesta, no una decisión automática. La selección final se revisa y se **congela** en un contrato versionado por dispositivo, que registra las columnas admitidas y el instante de congelación, y contra el cual se verifica el entrenamiento posterior. Los descartes y su motivo se documentan como parte del método, dado que constituyen evidencia de que la reducción de columnas obedeció a un criterio declarado y no a un ajuste posterior al resultado.

2.4. Fase 2. Desarrollo del modelo de aprendizaje automático

El problema se formuló como una clasificación supervisada binaria independiente por dispositivo. En CPU, cada observación corresponde a un intervalo real del controlador de memoria y no a una ventana de núcleo de aproximadamente 1 ms. La salida del modelo es la probabilidad de la clase *memory-bound*; la clase *compute-bound* corresponde a su complemento. La etiqueta objetivo se obtuvo fuera de línea con la regla Roofline descrita en la sección 1.1.2. Ninguno de sus insumos se utilizó como entrada del clasificador.

2.4.1. Construcción de las variables de entrada CPU

Las tasas base se recalcularon a partir de los deltas agregados dentro de cada intervalo *uncore*. Este procedimiento evita promediar cocientes calculados sobre ventanas con cantidades de trabajo diferentes. Sean N_{inst} , N_{cyc} , N_{ref} , N_{miss} y N_{stall} los conteos agregados de instrucciones, ciclos, referencias de caché, fallos de caché y ciclos detenidos por memoria, respectivamente. Sea Δt la duración del intervalo en segundos. Las seis variables base se calcularon como:

$$\text{IPC} = \frac{N_{\text{inst}}}{N_{\text{cyc}}}, \quad \text{MPKI} = 1000 \frac{N_{\text{miss}}}{N_{\text{inst}}}, \quad (2.3)$$

$$r_{\text{miss}} = \frac{N_{\text{miss}}}{N_{\text{ref}}}, \quad r_{\text{stall}} = \frac{N_{\text{stall}}}{N_{\text{cyc}}}, \quad (2.4)$$

$$\text{IPS} = \frac{N_{\text{inst}}}{\Delta t}, \quad f_{\text{obs}} = \text{mediana}(f_1, \dots, f_m). \quad (2.5)$$

La frecuencia observada f_{obs} es la mediana de las m lecturas de frecuencia de las ventanas cubiertas por el intervalo. Se conservó como covariable porque la misma carga puede cambiar de posición respecto del *ridge* cuando cambia el techo de cómputo. No representa la frecuencia solicitada ni identifica el nivel nominal del manifiesto.

La representación ampliada añadió seis transformaciones calculables con los mismos contadores, sin incorporar FLOPs ni tráfico de DRAM. Estas transformaciones fueron referencias de caché por millar de instrucciones, referencias por ciclo, fallos por ciclo, ciclos detenidos por millar de instrucciones, ciclos detenidos por fallo de caché y $\log(1 + \text{MPKI})$. Los cocientes cuyo denominador era cero se marcaron como indefinidos y no se imputaron. Las columnas de tiempo habilitado, tiempo corrido y su cociente se emplearon exclusivamente para aceptar

o rechazar contadores. Tampoco ingresaron al modelo los identificadores de kernel, corrida, repetición, nivel nominal o intervalo, dado que podrían actuar como huellas del experimento.

2.4.2. Auditoría de correlación y colinealidad

La auditoría se ejecutó sobre los 1 309 755 intervalos elegibles antes de realizar el submuestreo de evaluación. Para cada par de columnas numéricas se calcularon la correlación lineal de Pearson y la correlación de rangos de Spearman. Un par se marcó cuando el valor absoluto de cualquiera de las dos medidas superó 0.85. Después de retirar una variable por par marcado se calculó el factor de inflación de la varianza de cada columna restante:

$$\text{VIF}_j = \frac{1}{1 - R_j^2}, \quad (2.6)$$

donde R_j^2 se obtiene al ajustar la columna j mediante una regresión lineal sobre las demás columnas estandarizadas. Se adoptó 10 como umbral de alerta. El reporte automático se utilizó como diagnóstico y no como selector ciego. La revisión final prefirió tasas normalizadas frente a deltas crudos que dependen de la duración del intervalo y retiró las cantidades reservadas para control de calidad.

2.4.3. Submuestreo reproducible y ponderación

La evaluación anidada completa sobre más de un millón de intervalos repetidos dentro de 30 familias no aportaba 30 veces más evidencia de generalización. Para controlar el costo y el dominio por volumen se limitó cada celda observada de familia y clase a 1000 intervalos. Dentro de cada celda se generó un orden pseudoaleatorio con semilla 20260918 y se conservaron los primeros $\min(n_{fc}, 1000)$ elementos. Si una celda contenía menos de 1000 intervalos, se retuvieron todos. Si una clase no existía en una familia, no se fabricaron observaciones. La selección produjo 42 455 intervalos distribuidos entre las 30 familias.

El muestreo limita el número de filas, pero no iguala por sí mismo la contribución de todas las celdas. Durante cada ajuste se asignó a la observación i , perteneciente a la familia f y la clase c , un peso inverso al tamaño de su celda:

$$w_i = \frac{1/n_{fc}}{\frac{1}{N} \sum_{k=1}^N 1/n_{f_k c_k}}. \quad (2.7)$$

La normalización del denominador hace que el peso medio sea uno y no altera la masa

relativa entre celdas. La suma de pesos de cada celda observada resulta igual, con independencia de si contiene 5, 100 o 1000 intervalos. XGBoost se configuró con `scale_pos_weight=1.0`, puesto que aplicar simultáneamente una segunda corrección global por clase habría balanceado dos veces los mismos ejemplos.

2.4.4. Validación anidada por familia

La partición externa siguió un esquema *leave-one-family-out* con 30 pliegues. En cada pliegue se retiró por completo una familia algorítmica y se impidió que participara en el ajuste, la selección de representación o la elección del umbral. Sobre las 23 familias restantes se ejecutó otro LOFO para comparar la representación base con la representación de interacciones y los umbrales $\{0,50, 0,55, \dots, 0,90\}$. El modelo utilizado en esta selección corta fue XGBoost con 100 árboles, profundidad máxima 6, semilla 20260918 y los pesos de la ecuación (2.7). No se inició una nueva búsqueda masiva de hiperparámetros.

Sea p_i la probabilidad estimada de *memory-bound*. La confianza y la regla selectiva se definieron como:

$$q_i = \max(p_i, 1 - p_i), \quad (2.8)$$

$$\hat{y}_i = \begin{cases} \text{memory-bound}, & p_i \geq 0,5 \wedge q_i \geq \tau, \\ \text{compute-bound}, & p_i < 0,5 \wedge q_i \geq \tau, \\ \text{revisar}, & q_i < \tau. \end{cases} \quad (2.9)$$

Para cada combinación se calculó la F1 macro media entre familias y la cobertura media entre familias. Solo se consideraron combinaciones con cobertura media mínima de 0.60. Se eligió la de mayor F1 macro media; los empates se resolvieron por mayor cobertura y luego por menor umbral. La cobertura y la F1 macro se calcularon como:

$$\text{Cobertura}(\tau) = \frac{\sum_i \mathbb{I}(q_i \geq \tau)}{N}, \quad (2.10)$$

$$F1_{\text{macro}} = \frac{F1_{\text{compute}} + F1_{\text{memory}}}{2}, \quad (2.11)$$

$$F1_c = \frac{2P_c R_c}{P_c + R_c}. \quad (2.12)$$

La F1 de cada clase y la exactitud se calcularon solo sobre las observaciones cubiertas. Los casos con salida *revisar* se contabilizaron por separado y nunca como aciertos.

Métrica de reporte. La medida principal es la exactitud balanceada por celda de familia y clase. Sea $\mathcal{C}$ el conjunto de celdas (f, c) observadas, con n_{fc} intervalos cada una, y sea a_{fc} el número de esos intervalos clasificados correctamente:

$$\text{EB} = \frac{1}{|\mathcal{C}|} \sum_{(f,c) \in \mathcal{C}} \frac{a_{fc}}{n_{fc}}. \quad (2.13)$$

Esta medida asigna la misma importancia a cada combinación de familia y clase, de modo que no depende del número de intervalos que aporte una familia ni de la proporción entre clases dentro de ella, y está definida cuando una familia contiene una sola clase. Se descartó promediar la F1 macro calculada dentro de cada familia porque en una familia de una sola clase la clase ausente recibe F1 igual a cero y el promedio queda acotado a 0.5 aun con clasificación perfecta, lo que confunde dificultad de clasificación con composición del catálogo. Como referencia comparable con la literatura se reporta además la F1 macro agrupada, obtenida al concatenar todas las predicciones externas, junto con la exactitud, la precisión y la sensibilidad por clase, la exactitud balanceada clásica, el área bajo la curva ROC y el coeficiente de Matthews, definidos en la Tabla 3.8.

La incertidumbre se informa por dos vías. La variabilidad debida al submuestreo se estimó repitiendo la evaluación completa con cinco semillas. La variabilidad debida a la composición del catálogo se estimó mediante remuestreo con reemplazo de las 30 familias, con 2000 réplicas, del que se derivan los intervalos de confianza del 95 % y las comparaciones pareadas entre configuraciones.

2.4.5. Congelación y auditoría del candidato CPU

Terminada la evaluación anidada se fijó la representación con interacciones y se repitió el LOFO interno sobre las 30 familias para obtener un único umbral operativo. El umbral se congeló antes de auditar errores por familia y frecuencia. El modelo serializado se ajustó sobre los 42 455 intervalos con el mismo vector de 12 entradas, ponderación y semilla. Para producir predicciones fuera de muestra, la auditoría reajustó el mismo XGBoost en las 23 familias restantes de cada pliegue y predijo la familia retenida. La representación, los hiperparámetros y el umbral permanecieron fijos y no se modificaron a partir de esos resultados.

En GPU se aplicó el mismo principio de separación por familia, pero la unidad de obser-

vación fue una corrida completa resumida mediante estadísticos de NVML. Esta diferencia responde a la granularidad disponible para construir la etiqueta y no convierte las lecturas repetidas de una misma corrida en ejemplos independientes.

2.5. Fase 3: Implementación del agente de control DVFS

2.5.1. Arquitectura de dos bucles desacoplados

El agente de control se implementó como un proceso en espacio de usuario formado por dos bucles desacoplados y una señal de coordinación. El bucle de CPU operó sobre ventanas de aproximadamente 1 ms, coherentes con la telemetría de la Fase 1, y ejecutó la inferencia y la decisión de frecuencia a baja latencia. El bucle de GPU operó por fases de ejecución completas, dado que la conmutación de reloj en GPU implicó un coste que no se amortizaba a la cadencia del orden del milisegundo. Cuando el bucle de GPU detectó actividad sostenida por encima de un umbral, forzó un piso de frecuencia en el dominio CPU para evitar interpretar esperas bloqueantes como fases *compute-bound*.

2.5.2. Detección de fase en GPU

La detección de fases en GPU se implementó mediante sondeo periódico de la utilización reportada por la interfaz de gestión del dispositivo, sin instrumentar el binario objetivo. Se configuró la cadencia de sondeo y un umbral de ruido para emitir eventos de inicio de fase al cruzar el umbral en sentido ascendente; las transiciones descendentes se registraron para trazabilidad. Esta solución se adoptó por su bajo nivel de intrusión y su eficacia práctica.

2.5.3. Diseño del bucle de CPU e integración con el clasificador

El bucle de CPU incorporó una máquina de decisión independiente de la biblioteca de inferencia, de modo que su lógica se verificó aisladamente. En cada ciclo se construyó el vector de características a partir del snapshot de telemetría, se invocó el modelo exportado desde la Fase 2 y se consultó la tabla de política. Solo se aplicó un nuevo nivel de frecuencia cuando la clase inferida cambió respecto al ciclo anterior, para reducir oscilaciones. La integración se realizó en el mismo lenguaje del colector de telemetría para evitar cruces costosos entre runtimes.

2.5.4. Derivación *offline* de la tabla de política

La tabla de política se derivó *offline* a partir del barrido de frecuencia de la Fase 1. Para cada combinación de dispositivo, clase de régimen y nivel de frecuencia se calculó la mediana del Producto Energía–Retardo por kernel y se comparó con la referencia nativa mediante pruebas de significancia pareadas (Wilcoxon o *t-test*, según corresponda). Cuando ningún nivel mejoró el EDP de forma significativa, la política se fijó en *no actuar*. Para GPU se exigió además una latencia de conmutación verificada bajo carga real antes de habilitar cualquier actuación. La tabla resultante se serializó y el agente la cargó al arrancar.

2.5.5. Actuación y validación en modo simulado

La aplicación de niveles de frecuencia se centralizó en un componente único que registró el estado previo y garantizó su restauración incluso ante terminaciones anómalas. Antes de habilitar escrituras reales, el agente se ejecutó en modo de simulación, donde se registraron las decisiones sin modificar hardware, y se validó el comportamiento frente a corridas históricas. Solo tras estas comprobaciones se habilitó la actuación real bajo control administrativo y de reserva de recursos.

2.6. Fase 4: Validación experimental y análisis de resultados

2.6.1. Escenarios de comparación

La evaluación comparó el agente propuesto frente a tres escenarios de referencia: los gobernadores reactivos nativos (*ondemand*, *schedutil*) y un modo de frecuencia fija (*performance*). En cada escenario se ejecutó el mismo catálogo de cargas bajo las condiciones de aislamiento de la Fase 1 y se midieron tiempo de ejecución, energía, potencia media y Producto Energía–Retardo.

2.6.2. Conmutación de gobernadores y protocolo de medición

La conmutación de gobernadores se implementó verificando primero su disponibilidad en el nodo y aplicándola dentro de un contexto acotado que restauró el gobernador original al finalizar. Las condiciones de afinidad, número de hilos y asignación de núcleos se mantuvieron constantes entre escenarios para garantizar comparabilidad.

2.6.3. Cálculo del EDP y prueba de significancia

El EDP se calculó a partir de las trazas y se contrastó con las líneas base mediante las mismas pruebas pareadas empleadas en la generación de la política, reportándose los resultados desagregados por combinación de dispositivo y clase de régimen.

2.6.4. Generación del reporte comparativo

El reporte final se generó a partir de los archivos de telemetría identificados por campaña. El procedimiento manejó escenarios incompletos omitiendo comparaciones faltantes con aviso explícito, lo que permitió generar reportes parciales sin introducir supuestos.

2.7. Aspectos éticos

La presente investigación no involucra participantes humanos, datos personales ni información sensible, por lo que no requiere aprobación de un comité de ética en investigación con seres humanos. No obstante, se adoptan las siguientes consideraciones éticas, pertinentes a la naturaleza del trabajo.

En cuanto al uso responsable de infraestructura compartida, los experimentos se ejecutan sobre recursos de cómputo institucionales de acceso compartido. En consecuencia, se adopta como principio que ninguna acción del experimento debe degradar las condiciones de ejecución de terceros: las mediciones se realizan mediante asignación exclusiva otorgada por el gestor de recursos, y toda actuación sobre la frecuencia se restringe a los núcleos efectivamente asignados, verificando previamente que el dominio de frecuencia afectado no exceda dicha asignación, de acuerdo con lo expuesto en la sección 1.1.6. Del mismo modo, el estado de frecuencia del sistema se restaura al concluir la campaña, incluso ante terminaciones anómalas.

En cuanto a la integridad de los datos y de los resultados, se adopta como criterio que ninguna observación se descarte de forma silenciosa: las ventanas que no satisfacen los criterios de calidad se conservan con su anotación correspondiente, y las corridas rechazadas preservan sus artefactos. Las limitaciones conocidas del instrumento — en particular, el carácter aproximado de la estimación de bytes movidos discutido en la sección 1.1.2 — se declaran explícitamente junto con los resultados que dependen de ellas, en lugar de omitirse.

En cuanto al uso de software de terceros, las suites de benchmarks empleadas se utilizan sin modificación de su código fuente y respetando sus condiciones de distribución, y su procedencia se registra mediante la suma de verificación de los archivos originales obtenidos.

Finalmente, en cuanto a transparencia y reproducibilidad, el instrumento de medición, el orquestador y las definiciones de campaña se publican en un repositorio de acceso público, junto con la documentación de las decisiones metodológicas adoptadas durante el desarrollo, de modo que los resultados puedan ser verificados o refutados por terceros.

Capítulo 3

Resultados

Este capítulo organiza la evidencia por dispositivo. Primero se presentan los resultados de la campaña final de CPU. Después se reportan la validación del instrumento GPU, el diagnóstico exploratorio de su primer conjunto de entrenamiento y el cambio posterior de la estrategia de medición. Esta distinción evita presentar resultados intermedios como conclusiones finales.

3.1. Resultados de CPU

El conjunto de CPU procede de dos campañas con el mismo protocolo de medición. La campaña `pacca_cpu_final_20260913` cubrió 43 configuraciones de kernel en el nivel nativo y nueve niveles fijos de frecuencia, con tres repeticiones por combinación, y sus 1290 corridas finalizaron aceptadas. Una campaña posterior, `pacca_cpu_compute_sweep_20260919`, añadió seis cargas de cómputo denso (seis configuraciones más, con los mismos diez niveles y tres repeticiones, 180 corridas aceptadas) para reforzar la clase con menos representación. En total son 49 configuraciones y 1470 corridas. En la campaña final el warmup se calibró a partir de las ventanas medidas para cada kernel y el conjunto se reprocesó con esos valores; los transitorios fueron pequeños, con un máximo de 4.8 % de la duración de corrida y valores inferiores a 2 % en la mayoría de los kernels. En las seis cargas añadidas no se descartó un arranque calibrado, y solo se excluyen los transitorios de frecuencia mediante la validación por ventana.

La unidad que alimenta el conjunto de entrenamiento CPU es el intervalo real de *uncore*, no la ventana de muestreo de aproximadamente 1 ms. La Tabla 3.1 resume la cobertura obtenida después del filtrado de calidad. De 1 424 531 intervalos procesados, 1 309 755 cumplieron

simultáneamente los criterios de telemetría y de frecuencia. Los restantes se conservaron para auditoría, pero no recibieron una etiqueta de entrenamiento.

Tabla 3.1: Cobertura y composición del conjunto de intervalos CPU.

Medida	Valor
Configuraciones de kernel	49
Niveles de frecuencia	10
Corridas aceptadas	1470 de 1470
Intervalos procesados	1 424 531
Intervalos elegibles	1 309 755 (91.9 %)
Intervalos <i>compute_bound</i>	513 508 (39.6 %)
Intervalos <i>memory_bound</i>	783 473 (60.4 %)

3.1.1. Composición de fase por kernel

La Figura 3.1 muestra que el conjunto cubre ambos regímenes y conserva configuraciones próximas al *ridge*. Las cargas de cómputo denso, como *dgemm_n2048*, *dual_gemm_cpu_N2048*, *rodinia_lavamd_omp*, *rajaperf_polybench_3mm_omp* y las seis cargas RAJAPerf añadidas (*apps_edge3d*, *apps_fir*, *apps_mass3dpa*, *basic_mat_mat_shared*, *basic_pi_reduce* y *basic_trap_int*), concentran sus intervalos en *compute_bound*. En el extremo opuesto, las configuraciones grandes de AXPY, FFT, stencil y SpMV permanecen *memory_bound*.

La composición intermedia no se concentra en un solo algoritmo. *npb_bt*, *npb_lu*, *cpu_cholmod*, *hpccg_cpu_N46*, las variantes *phasic* y los tamaños dirigidos de FFT y Cholesky aportan intervalos de ambos regímenes. Estas configuraciones evitan que el conjunto quede limitado a ejemplos alejados del punto de inflexión.

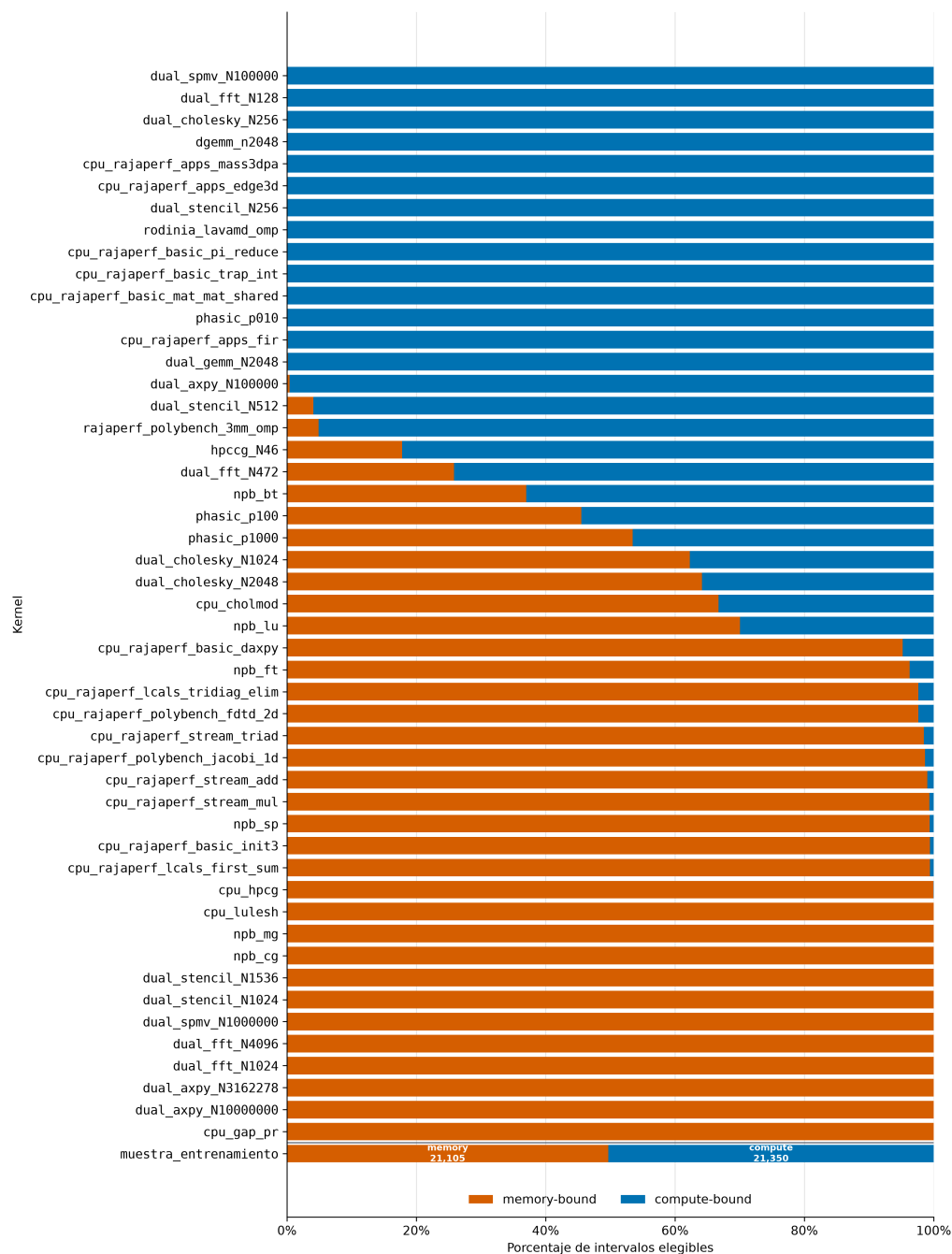


Figura 3.1: Porcentaje de intervalos elegibles clasificados *compute_bound* y *memory_bound* para cada configuración de kernel de la campaña CPU. La barra final resume la muestra usada por el clasificador, con 21 350 intervalos *compute_bound* y 21 105 intervalos *memory_bound*. Fuente: Elaboración propia.

3.1.2. Variación de la composición con la frecuencia

La proporción global de intervalos *compute_bound* aumentó desde 29.0% en el nivel nativo hasta 45.1% en F8, el nivel fijo más bajo. La Figura 3.2 presenta esta variación. El resultado es consistente con un *ridge* que disminuye cuando baja el techo de cómputo, de modo que una parte de los intervalos cercanos a la frontera cambia de etiqueta. La figura agrega todas las configuraciones y debe leerse junto con la composición por kernel, porque las cargas no aportan el mismo número de intervalos.

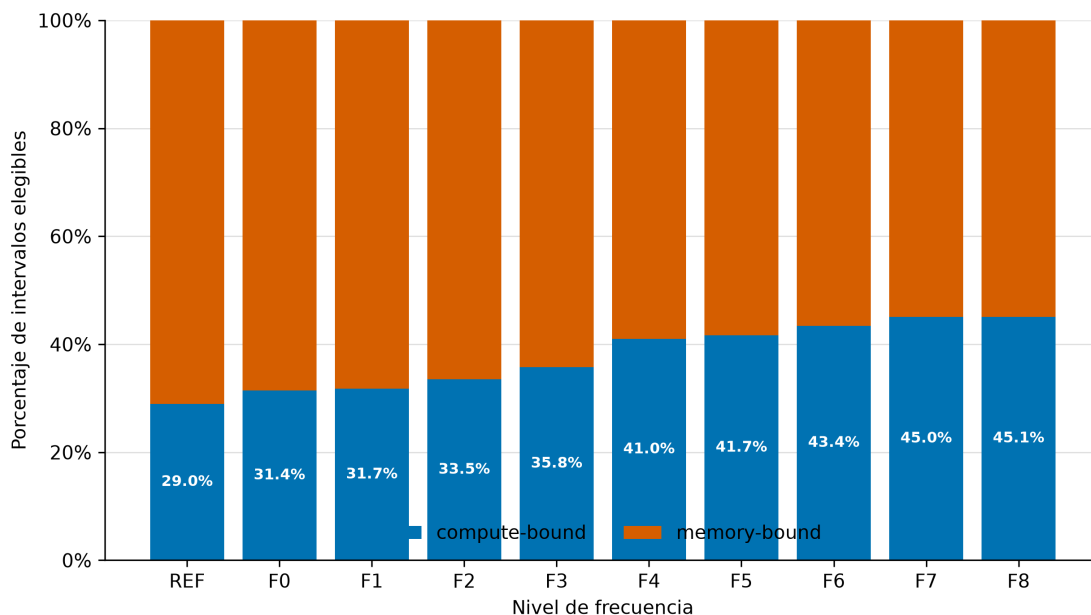


Figura 3.2: Porcentaje de intervalos elegibles clasificados *compute_bound* y *memory_bound* en cada nivel de frecuencia de la campaña CPU. Fuente: Elaboración propia.

3.1.3. Actuación DVFS, telemetría y techos Roofline de CPU

La actuación de frecuencia por núcleo se verifica en tres niveles, no solo por el código de retorno de la escritura. Primero, la escritura restringe siempre, junto con cada procesador lógico delegado, a sus hermanos de *SMT* según la topología real de la plataforma; de lo contrario los hermanos sin restringir permiten que el planificador entregue ciclos adicionales al núcleo físico compartido y se diluye el límite solicitado sobre el procesador delegado. Segundo, la confirmación de que el nuevo límite se hizo efectivo se realiza generando actividad de cómputo real sobre los mismos procesadores mientras se lee el reloj reportado por el sistema operativo, porque esa lectura deriva de la proporción entre ciclos reales y de referencia acumulados y solo es representativa bajo carga activa. Tercero, la verificación por ventana excluye un margen de ~ 10 ms inmediatamente posterior a cada transición de nivel, correspondiente

al tiempo físico de asentamiento del reloj entregado.

Verificado experimentalmente, al restringir únicamente los procesadores lógicos delegados la razón de rendimiento entre el nivel más alto y el más bajo del barrido se mantiene en 0.995 (sin escalar); al restringir además sus hermanos de *SMT*, la razón sube a 3.78, cercana a la razón de cuatro esperada entre los niveles comparados. El ancho de banda calibrado, limitado por memoria y no por reloj de núcleo, no depende de esta verificación: 58.0 GB/s bajo el nivel más alto del barrido frente a 57.45 GB/s en estado nativo sin restricción, dentro del margen de ruido de medición.

La campaña final exige turbo desactivado como precondition de cada corrida y valida la frecuencia observada en cada ventana con una tolerancia de $\pm 5\%$ respecto del nivel solicitado, con márgenes de gracia de 0.15 s al inicio y al final. La aceptación completa de la matriz confirma que cada corrida conservó una observación de frecuencia válida o, en el nivel nativo, el estado no aplicable a una frecuencia fija. La etiqueta se obtuvo comparando los FLOPs medidos con los bytes de DRAM observados por *uncore* contra el *ridge* calibrado para cada nivel. El *ridge* calibrado toma los valores 8.63, 8.20, 7.31, 6.58, 5.74, 5.03, 4.33, 3.64 y 2.92 FLOP/byte para F0 a F8 y 8.69 FLOP/byte para el nivel nativo.

Medición directa de bytes movidos en memoria (*uncore*)

La lectura de los contadores de *uncore* del controlador de memoria (sección 1.1.3) usa la capacidad CAP_PERFMON asignada al binario `perf`, sin reducir la restricción de `perf_event Paranoid` para todo el sistema. El instrumento invoca a `perf` como subproceso, fijado a un núcleo reservado para telemetría (nunca a los núcleos donde corre la carga medida, para no contaminar los contadores por proceso que allí se miden), y procesa en vivo su salida periódica en vez de abrir los contadores directamente. El presupuesto simultáneo de eventos de PMU es de nueve, no diez: la vigilancia por interrupción no enmascarable del sistema operativo, ajena a este proyecto y presente en cualquier despliegue de servidor típico, reserva de forma permanente un contador de hardware por núcleo; el evento descartado es el *proxy* de tráfico de caché de segundo nivel, sin papel en la clasificación desde que *uncore* lo reemplaza como fuente admitida.

La medición directa de *uncore* reemplaza por completo al *proxy* de fallos de caché en la clasificación, como única fuente admitida, aunque no a la granularidad fina de una ventana individual: `perf stat` reporta en intervalos de al menos ~ 10 ms, mientras que una ventana de CPU dura ~ 1 ms. El instrumento agrega los FLOPs de todas las ventanas cubiertas por cada intervalo de *uncore* y calcula la intensidad operacional real a esa granularidad más gruesa, difundida a cada una de esas ventanas como `operational_intensity` y `phase_label_train`.

Una ventana que ningún intervalo real llega a cubrir queda deliberadamente indefinida y nunca recurre al *proxy* como alternativa; `bytes_moved_window` (el *proxy*) se sigue calculando en su propia columna, únicamente como referencia de comparación.

Verificado de extremo a extremo con una carga de tráfico de memoria real, de 332 ventanas de una corrida de referencia, 322 quedaron cubiertas por al menos un intervalo real de *uncore*, con conteos de lectura y escritura variables y coherentes entre intervalos consecutivos, nunca en cero, la señal esperada de una medición genuina bajo carga activa. De esas 322, 301 quedaron clasificadas con el valor real; las 21 restantes tenían cobertura de bytes pero al menos una ventana del grupo sin FLOPs medidos válidos, y quedaron indefinidas en vez de calcular una intensidad con un numerador incompleto.

Validación de esfuerzo y aislamiento de la campaña

Antes de comprometer una campaña real de varias horas se ejecutó una prueba de esfuerzo dirigida a las condiciones que una corrida aislada no expone: cinco repeticiones consecutivas del patrón de arranque y cierre del subproceso de medición, una carga de duración extrema por debajo del piso práctico de intervalo, y una carga más prolongada para observar la cadencia de intervalos de principio a fin. Las cinco repeticiones completaron con datos reales y sin procesos remanentes, y el espaciado mínimo entre intervalos consecutivos se sostuvo en los ~ 10 ms esperados a lo largo de toda la corrida, incluidos sus primeros instantes.

Los contadores de *uncore*, de alcance de sistema, exigen reserva exclusiva del nodo, verificada como precondition bloqueante antes de iniciar cualquier campaña que los solicite, para que ningún proceso ajeno pueda contaminar una lectura que, por diseño, no puede atribuirse a un proceso concreto. Una prueba de humo a escala reducida ejercita el mismo camino de código de principio a fin (verificación previa, calibración, escritura de frecuencia, *uncore*, posprocesamiento y validación de ventanas) sobre una única carga, antes de repetir el procedimiento contra la campaña completa.

Con dos campañas de humo completas, la razón mínima de tiempo contando sobre tiempo habilitado se sostuvo en 1.0 a lo largo de toda una corrida de prueba, sin ningún delta negativo en los cuatro sub-eventos de punto flotante; una campaña con barrido de frecuencia (dieciocho combinaciones) aceptó la totalidad, y una segunda, sobre siete kernels reales del catálogo a frecuencia nativa (veintiuna combinaciones), aceptó también la totalidad con clasificaciones físicamente coherentes: la multiplicación densa de matrices resultó *compute-bound* en el 100 % de sus ventanas válidas, con intensidad operacional entre 15.0 y 29.6 FLOP/byte, muy por encima del punto de inflexión (sección 3.1.3); los seis kernels de la suite NAS Parallel Benchmarks resultaron predominantemente *memory-bound*, con una fracción minoritaria

de ventanas *compute-bound* en el kernel de resolución de sistemas por métodos implícitos, consistente con el desplazamiento del punto de inflexión.

Validación cruzada de los techos Roofline con una herramienta independiente

Los techos del modelo Roofline se determinan, como se explicó en la sección 1.1.2, mediante calibración empírica sobre la propia plataforma. Verificar el resultado de esa calibración exige una vía independiente del mecanismo de calibración: se empleó Intel Advisor, que construye su propio análisis Roofline mediante instrumentación binaria directa del bucle más costoso de un programa, contabilizando FLOPs y bytes ejecutados realmente sin pasar por contadores de PMU, un mecanismo de medición distinto y por tanto una validación cruzada real [31].

Los microbenchmarks de calibración solicitan explícitamente el ancho vectorial máximo de la plataforma (AVX-512, ocho elementos de doble precisión por instrucción) en el microbenchmark de densidad aritmética, y no en el de ancho de banda, limitado por memoria y no por ejecución vectorial, donde el ancho vectorial máximo produce una regresión de rendimiento medible. El mismo criterio se aplica a la cadena de construcción de los kernels de la suite NAS Parallel Benchmarks, sin la bandera de ancho vectorial máximo, por combinar regímenes *memory-bound* y *compute-bound* bajo una configuración de compilación compartida.

Contrastado sobre una multiplicación de matrices densas de referencia, Advisor midió 577.7 GFLOP/s con una intensidad operacional de 1.091 FLOP/byte sobre los mismos seis núcleos de la calibración de producción, coherente con el punto de inflexión propio del catálogo. El ancho de banda calibrado valida contra Advisor dentro de 12 a 15 % (58.4 a 58.8 GB/s frente a 64.0 a 66.8 GB/s), y el techo de cómputo calibrado por este instrumento (417.8 a 555 GFLOP/s) es consistente con la referencia independiente de Advisor. El punto de inflexión del nivel nativo queda en el rango de **7.0 a 9.3 FLOP/byte**, coherente con el valor de 8.69 FLOP/byte usado por la campaña final en ese nivel, y desciende con el reloj de núcleo hasta 2.92 FLOP/byte en F8 al bajar el techo de cómputo mientras el ancho de banda permanece limitado por memoria.

3.1.4. Kernels CPU de patrón de acceso irregular

El plan de trabajo exige, junto a los kernels de acceso regular (*dual_**), kernels de patrón de acceso irregular de la suite Rodinia. Se compilieron cuatro binarios de CPU (*kmeans*, *srad*, *nw* y *particlefilter*) con *scripts* de construcción reproducibles que fijan el *commit* de origen y el *hash* de cada archivo fuente; los cuatro resultaron reproducibles a nivel de binario, con el

mismo *hash sha256* en dos compilaciones independientes. Los cuatro se calibraron al piso de duración de corrida de la campaña (~ 25 a 30 s de ventana de estado estable); la Tabla 3.2 resume sus parámetros finales. La compilación de *nw* (Needleman–Wunsch) requirió retipar a *long* las dos variables de dimensión del código original, cuyo producto desborda enteros de 32 bits para $N \gtrsim 46\,000$, sin otro cambio al algoritmo y sin diferencia de rendimiento medible frente al binario original ($N = 46\,000$: 3.72 s original frente a 3.68 s parcheado).

Tabla 3.2: Parámetros finales calibrados de los cuatro kernels CPU de patrón de acceso irregular.

Kernel	Parámetros finales	Tiempo
<i>kmeans_omp</i>	1 000 000 puntos $\times$ 32 <i>features</i> , 6 hilos	31.4 s
<i>srad_omp</i>	6144 $\times$ 6144, 100 iteraciones, 6 hilos	27.5 s
<i>nw_omp</i>	$N = 122\,000$, <i>penalty</i> =10, 6 hilos	25.2 s
<i>particlefilter_omp</i>	212 000 partículas, 128 $\times$ 128 $\times$ 10, 6 hilos	28.3 s

Tres de estos kernels (*srad*, *particlefilter* y *kmeans*) se midieron en una campaña independiente (*pacca_cpu_new_families_20260918_r2*: nivel nativo y nueve niveles fijos, tres repeticiones, 90 corridas aceptadas) sobre el mismo nodo y la misma CPU de la campaña final, con turbo desactivado. La Tabla 3.3 resume los intervalos que superan el filtro de calidad. Los tres resultaron casi puramente *memory-bound*, de modo que amplían la diversidad de algoritmos de esa clase pero no aportan ejemplos *compute-bound*. *particlefilter* conserva frecuencia válida en todos los niveles; *kmeans* y *srad* sostienen la frecuencia solicitada solo desde F5 (y F4 en *srad*) hasta F8 y en el nivel nativo, de modo que en los niveles de mayor frecuencia casi todas sus ventanas quedan rechazadas por frecuencia no verificable. *nw* está compilado y calibrado, pero no se ha medido.

Tabla 3.3: Intervalos de las familias CPU nuevas tras el filtro de calidad (campaña *pacca_cpu_new_families_20260918_r2*).

Familia	Intervalos totales	Elegibles	<i>compute_bound</i>	<i>memory_bound</i>
<i>rodinia_kmeans_omp</i>	150 379	78 413 (52.1 %)	0	78 413
<i>rodinia_particlefilter_omp</i>	66 406	65 798 (99.1 %)	1 025	64 773
<i>rodinia_srad_omp</i>	135 615	72 800 (53.7 %)	0	72 800

3.1.5. Clasificador de fase en CPU

Esta sección reporta la configuración final del clasificador de CPU y su desempeño. El procedimiento está descrito en las secciones 2.4.1 a 2.4.5. Cuando una decisión de diseño se tomó comparando alternativas se reporta la comparación completa, no solo la opción elegida, porque en varios casos el resultado relevante es que las alternativas no se distinguen entre sí.

Qué mide la evaluación

El conjunto contiene 30 familias algorítmicas. Un *pliegue* consiste en retirar una familia completa, ajustar el modelo con las 29 restantes y clasificar después los intervalos de la familia retirada. El procedimiento se repite 30 veces, una por familia, y ningún intervalo de la familia evaluada interviene en el ajuste ni en ninguna decisión de diseño. Este esquema responde a la pregunta operativa pertinente: si el agente encuentra un algoritmo que nunca vio, ¿acierta su régimen?

La pregunta alternativa, más débil, es si el modelo acierta sobre otra ejecución de un algoritmo ya conocido. La diferencia entre ambas no es menor y se midió con el mismo modelo y los mismos datos (Tabla 3.4). Repartir intervalos al azar produce 0.877, porque dos intervalos consecutivos de una misma corrida son casi idénticos y el modelo reconoce la carga en lugar de su régimen. Dejar fuera un kernel completo baja a 0.732 y dejar fuera una familia completa, a 0.728. Todas las cifras de este capítulo corresponden al último esquema.

Tabla 3.4: Efecto del esquema de partición sobre el resultado, con el mismo modelo y los mismos datos.

Esquema de partición	Qué mide	Exactitud balanceada
Intervalos repartidos al azar (80/20)	Memorización de la carga	0.877 ± 0.013
Dejar un kernel fuera (49 pliegues)	Otro tamaño o variante	0.732 ± 0.010
Dejar una familia fuera (30 pliegues)	Algoritmo nunca visto	0.728 ± 0.009

El modelo se ajusta con la muestra estratificada de 42 455 intervalos descrita en la Tabla 3.5 y se evalúa sobre *todos* los intervalos elegibles de la familia retirada. El tope de 1000 intervalos por celda de familia y clase responde al costo de la validación anidada, y se verificó que no descarta información útil: al repetir la evaluación con topes entre 100 y 30 000, es decir, entre 4725 y 722 492 intervalos de ajuste, el resultado se mantiene entre 0.724 y 0.732 sin tendencia creciente (Tabla 3.7). Un millón de intervalos no equivale a un millón de ejemplos independientes: son observaciones repetidas de 30 algoritmos.

Tabla 3.5: Flujo de observaciones desde la campaña CPU hasta la evaluación del modelo.

Etapas	Intervalos	Cálculo o criterio
Intervalos <i>uncore</i> procesados	1 424 531	Salida total del reprocesamiento de 1470 corridas
Intervalos elegibles	1 309 755	Estado de entrenamiento correcto y frecuencia utilizable
Intervalos rechazados	114 776	Diferencia entre procesados y elegibles, equivalente a 8.1 %
Intervalos evaluados	1 296 981	Elegibles con todas las tasas definidas (se excluyen 12 774 con contadores en cero)
Muestra reproducible	42 455	Suma de $\min(n_{fc}, 1000)$ para cada celda observada
Clase <i>compute-bound</i> en la muestra	21 350	Conteo posterior al límite por familia y clase
Clase <i>memory-bound</i> en la muestra	21 105	Conteo posterior al límite por familia y clase

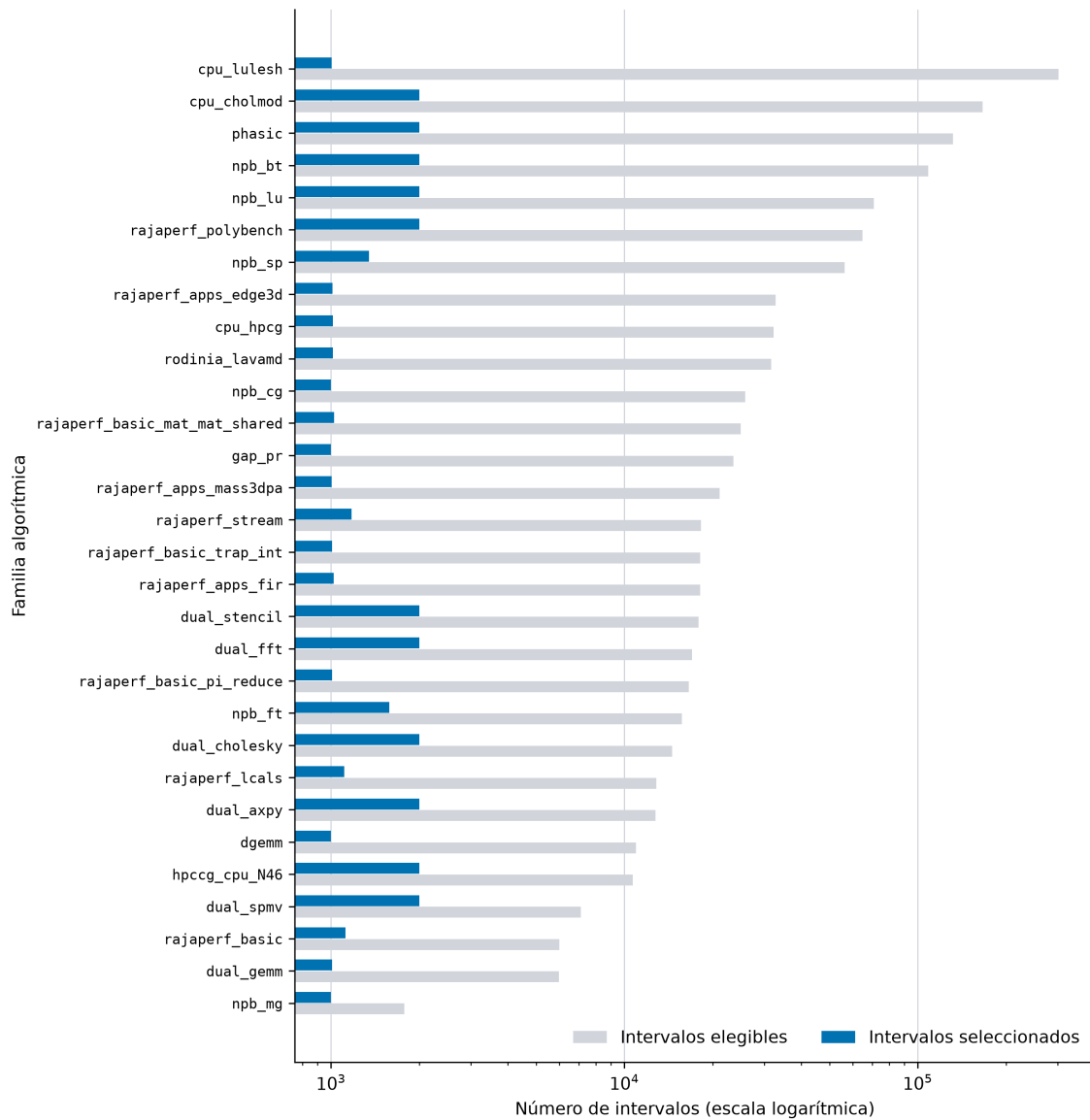


Tabla 3.6: Intervalos elegibles y seleccionados por familia y clase.

Familia	Elegibles		Seleccionados	
	Cómputo	Memoria	Cómputo	Memoria
cpu_cholmod	55 323	110 991	1000	1000
cpu_hpcg	15	32 230	15	1000

Familia	Elegibles		Seleccionados	
	Cómputo	Memoria	Cómputo	Memoria
cpu_lulesh	5	302 135	5	1000
dgemm	10 938	0	1000	0
dual_axpy	5219	7554	1000	1000
dual_cholesky	9469	5074	1000	1000
dual_fft	7240	9757	1000	1000
dual_gemm	5981	9	1000	9
dual_spmv	4040	3060	1000	1000
dual_stencil	9894	8005	1000	1000
gap_pr	0	23 542	0	1000
hpccg_cpu_N46	8767	1899	1000	1000
npb_bt	68 397	40 208	1000	1000
npb_cg	0	25 770	0	1000
npb_ft	582	15 097	582	1000
npb_lu	21 194	49 563	1000	1000
npb_mg	0	1778	0	1000
npb_sp	346	55 927	346	1000
phasic	89 136	42 787	1000	1000
rajaperf_apps_edge3d	32 750	11	1000	11
rajaperf_apps_fir	18 102	22	1000	22
rajaperf_apps_mass3dpa	21 108	6	1000	6
rajaperf_basic	118	5874	118	1000
rajaperf_basic_mat_mat_shared	24 891	25	1000	25
rajaperf_basic_pi_reduce	16 551	8	1000	8
rajaperf_basic_trap_int	18 116	9	1000	9
rajaperf_lcal	109	12 746	109	1000
rajaperf_polybench	53 398	11 322	1000	1000
rajaperf_stream	175	18 049	175	1000
rodinia_lavamd	31 644	15	1000	15

Tabla 3.7: Sensibilidad del resultado al tope de intervalos por celda de familia y clase.

Tope por celda	Intervalos de ajuste	Exactitud balanceada
100	4 725	0.724
300	13 427	0.728
1000	42 455	0.725
3000	122 132	0.731
10 000	355 066	0.724
30 000	722 492	0.732

Métricas de evaluación

El modelo emite una clase para cada uno de los 1 296 981 intervalos evaluados. Resumir esas decisiones en una cifra exige promediar, y cada forma de promediar responde a una pregunta distinta. Esta sección define las métricas empleadas, explica cuál se adopta como principal y por qué, y muestra que las demás cuentan una historia compatible.

Qué es una celda. Una *celda* es la combinación de una familia algorítmica y una clase de fase. Por ejemplo, los intervalos *compute-bound* de `cpu_cholmod` forman una celda y sus intervalos *memory-bound* forman otra. El conjunto tiene 56 celdas: 30 familias por dos clases son 60 combinaciones posibles, de las cuales cuatro están vacías porque esa familia nunca presenta esa clase (por ejemplo, `dgemm` no tiene intervalos *memory-bound*). Cada celda tiene su propio tamaño, que varía desde 5 hasta más de 300 000 intervalos, y su propia fracción de aciertos. La Figura 3.4 muestra las 56 celdas del modelo final con su acierto y su tamaño.

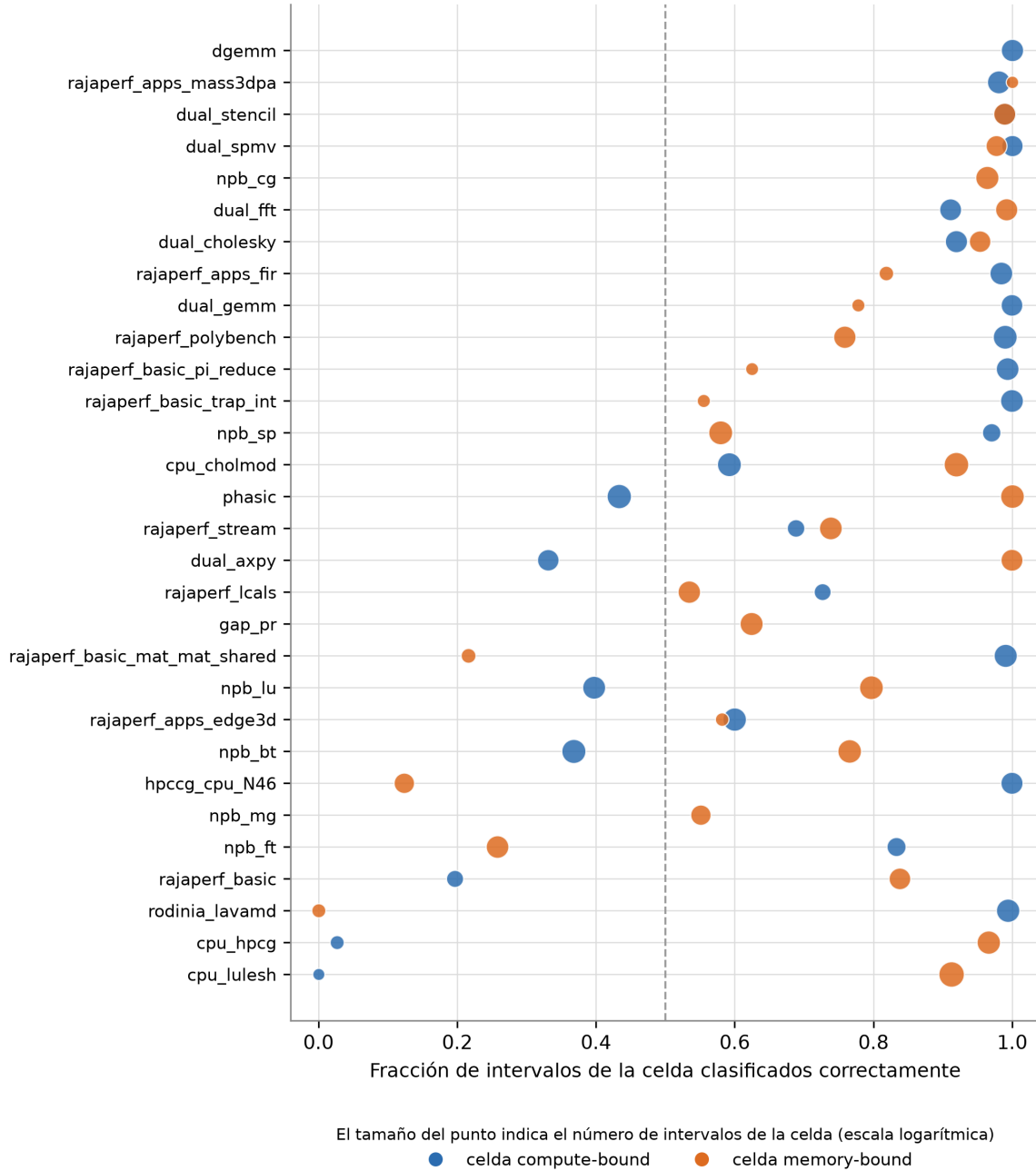


Figura 3.4: Fracción de aciertos del modelo final en cada una de las 56 celdas de familia y clase, con evaluación sobre todos los intervalos de la familia retirada. Las familias con una sola celda son las de una sola clase. Nota. Elaboración propia.

Métricas empleadas. La Tabla 3.8 define las nueve métricas. Para una clase c , VP_c son los intervalos de esa clase correctamente clasificados, FP_c los intervalos de otra clase clasificados como c y FN_c los intervalos de c clasificados como otra clase. Las ocho primeras agrupan

todos los intervalos y por ello pesan cada familia según su volumen. La última pesa cada celda por igual.

Tabla 3.8: Métricas de evaluación empleadas y pregunta que responde cada una.

Métrica	Definición	Pregunta que responde y límite
Exactitud	$\frac{\sum_c VP_c}{N}$	¿Qué fracción de todos los intervalos se clasifica bien? Premia predecir la clase mayoritaria y la dominan las familias con más intervalos
Precisión de la clase c	$\frac{VP_c}{VP_c + FP_c}$	Cuando el modelo dice c , ¿cuántas veces acierta?
Sensibilidad de la clase c	$\frac{VP_c}{VP_c + FN_c}$	De los intervalos que son c , ¿cuántos detecta?
F1 de la clase c	$\frac{2 P_c S_c}{P_c + S_c}$	Media armónica de precisión y sensibilidad de esa clase
F1 macro agrupada	$\frac{1}{2}(F1_{\text{compute}} + F1_{\text{memory}})$	Promedio de las F1 de las dos clases, calculadas sobre todos los intervalos concatenados
Exactitud balanceada clásica	$\frac{1}{2}(S_{\text{compute}} + S_{\text{memory}})$	Promedio de las dos sensibilidades; corrige el desbalance entre clases pero no el volumen entre familias
AUC-ROC	Área bajo la curva sensibilidad frente a tasa de falsos positivos	¿Ordena el modelo los intervalos <i>memory-bound</i> por encima de los <i>compute-bound</i> ? No depende del umbral de decisión
Coeficiente de Matthews (MCC)	Correlación de Pearson entre la clase predicha y la clase real	Vale 0 para un modelo sin información y 1 para uno perfecto; usa las cuatro casillas de la matriz de confusión
Exactitud balanceada por celda	$\frac{1}{ \mathcal{C} } \sum_{(f,c) \in \mathcal{C}} \frac{a_{fc}}{n_{fc}}$	¿Con qué fracción de aciertos se resuelve una celda típica de familia y clase? Ecuación (2.13)

Un ejemplo con cifras reales. La Tabla 3.9 toma cuatro celdas del modelo final. La exactitud, que pesa por intervalos, da 0.876: casi todos los intervalos pertenecen a `cpu_lulesh` *memory-bound* y se clasifican bien. La exactitud por celda da 0.606, porque cada celda cuenta una vez, y muestra que el modelo falla por completo en la celda *compute-bound* de `cpu_lulesh` y solo acierta 59% en la de `cpu_cholmod`. Ambas cifras son correctas y responden preguntas diferentes: la primera dice qué fracción del tráfico de intervalos se clasifica bien; la segunda, qué tan bien se resuelve cada situación distinta.

Tabla 3.9: Ejemplo de cálculo con cuatro celdas reales del modelo final.

Familia	Clase de la celda	Intervalos	Aciertos	Fracción
cpu_cholmod	<i>compute-bound</i>	55 323	32 760	0.592
cpu_cholmod	<i>memory-bound</i>	110 991	102 020	0.919
cpu_lulesh	<i>compute-bound</i>	5	0	0.000
cpu_lulesh	<i>memory-bound</i>	302 135	275 583	0.912
Exactitud por intervalos (suma de aciertos entre suma de intervalos)		468 454	410 363	0.876
Exactitud por celda (promedio de las cuatro fracciones)				0.606

Por qué la exactitud por celda es la métrica principal. Hay tres razones estructurales. No depende del volumen: una corrida larga no decide el resultado. No depende de la proporción entre clases dentro de una familia. Y está definida también para las cuatro familias que contienen una sola clase (*dgemm*, *gap_pr*, *npb_cg* y *npb_mg*), mientras que el F1 macro calculado dentro de cada familia queda acotado a 0.5 en ellas aunque todos sus intervalos se clasifiquen bien.

Hay además una razón empírica, de alcance más modesto: la métrica por celda es la que menos depende de qué intervalos se muestrean. Al repetir la evaluación completa con cinco semillas de muestreo distintas, su desviación es 0.009, del mismo orden que la de las métricas por volumen, que varían entre 0.002 y 0.011 (Tabla 3.10). Sí se mantiene una diferencia en el intervalo de confianza por remuestreo de las 30 familias, cuya amplitud es 0.133 para la exactitud por celda frente a 0.153 para la exactitud global y 0.173 para la exactitud balanceada clásica. Las métricas por volumen dependen de un puñado de familias con cientos de miles de intervalos, entre ellas *cpu_lulesh*, *phasic*, *cpu_cholmod* y *npb_bt*, y por eso su intervalo es más ancho.

Tabla 3.10: Valores de todas las métricas para el modelo final, sin abstención. Media de cinco semillas de muestreo, desviación entre semillas e intervalo de confianza del 95 % por remuestreo de las 30 familias.

Métrica	Valor	Desv. semillas	IC95 familias
Exactitud	0.794	0.006	0.709 a 0.863
Precisión <i>compute-bound</i>	0.756	0.012	–
Sensibilidad <i>compute-bound</i>	0.709	0.005	–
F1 <i>compute-bound</i>	0.732	0.005	–
Precisión <i>memory-bound</i>	0.817	0.002	–
Sensibilidad <i>memory-bound</i>	0.850	0.011	–
F1 <i>memory-bound</i>	0.833	0.005	–
F1 macro agrupada	0.782	0.005	–
Exactitud balanceada clásica	0.779	0.004	0.698 a 0.871
AUC-ROC agrupada	0.855	0.003	–
Coefficiente de Matthews	0.566	0.011	–
Exactitud balanceada por celda	0.727	0.009	0.670 a 0.803

Los intervalos de las métricas por volumen se calcularon con la primera semilla de muestreo y el de la exactitud por celda con el promedio de las cinco, de modo que solo las amplitudes son comparables entre sí, no los extremos.

Lectura conjunta. Las métricas cuentan una historia compatible en lo esencial, con una diferencia que conviene explicar. La exactitud balanceada clásica (0.779) y la F1 macro agrupada (0.782) coinciden entre sí, y la exactitud por celda (0.727) queda unos cinco puntos por debajo porque, al pesar cada celda por igual, castiga las celdas de muy pocos intervalos donde el modelo falla (véase el límite descrito más abajo). El AUC-ROC de 0.855 indica que el modelo ordena los intervalos mejor que el azar con margen apreciable, y el coeficiente de Matthews de 0.566 confirma una correlación moderada a alta entre predicción y clase real. La exactitud global de 0.794 es la más alta de todas, pero es la que más depende de la composición del conjunto y no debe leerse como la capacidad del modelo.

La asimetría entre clases también depende de cómo se promedia. Por volumen, la sensibilidad es 0.709 para *compute-bound* y 0.850 para *memory-bound*. Por celdas, con cada celda pesando igual, es 0.738 y 0.718: por volumen la clase *compute* es la más débil, por celdas no. La diferencia se debe a que las clases raras dentro de familias casi puras (celdas de menos de 30 intervalos) son en su mayoría *memory-bound* y varias se reconocen mal, mientras que las

familias con más intervalos *compute-bound*, como `npb_bt`, `phasic` y `cpu_cholmod`, se clasifican con 0.37 a 0.59 de acierto en esa clase. La asimetría es real pero cambia de signo según el promedio.

Límite de la exactitud por celda. La métrica asigna el mismo peso a una celda de 5 intervalos que a una de 300 000. Diez de las 56 celdas tienen menos de 100 intervalos evaluados: *compute-bound* de `cpu_lulesh` (5) y de `cpu_hpcg` (15), *memory-bound* de `dual_gemm` (9), de `rodinia_lavamd` (15) y de las seis familias de cómputo añadidas (entre 6 y 25 intervalos cada una). Suman 125 intervalos y pesan, sin embargo, 17.9% de la métrica. Excluir las subiría la métrica de 0.727 a 0.785. Se conservan por dos razones: retirar celdas después de ver que fallan sería seleccionar datos por el resultado, y esos casos son informativos, porque muestran que las clases raras dentro de una familia casi pura no se reconocen. Pero implican que parte de la incertidumbre de la métrica proviene de celdas muy pequeñas, y que el intervalo de confianza debe leerse con esa cautela.

Incetidumbre. Se informan dos fuentes. La semilla del submuestreo se varió cinco veces. La composición del catálogo se evaluó mediante remuestreo con reemplazo de las 30 familias, con 2000 réplicas. Esta segunda fuente domina y es la que debe usarse al comparar configuraciones: con 30 familias, cualquier diferencia menor que 0.03 es indistinguible del ruido.

Variables de entrada y tratamiento de la correlación

El modelo final usa las 12 variables de la Tabla 3.11. Las seis primeras son tasas normalizadas y la frecuencia física observada; las seis restantes son transformaciones de los mismos contadores.

Tabla 3.11: Variables de entrada del candidato CPU congelado.

Variable	Cálculo	Interpretación
ipc	$N_{\text{inst}}/N_{\text{cyc}}$	Trabajo retirado por ciclo
mpki	$1000N_{\text{miss}}/N_{\text{inst}}$	Fallos de caché por millar de instrucciones
cache_miss_rate	$N_{\text{miss}}/N_{\text{ref}}$	Fracción de referencias que fallan
stall_mem_ratio	$N_{\text{stall}}/N_{\text{cyc}}$	Fracción de ciclos detenidos por memoria
ips	$N_{\text{inst}}/\Delta t$	Tasa de instrucciones del intervalo
freq_khz_observed	Mediana de las lecturas cubiertas	Frecuencia física observada
cache_references_per_ki	$1000N_{\text{ref}}/N_{\text{inst}}$	Presión de referencias por instrucción
cache_references_per_cycle	$N_{\text{ref}}/N_{\text{cyc}}$	Presión de referencias por ciclo
cache_misses_per_cycle	$N_{\text{miss}}/N_{\text{cyc}}$	Fallos observados por ciclo
stalls_mem_per_ki	$1000N_{\text{stall}}/N_{\text{inst}}$	Espera de memoria normalizada por trabajo
stalls_mem_per_cache_miss	$N_{\text{stall}}/N_{\text{miss}}$	Ciclos detenidos por cada fallo observado
log1p_mpki	$\log(1 + \text{MPKI})$	Compresión monotonica de la cola de MPKI

Cuatro grupos de columnas quedaron fuera, por razones distintas. El primero son los insumos de la etiqueta: FLOPs, bytes de DRAM medidos por *uncore*, intensidad operacional y *ridge*. Con ellos el modelo no predice la etiqueta, la calcula; el código aborta si alguno aparece en el vector de entrada. El segundo son los identificadores de kernel, nivel solicitado y repetición, que permiten reconocer la carga en lugar de su régimen. El tercero son las señales de calidad del contador (`delta_enabled_ns`, `delta_running_ns` y `running_ratio`), que describen la adquisición y no el comportamiento de la carga, y que se usan solo como criterio de aceptación de filas. El cuarto son los deltas crudos de instrucciones, ciclos, referencias y fallos, que dependen de la duración del intervalo y que la auditoría de correlación marcó como redundantes frente a sus tasas (Tabla 3.12).

Tabla 3.12: Pares marcados por la auditoría de correlación sobre el conjunto CPU elegible.

Primera columna	Segunda columna	Pearson	Spearman
delta_instructions	ips	0.9973	0.9986
delta_cycles	freq_khz_observed	0.9225	0.9201
delta_stalled_cycles_ mem_any	stall_mem_ratio	0.8802	0.9437
delta_running_ns	delta_enabled_ns	1.0000	1.0000
delta_cache_misses	mpki	0.0014	0.8647

Tabla 3.13: Factor de inflación de la varianza posterior al filtro de correlación.

Columna	VIF	Columna	VIF
delta_instructions	5.954666	ipc	5.677144
delta_cache_references	4.979069	cache_miss_rate	2.017495
delta_cache_misses	5.852083	stall_mem_ratio	2.684735
delta_enabled_ns	1.169377	freq_khz_observed	3.345568
running_ratio	1.000004		

Por qué permanecen variables correlacionadas entre sí. Entre las 12 entradas finales quedan pares con correlación alta. El caso extremo es $\log(1 + \text{MPKI})$, transformación monótona de MPKI y por tanto con correlación de rangos igual a uno con ella; el par formado por MPKI y las referencias de caché por millar de instrucciones alcanza 0.892 de correlación lineal (Figura 3.5). No se retiraron, por tres razones.

Primero, el criterio de colinealidad protege la interpretación de los coeficientes de un modelo lineal, no la capacidad predictiva de un ensamble de árboles, que es invariante a transformaciones monótonas de una variable y no exige columnas mutuamente independientes. Segundo, el factor de inflación de la varianza calculado tras el filtro se mantuvo entre 1.00 y 5.95, por debajo del umbral de alerta de 10 fijado de antemano. Tercero, y de forma decisiva, se comprobó que la redundancia no cambia el resultado: ajustar el modelo solo con las seis tasas base produce 0.738 y hacerlo con las doce produce 0.727, una diferencia de $-0,011$ (a favor de las seis) cuyo intervalo de confianza va de $-0,028$ a $+0,003$ y contiene el cero (Tabla 3.14). Las seis transformaciones ni ayudan ni perjudican de forma medible, y su costo de inferencia es el mismo (mediana de 243 μs con seis o con doce variables y percentil 99 de 0.40 ms en ambos casos), de modo que retirarlas o conservarlas es indiferente. Se conservó el vector de doce variables por ser la configuración ya serializada y auditada.

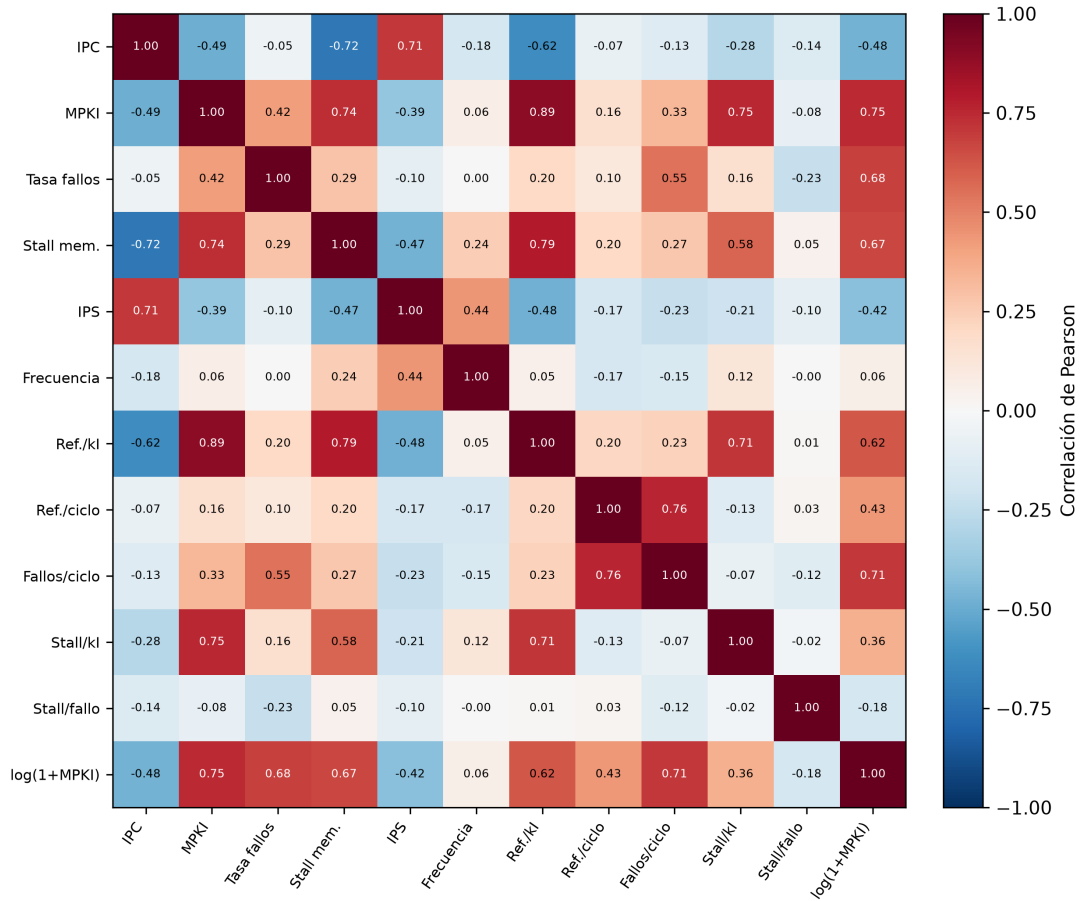


Figura 3.5: Correlación de Pearson entre las 12 variables del candidato congelado sobre la muestra estratificada de 42 455 intervalos. Nota. Elaboración propia.

Tabla 3.14: Variantes del vector de entrada bajo el mismo modelo y el mismo protocolo. La diferencia se calcula frente al vector de 12 variables.

Vector de entrada	Exact. bal.	F1 macro	Diferencia (IC95)
12 variables (configuración final)	0.727	0.782	referencia
6 tasas base	0.738	0.763	+0,011 (−0,003 a +0,027)
12 variables sin frecuencia	0.713	0.767	−0,014 (−0,027 a −0,003)
7 variables con <code>running_ratio</code>	0.738	0.763	+0,011 (−0,003 a +0,027)
Solo frecuencia observada	0.589	0.498	−0,138 (−0,217 a −0,064)

Esa misma tabla responde otra pregunta razonable sobre la frecuencia observada. Como el *ridge* depende de ella, cabía sospechar que el modelo la usara como atajo para deducir la etiqueta. No ocurre: un modelo que solo recibe la frecuencia alcanza 0.589, apenas por

encima del azar, y retirarla del vector completo reduce el resultado solo 0.014. La frecuencia aporta contexto, no la señal que decide.

La importancia por permutación, medida sobre la familia retirada, confirma la redundancia interna del vector: solo MPKI, con una caída de 0.031, y la tasa de fallos de caché, con 0.030, producen un deterioro apreciable al permutarlas, mientras que $\log(1 + \text{MPKI})$, la fracción de ciclos detenidos y las referencias por ciclo tienen aporte nulo o ligeramente negativo. En términos prácticos, el clasificador es un detector de presión sobre la jerarquía de memoria con contexto adicional, y las doce columnas contienen bastante menos información que doce variables independientes.

Selección de modelo e hiperparámetros

La comparación de modelos se ejecutó con hiperparámetros fijos, el mismo protocolo y las mismas cinco semillas de muestreo (Tabla 3.15 y Figura 3.6). El resultado central es que los modelos con capacidad suficiente quedan indistinguibles entre sí: desde la regresión logística con seis variables, con 0.683, hasta XGBoost con seis variables, con 0.738, todas las diferencias caen dentro del intervalo de confianza; la regresión logística con doce variables, con 0.662, queda en el extremo inferior. Solo se separan del grupo las dos referencias sin capacidad, la regla mayoritaria con 0.464 y el árbol de profundidad uno con 0.631. La diferencia grande entre modelos no está en la exactitud sino en la latencia: los dos bosques cuestan veinte veces más por inferencia que XGBoost o la regresión logística.

Tabla 3.15: Comparación de modelos con hiperparámetros fijos. La latencia corresponde a una fila, un hilo, medida en el nodo de destino.

Modelo	Exact. bal.	IC95	F1 macro	Latencia p99
Regla mayoritaria	0.464	0.423 a 0.492	0.205	–
Árbol de profundidad 1	0.631	0.567 a 0.701	0.705	102 μs
Regresión logística (6 var.)	0.683	0.619 a 0.747	0.766	–
Regresión logística (12 var.)	0.662	0.607 a 0.727	0.778	325 μs
Extra Trees (12 var.)	0.732	0.660 a 0.804	0.761	5367 μs
Random Forest (12 var.)	0.734	0.667 a 0.804	0.813	5337 μs
XGBoost (6 var.)	0.738	0.679 a 0.801	0.763	382 μs
XGBoost (12 var.)	0.727	0.665 a 0.794	0.782	383 μs

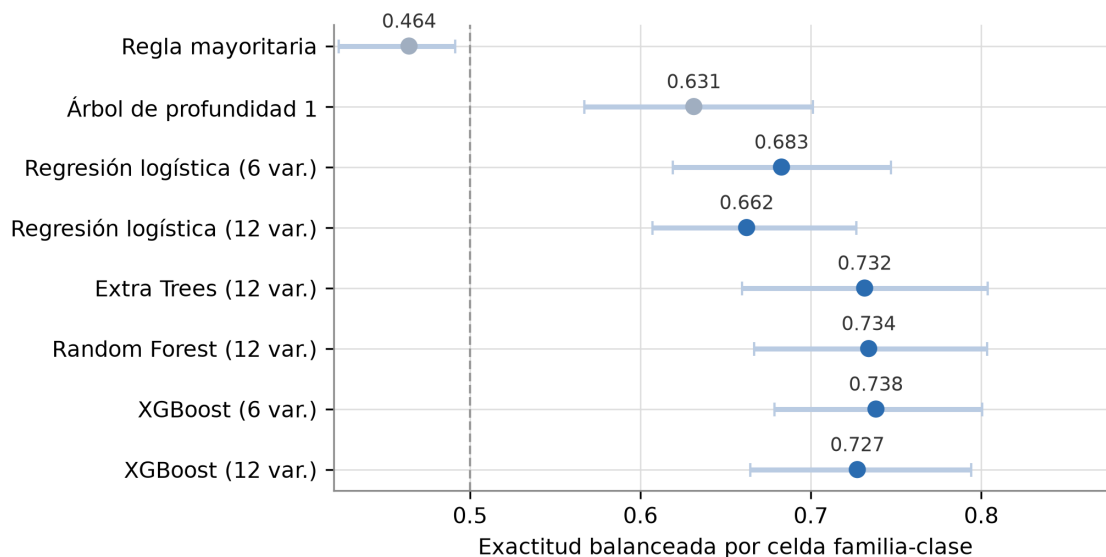


Figura 3.6: Exactitud balanceada por celda de familia y clase de cada modelo, con intervalo de confianza del 95 % obtenido por remuestreo de las 30 familias. Los intervalos se solapan entre todos los modelos con capacidad suficiente. Nota. Elaboración propia.

Búsqueda de hiperparámetros. Sobre esa base se ejecutó la búsqueda con el muestreador TPE de Optuna [1], que propone configuraciones nuevas a partir del desempeño de las ya evaluadas en lugar de recorrer una rejilla fija. La búsqueda se ejecuta *dentro* de cada pliegue externo: con la familia retirada apartada, se recorre un LOFO interno sobre las 29 familias restantes para puntuar cada configuración propuesta, se ajusta el modelo con la mejor y solo entonces se evalúa una vez sobre la familia retirada. Esta anidación es lo que impide que el resultado se infle, porque una búsqueda que puntúe sus configuraciones contra la misma familia que después se reporta escogería, entre muchas alternativas, la que mejor se ajusta a ese pliegue concreto. El espacio de búsqueda es el de la Tabla 3.16, con 15 pruebas por pliegue para XGBoost y 8 para la regresión logística, es decir, 450 y 240 configuraciones evaluadas en total.

Tabla 3.16: Espacio de búsqueda de hiperparámetros.

Modelo	Hiperparámetro	Rango
XGBoost	Número de árboles	50 a 300, en pasos de 25
	Profundidad máxima	2 a 10
	Tasa de aprendizaje	10^{-2} a 0.5, escala logarítmica
	Peso mínimo por hoja	1 a 10
Regresión logística	Regularización C	10^{-3} a 10^2 , escala logarítmica

El resultado es que la búsqueda no mejora la configuración fija, y de hecho la empeora ligeramente (Tabla 3.17). Para XGBoost, la configuración ajustada dentro de cada pliegue obtiene 0.696 frente a 0.736 de la configuración fija. Por familia, la búsqueda mejora el resultado en 10 de las 30 y lo empeora en 19; la diferencia media por familia es de $-0,029$, con un intervalo de confianza del 95 % de $-0,052$ a $-0,010$. Para la regresión logística la diferencia es de $+0,004$, indistinguible de cero.

Tabla 3.17: Búsqueda de hiperparámetros anidada frente a la configuración fija, con el mismo protocolo.

Configuración	Exact. bal.	F1 macro	Exactitud
XGBoost fijo	0.736	0.776	0.787
XGBoost con búsqueda TPE anidada	0.696	0.689	0.700
Regresión logística fija	0.664	0.779	0.792
Regresión logística con búsqueda TPE anidada	0.668	0.772	0.785

El motivo se ve en los datos de la propia búsqueda. El mejor puntaje interno de cada pliegue se mueve en un rango estrecho, de 0.727 a 0.759 con media 0.740, mientras el resultado externo del mismo modelo va de 0.295 a 1.000. Es decir, el criterio interno no distingue configuraciones buenas de malas para una familia nueva: escoger el máximo de 15 propuestas sobre un criterio con ese nivel de ruido equivale a ajustarse a los pliegues internos. Como consecuencia, las configuraciones elegidas cambian por completo entre pliegues, con profundidades entre 2 y 10, tasas de aprendizaje entre 0.031 y 0.473 y entre 50 y 300 árboles, sin que ninguna región del espacio se imponga. Restringir la complejidad tampoco cambia el panorama: con profundidades fijas de 2, 3, 4 y 6 el resultado va de 0.720 a 0.736, y elegir la profundidad dentro de cada pliegue produce 0.726.

Por este motivo se conservó la configuración fija de XGBoost, que además es la más simple de documentar y reproducir. La conclusión de fondo, coherente con la comparación de modelos y con las variantes del vector de entrada, es que el desempeño de este problema no está limitado por la capacidad del modelo ni por el ajuste de sus hiperparámetros.

Resultado del modelo final

La configuración congelada es la de la Tabla 3.18. Se evaluó sobre los 1 296 981 intervalos elegibles, cada uno clasificado por el modelo del pliegue que no vio su familia, promediando las cinco semillas de muestreo.

Tabla 3.18: Configuración final del clasificador CPU.

Elemento	Valor
Modelo	XGBoost, 100 árboles, profundidad máxima 6, objetivo <code>binary:logistic</code>
Entradas	12 variables PMU de la Tabla 3.11
Ponderación	Peso total igual por celda de familia y clase, ecuación (2.7), con <code>scale_pos_weight = 1</code>
Salida	Probabilidad de <i>memory-bound</i> ; clase con confianza $\geq 0,85$ y <i>revisar</i> por debajo
Semilla	20260918

Si el modelo debe emitir siempre una clase, la exactitud balanceada por celda es 0.727, la F1 macro agrupada 0.782 y la exactitud global 0.794. La Tabla 3.19 desagrega el resultado por clase y la Figura 3.7 muestra las matrices de confusión. El comportamiento es asimétrico: reconoce *memory-bound* con sensibilidad 0.850 y *compute-bound* con 0.709. Parte de esta asimetría depende de cómo se promedia (por volumen la clase *compute* es la más débil y por celdas no, como se explica en la sección 3.1.5); el resto proviene de que los intervalos *compute-bound* de una familia nueva se parecen a intervalos *memory-bound* de otras familias ya vistas, como se detalla en la sección 3.1.5. No proviene del desbalance del ajuste, que está corregido por la ponderación.

Tabla 3.19: Métricas por clase del modelo final sobre los 1 296 981 intervalos evaluados.

Clase	Intervalos	Precisión	Sensibilidad	F1
<i>compute-bound</i>	513 508	0.756	0.709	0.732
<i>memory-bound</i>	783 473	0.817	0.850	0.833

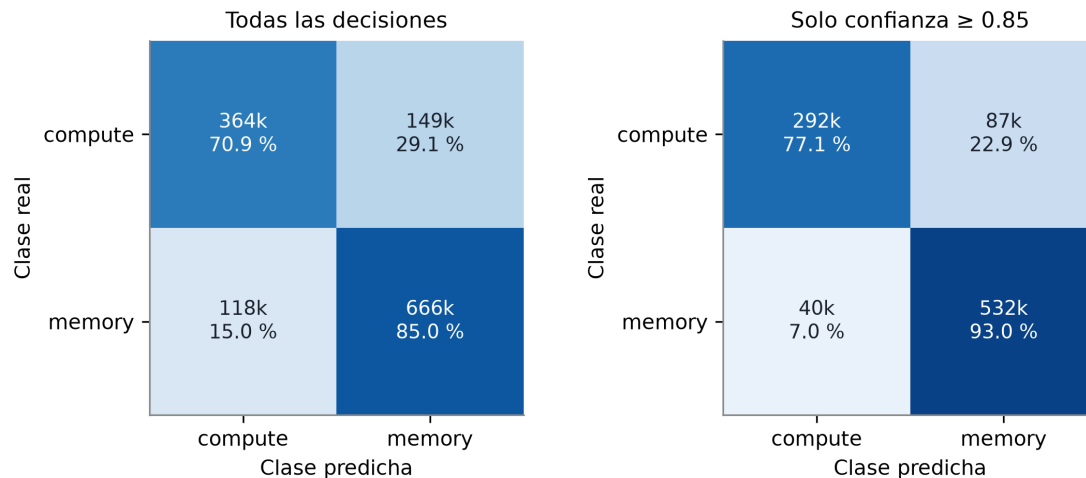


Figura 3.7: Matrices de confusión del modelo final. A la izquierda, cuando se obliga a emitir una clase en todos los intervalos; a la derecha, solo sobre los intervalos con confianza mayor o igual a 0.85. El porcentaje está calculado por fila, es decir, respecto de la clase real. Nota. Elaboración propia.

El resultado no es uniforme entre familias (Tabla 3.20 y Figura 3.8). Dieciséis familias superan 0.9 de exactitud y tres quedan por debajo de 0.55, con *npb_ft* en 0.279. Las familias con mejor resultado son las de comportamiento extremo y estable, mientras que las peores son aquellas cuyo perfil de contadores coincide con el de familias de la clase contraria.

Tabla 3.20: Resultado del modelo final por familia retirada, sobre todos sus intervalos elegibles.

Familia	Intervalos	Frac. memoria	Exactitud	Cobertura	Exact. c
dgemm	10 938	0.000	1.000	1.000	
rajaperf_basic_trap_int	18 125	0.001	1.000	0.994	
dual_gemm	5990	0.002	0.999	1.000	
rodinia_lavamd	31 659	0.001	0.993	0.962	
rajaperf_basic_pi_reduce	16 559	0.001	0.993	0.977	
dual_spmv	7100	0.431	0.990	0.908	
rajaperf_basic_mat_mat_shared	24 916	0.001	0.990	0.604	
dual_stencil	17 899	0.447	0.990	0.906	
rajaperf_apps_fir	18 124	0.001	0.984	0.952	
rajaperf_apps_mass3dpa	21 114	0.000	0.981	0.844	
cpu_hpcg	32 245	1.000	0.966	0.894	
npb_cg	25 770	1.000	0.964	0.809	

Familia	Intervalos	Frac. memoria	Exactitud	Cobertura	Exact. c
dual_fft	16 997	0.574	0.958	0.851	
rajaperf_polybench	64 720	0.175	0.949	0.938	
dual_cholesky	14 543	0.349	0.931	0.796	
cpu_lulesh	302 140	1.000	0.912	0.737	
hpccg_cpu_N46	10 666	0.178	0.844	0.822	
rajaperf_basic	5992	0.980	0.825	0.658	
cpu_cholmod	166 314	0.667	0.810	0.798	
rajaperf_stream	18 224	0.990	0.738	0.595	
dual_axpy	12 773	0.591	0.726	0.835	
npb_lu	70 757	0.701	0.677	0.385	
gap_pr	23 542	1.000	0.624	0.629	
phasic	131 923	0.324	0.617	0.906	
rajaperf_apps_edge3d	32 761	0.000	0.600	0.424	
npb_sp	56 273	0.994	0.582	0.526	
npb_mg	1778	1.000	0.551	0.569	
rajaperf_lcalcs	12 855	0.992	0.536	0.550	
npb_bt	108 605	0.370	0.515	0.411	
npb_ft	15 679	0.963	0.279	0.784	

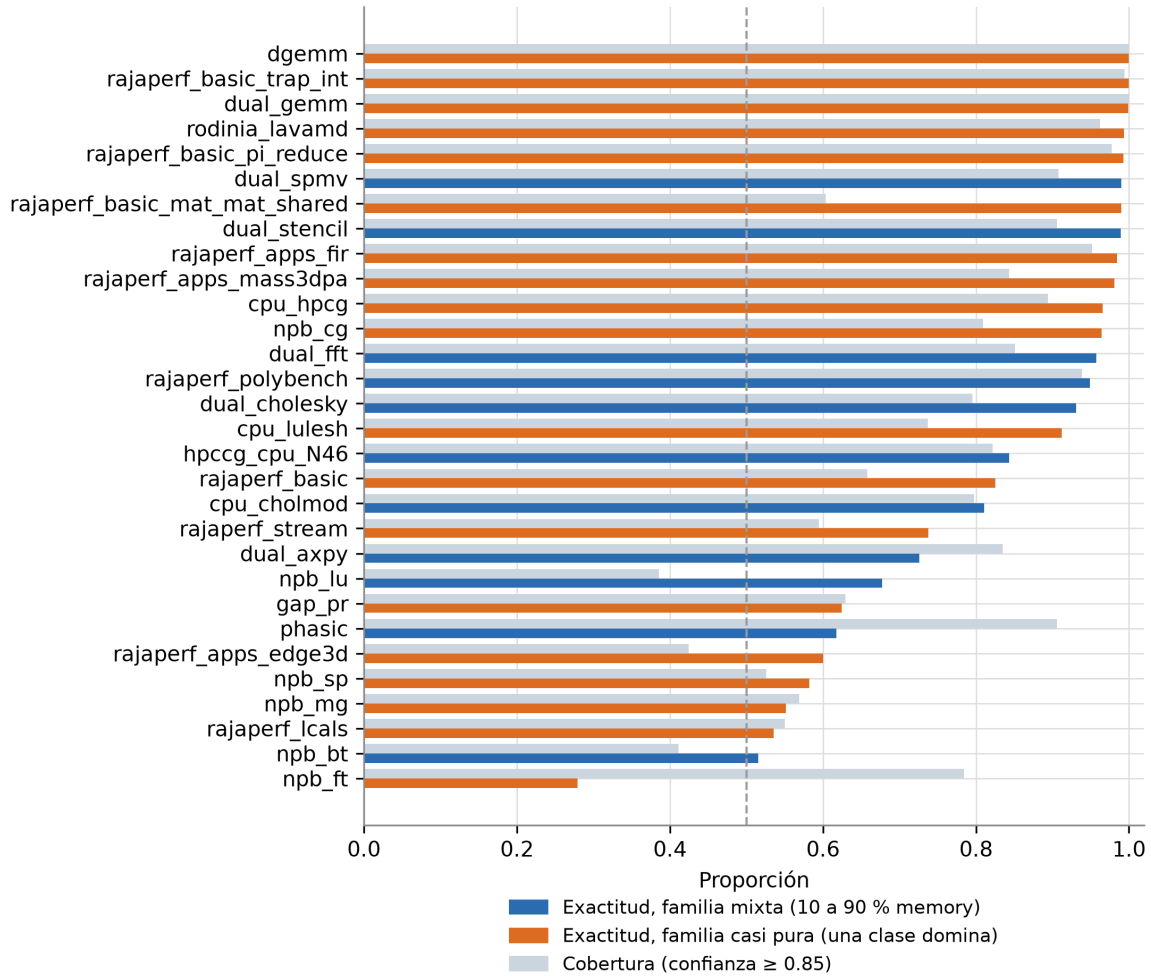


Figura 3.8: Exactitud del modelo final y cobertura de la decisión automática para cada familia retirada. El color distingue las familias mixtas (entre 10 y 90 % de intervalos memory-bound) de las casi puras. Nota. Elaboración propia.

Por nivel de frecuencia el resultado va de 0.872 en el nivel nativo a 0.749 en F7 (Tabla 3.21) y decrece al bajar el reloj. En los niveles bajos el *ridge* desplaza más intervalos hacia la clase de cómputo (la fracción *memory-bound* baja de 0.71 a 0.55) y la cobertura de la decisión automática cae de 0.83 a 0.67, lo que es consistente con que allí más intervalos quedan cerca de la frontera entre clases.

Tabla 3.21: Resultado del modelo final por nivel de frecuencia.

Nivel	Intervalos	Frac. memoria	Exactitud	Cobertura
REF	94 376	0.710	0.872	0.833
F0	85 343	0.686	0.863	0.828
F1	89 926	0.683	0.839	0.817
F2	95 825	0.665	0.834	0.801
F3	103 128	0.642	0.816	0.768
F4	113 974	0.590	0.771	0.710
F5	129 321	0.583	0.771	0.714
F6	152 766	0.566	0.764	0.717
F7	182 126	0.550	0.749	0.670
F8	250 196	0.549	0.773	0.667

Decisión selectiva y costo de inferencia

El modelo puede abstenerse cuando su confianza, definida como $\max(p, 1 - p)$, no alcanza un umbral. El umbral de 0.85 se eligió con un LOFO interno sobre las 29 familias de cada pliegue, exigiendo una cobertura media mínima de 0.60, y nunca observando la familia evaluada. Elegirlo de forma anidada, con un valor distinto por pliegue y una rejilla ampliada hasta 0.98, produce prácticamente lo mismo, 0.761 sobre lo decidido con 0.72 de cobertura, de modo que congelar un único valor no infla el resultado.

Con ese umbral el modelo decide sobre el 73.3 % de los intervalos y responde *revisar* en el resto. Sobre lo que decide, la exactitud balanceada por celda sube de 0.727 a 0.750, la F1 macro agrupada de 0.782 a 0.858 y la sensibilidad de *compute-bound* de 0.709 a 0.771 (Tabla 3.22). La ganancia es real pero modesta, y conviene dimensionarla: abstenerse al azar en la misma proporción de intervalos deja la exactitud balanceada en 0.732, es decir, sin cambio, de modo que la mejora proviene efectivamente de que la confianza identifica casos difíciles y no del simple hecho de decidir menos.

Tabla 3.22: Efecto de la decisión selectiva con umbral de confianza 0.85.

Medida	Sin abstención	Con umbral 0.85
Cobertura	1.000	0.733
Cobertura media por familia	1.000	0.769
Cobertura de la familia peor cubierta	1.000	0.385
Exactitud balanceada por celda	0.727	0.750
F1 macro agrupada	0.782	0.858
Exactitud	0.794	0.867
Sensibilidad <i>compute-bound</i>	0.709	0.771
Sensibilidad <i>memory-bound</i>	0.850	0.930

El límite de la abstención es que la confianza del modelo está mal calibrada fuera de su familia de entrenamiento (Figura 3.9). La confianza media declarada es 0.899 frente a una exactitud observada de 0.787, con un error de calibración esperado de 0.113. En consecuencia hay familias que fallan *con* confianza alta y a las que el umbral no protege: entre los intervalos con confianza mayor o igual a 0.9, *npb_ft* acierta el 24 % y *npb_bt* el 45 %. Abstenerse es útil, pero no sustituye evidencia física cuando el modelo se equivoca con seguridad.

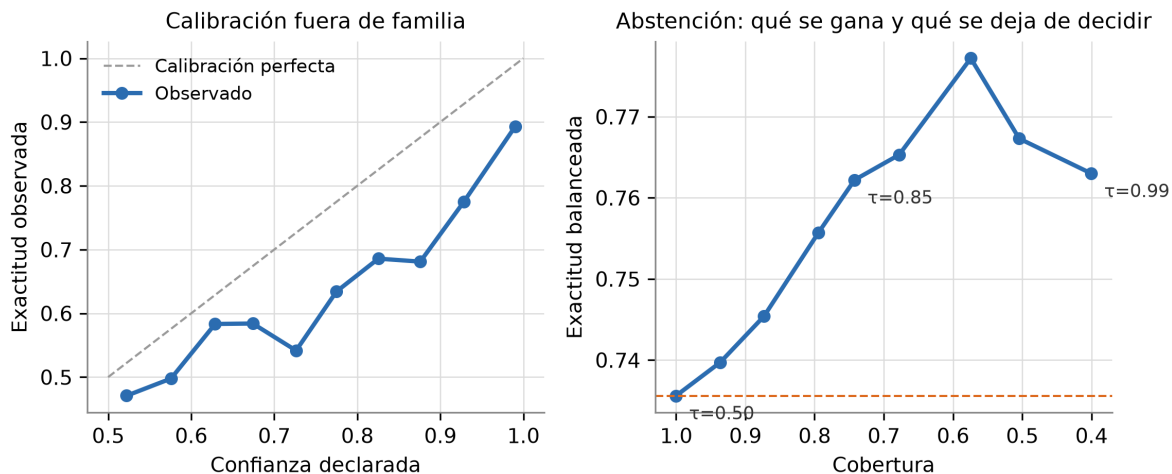


Figura 3.9: A la izquierda, confianza declarada frente a exactitud observada sobre familias no vistas; la curva por debajo de la diagonal indica sobreconfianza. A la derecha, exactitud balanceada alcanzable en función de la cobertura al variar el umbral. Nota. Elaboración propia.

La salida *revisar* activa una política conservadora: el sistema conserva la frecuencia vigente y espera dos intervalos *uncore* válidos consecutivos antes de aceptar una caracteriza-

ción física. Esta regla evita forzar una clase donde el modelo no discrimina, pero todavía no selecciona la frecuencia de menor EDP, que depende del trabajo pendiente de la Fase 4.

En cuanto al costo, la inferencia de una fila en el nodo de destino toma 232 μ s de mediana y 383 μ s en el percentil 99 para XGBoost, frente a 263 y 325 μ s de la regresión logística y 5.2 ms de los bosques. Estas cifras incluyen la sobrecarga del intérprete de Python y por tanto son una cota superior para una implementación integrada en el agente, que es donde debe hacerse la medición definitiva.

Dónde está el límite del resultado

Cuatro resultados independientes indican que el límite proviene de la información contenida en los contadores y no del modelo.

El primero es que un clasificador sin ajuste, de 25 vecinos más cercanos sobre cuatro tasas estandarizadas, alcanza 0.683 con el mismo protocolo, y 0.703 si recibe además la frecuencia. Queda a menos de 0.045 del modelo final, de modo que casi todo lo que puede extraerse de estas señales ya está extraído.

El segundo es la existencia de perfiles gemelos entre familias de clase opuesta (Tabla 3.23). Tomando la mediana de las cuatro tasas principales de cada familia y clase, hay celdas cuyo vecino más parecido pertenece a la clase contraria. El caso más claro enfrenta a *gap_pr*, etiquetada *memory-bound*, con la fase *compute-bound* de *phasic*: la primera tiene MPKI de 0.82, IPC de 1.11 y tasa de fallos de 0.76; la segunda, MPKI de 1.11, IPC de 0.96 y tasa de fallos de 0.735. Ningún modelo que reciba solo esas señales puede separarlas, y ambas están entre las familias con peor resultado.

Tabla 3.23: Celdas de familia y clase cuyo perfil de contadores más cercano pertenece a otra familia de la clase opuesta. La distancia es euclídea sobre las medianas estandarizadas de IPC, MPKI, tasa de fallos y fracción de ciclos detenidos.

Celda	Vecino de clase opuesta	Distancia	Dist. misma clase
<i>gap_pr</i> (memoria)	<i>phasic</i> (cómputo)	0.375	0.760
<i>rajaperf_lcal</i> s (memoria)	<i>rajaperf_stream</i> (cómputo)	0.382	0.424
<i>cpu_hpcg</i> (cómputo)	<i>npb_cg</i> (memoria)	0.411	1.297
<i>rajaperf_polybench</i> (memoria)	<i>rajaperf_basic</i> (cómputo)	0.446	0.799
<i>dual_spmv</i> (memoria)	<i>cpu_hpcg</i> (cómputo)	0.467	0.488

Que el problema esté en la transferencia entre familias, y no en la señal misma, se comprueba con *phasic*: al validar dentro de esa familia, separando por nivel de frecuencia,

sus dos fases se distinguen con 0.999 de exactitud balanceada, porque allí las medianas de MPKI son 1.1 y 324. Esa misma regla no sirve para otras familias. El caso opuesto es *npb_ft*, donde los intervalos de una clase y otra tienen MPKI de 2.44 y 2.46: no hay señal que separarlos, ni dentro ni fuera de la familia.

El tercero es la posición respecto del *ridge* (Figura 3.10). Los intervalos situados a menos de un factor dos del punto de inflexión son el 12.6 % del total y se clasifican con 0.575 de exactitud, frente a 0.817 en el resto; concentran el 25.1 % de los errores. La ambigüedad física cerca de la frontera explica, entonces, una parte del error, pero tres cuartas partes de los fallos ocurren lejos del *ridge*, donde la etiqueta no es ambigua.

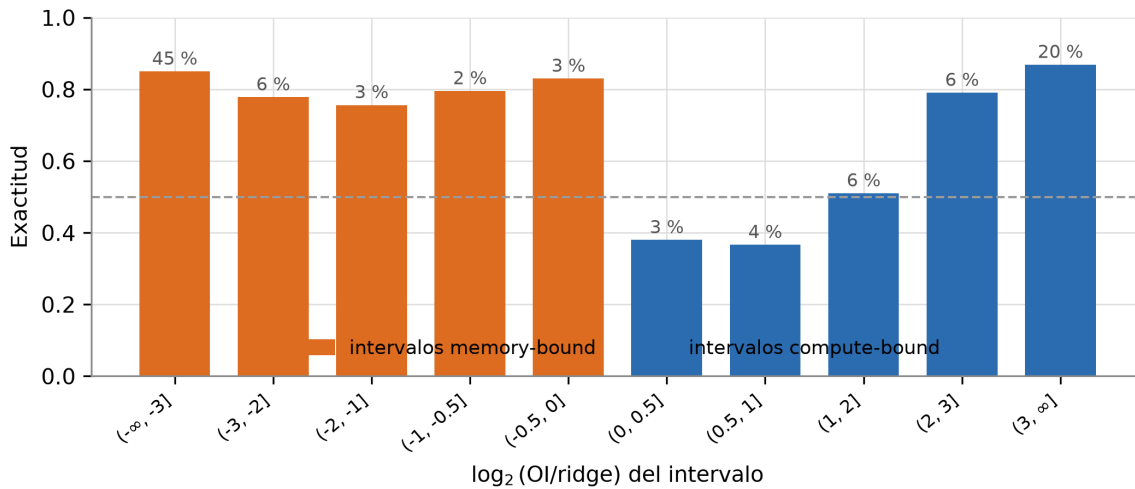


Figura 3.10: Exactitud del modelo final por franja de $\log_2(\text{OI}/\text{ridge})$. El porcentaje sobre cada barra indica qué fracción de los intervalos cae en esa franja. Las franjas negativas corresponden a intervalos *memory-bound* y las positivas a *compute-bound*. Nota. Elaboración propia.

El cuarto es la curva de aprendizaje por número de familias (Figura 3.11). Al ajustar con subconjuntos de 3 a 23 familias, el resultado sobre la familia retirada sube de 0.615 a 0.725, con una pendiente de 0.056 por unidad de logaritmo natural del número de familias. Esto implica aproximadamente 0.039 de ganancia cada vez que se duplica el catálogo. La diversidad algorítmica es la variable que gobierna este resultado, y la magnitud de la mejora disponible por esa vía, dentro del alcance de este trabajo, es pequeña.

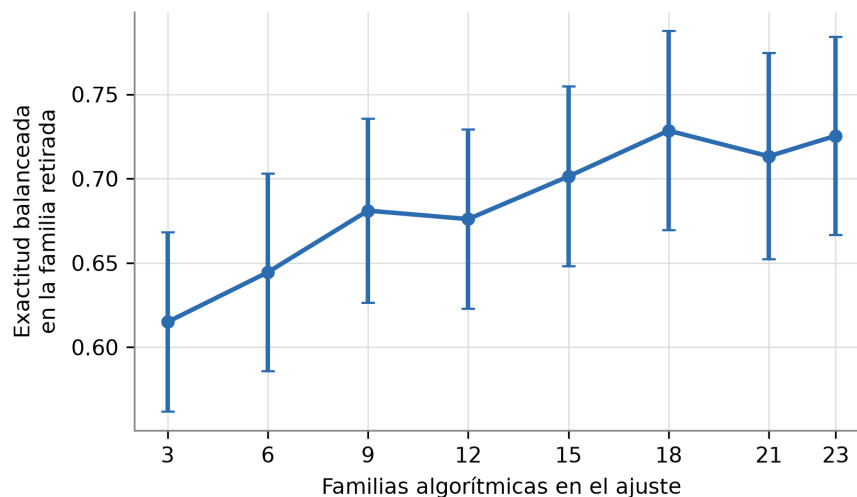


Figura 3.11: Exactitud balanceada sobre la familia retirada en función del número de familias empleadas en el ajuste. Cada punto promedia las 30 familias retiradas y tres repeticiones de la selección; las barras indican el intervalo de confianza del 95 % entre familias. Nota. Elaboración propia.

Como referencia de límite superior, se evaluó el mismo modelo con la intensidad operacional medida como entrada adicional. Alcanza una exactitud balanceada por celda de 0.9993 (recuperación de 0.9995 en compute y 0.9999 en memory), frente a 0.727 sin ella. La diferencia no representa una mejora del modelo: la intensidad operacional es, por construcción, la cantidad con la que se define la etiqueta, de modo que incluirla equivale a leerla. Por eso queda excluida del vector de entrada y se reporta solo para mostrar que la brecha entre 0.727 y 1 es información que los contadores de rendimiento no transportan.

Síntesis del clasificador CPU

El modelo disponible es un XGBoost de 100 árboles y profundidad 6 que recibe 12 tasas derivadas de los contadores de rendimiento y la frecuencia observada, se ajusta dando el mismo peso total a cada combinación de familia y clase, y responde *compute-bound*, *memory-bound* o *revisar* según su confianza supere o no 0.85. Ante un algoritmo que nunca vio clasifica correctamente el 72.7 % de los casos en la métrica balanceada por celda, cifra que sube a 75.0 % si se le permite abstenerse en el 26.7 % de los intervalos. La referencia sin capacidad de discriminar queda en 0.464 y la inferencia cuesta 383 μ s en el peor percentil medido.

Tres decisiones de diseño resultaron indiferentes y conviene dejarlo escrito, porque evita atribuir al modelo un mérito que no tiene: el algoritmo de aprendizaje, el tamaño del vector

de entrada y los hiperparámetros. Elegir Random Forest en lugar de XGBoost, usar seis variables en lugar de doce o ajustar los hiperparámetros con Optuna mueve el resultado menos de lo que lo mueve cambiar de conjunto de familias. La comparación entre seis y doce variables se midió de forma pareada, con el mismo protocolo y remuestreando familias: la exactitud balanceada por celda es 0.738 con seis y 0.727 con doce, una diferencia de 0.011 cuyo intervalo de confianza del 95 % ($[-0,028, +0,003]$) incluye el cero, y la latencia de inferencia de una fila es idéntica (mediana de 232 μ s en ambos casos, percentil 99 de 0.38 ms). Al no existir diferencia medible ni en error ni en costo, se conservó XGBoost con doce variables, que es la configuración ya serializada y auditada, y cuyo costo de inferencia es dos órdenes de magnitud menor que el de los bosques.

Lo que sí gobierna el resultado es la diversidad del catálogo frente a la información que contienen los contadores. La etiqueta se define por FLOPs sobre bytes de DRAM, mientras que el modelo observa fallos de caché, ciclos detenidos e instrucciones retiradas, que no guardan una relación fija con aquella razón cuando cambia el algoritmo. De ahí que existan familias con perfiles casi idénticos y clases opuestas, que un clasificador sin ajuste llegue casi tan lejos como el modelo final y que la mejora disponible al añadir familias sea de unas cuatro centésimas por duplicación del catálogo.

Quedan cuatro líneas de trabajo, en orden de rendimiento esperado. La primera es la calibración en línea: si el agente puede medir la etiqueta física de unos pocos intervalos de la carga en ejecución, e incorporarlos al ajuste, la exactitud balanceada sobre bloques con cambios de fase reales pasa de 0.673 a 0.849 etiquetando el 2.6 % de los intervalos. Es, con diferencia, la palanca más grande medida hasta ahora, y convierte la salida *revisar* en una acción concreta en lugar de una espera. La segunda es ampliar el catálogo con familias elegidas por su distancia a los perfiles ya cubiertos, en lugar de por volumen, y reservar algunas sin usarlas en ninguna decisión, para disponer de una prueba externa sellada. Las seis familias de cómputo denso añadidas provienen de una misma suite, de modo que su independencia entre sí es menor que la de familias de suites distintas. La tercera es medir la latencia dentro del agente y no en el intérprete, que es el criterio que falta para cerrar formalmente la elección entre la variante de seis y la de doce variables. La cuarta es la política de frecuencia de menor EDP, que no depende del clasificador sino del trabajo de la Fase 4.

Los datos, las figuras y las tablas de esta sección se reproducen con `fase2_clasificador/analysis/cpu_quality_report.py` y `docs/libro/scripts/generar_figuras_modelo_cpu.py`, sobre los artefactos depositados en `docs/libro/datos/cpu_calidad_30fam`.

3.1.6. Política de frecuencia por clase en CPU

La tabla clase–frecuencia de menor producto energía–retardo se derivó de las dos campañas con la energía y el tiempo de cada corrida completa. Toda corrida de un mismo kernel ejecuta el mismo trabajo en los diez niveles, de modo que el producto energía–retardo (energía de paquete más DRAM, multiplicada por el tiempo) es comparable entre niveles. Para cada clase se tomó la mediana de las tres repeticiones de cada kernel y se comparó cada nivel fijo con la referencia nativa mediante una prueba de Wilcoxon pareada por kernel. Un nivel solo se adopta si mejora el producto agregado de forma significativa. La clase de cada kernel es la mayoritaria entre sus intervalos etiquetados; se excluyen las sondas de calibración y las cargas de duración fija.

El resultado es *no actuar* en ambas clases (22 kernels compute y 28 memory, estos últimos con `ptrchase`, medido solo para esta política por ser un recorrido de memoria por latencia fuera del dominio del clasificador). En compute, el nivel más alto queda 0.3% por debajo de la referencia y el más bajo multiplica el producto por 8.5; de F1 a F8 la diferencia es significativa ($p < 0.03$), pero desfavorable. En memory, ningún nivel bajo mejora el producto agregado: de F3 a F8 la diferencia es significativa, también en sentido desfavorable. `ptrchase`, con $\alpha = 0.22$, apenas gana 1.0% en F1. Solo cinco kernels del tipo STREAM, con exponente de tiempo α entre 0.18 y 0.22, ganan entre 4 y 12% en los niveles F1 a F3. Aquí α es la pendiente negativa de $\log T$ frente a $\log f$.

La causa es el piso de potencia. Ajustando $P(f) = P_0 + c f^k$ a cada kernel (error mediano de 0.35 W), P_0 resulta de 85 W en mediana, casi idéntico entre kernels (entre 80 y 100 W), y representa el 83% de la potencia a 3.2 GHz; con $k \approx 2$, reducir el reloj a 0.8 GHz ahorra solo 15% de la potencia. Con ese modelo, bajar la frecuencia mejora el producto únicamente si α es menor que un valor crítico, cuya mediana es 0.14 y que en ningún kernel supera 0.33. Los kernels compute densos tienen el mayor margen de potencia, pero un α cercano a 1. Incluso con un clasificador perfecto que eligiera el mejor nivel de cada kernel, la ganancia media sería de 2.5% y la máxima de 12.5%. Este desenlace es un resultado válido de la evaluación: la decisión de actuar recae en identificar la región de α bajo, y la clase Roofline por sí sola no la determina.

3.2. Resultados de GPU

3.2.1. Validación de control y etiquetado

El actuador de GPU sostuvo el bloqueo del reloj de *streaming multiprocessors* bajo carga. Las pruebas con objetivos de 600 y 1200 MHz registraron, respectivamente, 305 y 174 lecturas activas coincidentes con el valor solicitado. La restauración de frecuencia se verificó al finalizar las campañas dedicadas.

La etiqueta Roofline se construyó con la precisión observada mediante Nsight Compute. Este control excluyó kernels con precisión mixta cuya intensidad operacional no admite comparación con un único techo y evitó atribuir a los contadores convencionales el trabajo ejecutado por Tensor Cores. En el nivel de referencia, los puntos de inflexión medidos fueron 14.965852 FLOP/byte para FP32 y 6.898377 FLOP/byte para FP64.

3.2.2. Diagnóstico del primer conjunto GPU

La primera campaña GPU reunió 540 corridas planificadas para 18 kernels. Después de calibrar el warmup, 334 corridas de 16 kernels y 11 familias quedaron disponibles para el diagnóstico inicial. El conjunto estaba desbalanceado, con 73 corridas *compute_bound* y 261 *memory_bound*, y la mayoría de las familias era homogénea respecto de su etiqueta.

El árbol de profundidad 1 obtuvo un F1 macro de 0.617 bajo validación *leave-one-familia-out*, frente a 0.553 del clasificador mayoritario. Este valor no se adopta como resultado final. La evaluación mostró que, al retirar familias completas, muchos pliegues quedaban sin ejemplos de una clase. Además, la utilización de SM no separó de forma universal los dos regímenes. *rodinia_gaussian*, por ejemplo, fue *memory_bound* según Roofline, pero alcanzó utilidades de SM suficientemente altas para que el modelo lo confundiera con una carga *compute_bound*.

La evidencia no indicó que bastara ajustar el modelo sobre el mismo conjunto. Mostró que faltaban familias *compute_bound* variadas y alejadas del punto de inflexión. Por esta razón se cambió la estrategia desde la búsqueda de hiperparámetros hacia la ampliación experimental del catálogo y la revisión del protocolo de validación.

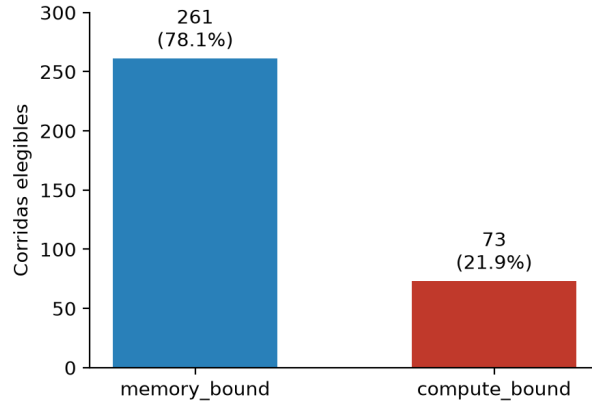


Figura 3.12: Distribución de las 334 corridas elegibles del conjunto GPU empleado en el diagnóstico exploratorio. Fuente: elaboración propia.

3.2.3. Reanálisis de campañas históricas con NVML

El bloqueo de la nueva campaña multi-frecuencia impidió incorporar observaciones nuevas. Se reanalizaron entonces campañas ya aceptadas, cuya etiqueta Roofline fue construida fuera de línea y cuya inferencia potencial dependía exclusivamente de NVML. El conjunto A, formado por las campañas modernas, contiene 394 corridas de 13 familias luego de excluir *minibude_cuda_bm1*. Esta familia aportó solo tres corridas nativas con bloqueo de reloj válido y no constituye evidencia suficiente para generalización. La composición es de 133 corridas *compute_bound* y 261 *memory_bound*. El conjunto B es una campaña histórica independiente de agosto, con las ocho familias DGEMM, Rodinia Backprop, DWT2D, Gaussian, Heartwall, LavaMD, LUD y Myocyte. Se mantuvo sellado hasta la evaluación final.

En estas campañas históricas, *ncu* aportó una intensidad operacional fuera de línea por kernel. El posproceso comparó esa intensidad con el *ridge* y difundió la misma etiqueta a las lecturas NVML de la corrida. No se registraron eventos CUPTI de lanzamiento ni contadores Roofline por lanzamiento que permitieran identificar qué kernel originó cada muestra NVML. Por esta razón, cada corrida se transformó en una sola observación mediante agregados robustos. Esta decisión conserva la correspondencia real entre el intervalo etiquetado y sus entradas, y evita convertir lecturas repetidas del dispositivo en ejemplos aparentemente independientes. Se compararon medianas, medias ponderadas por tiempo, IQR, percentiles altos, medias recortadas y razones derivadas entre potencia, utilización, reloj y actividad de memoria. También se extrajeron variables temporales, tales como ráfagas, cambios y correlaciones entre utilización, memoria y potencia. Las decisiones de representación y modelo se hicieron dentro de los datos de entrenamiento, con familias completas fuera de la partición de evaluación. Se aplicaron pesos de clase y pesos inversos al número de corridas por familia,

sin duplicar ni sintetizar observaciones.

La representación que permaneció estable fue la mediana y el IQR de utilización GPU, utilización de memoria, potencia y reloj SM. Una regresión logística regularizada con esta representación, pesos de clase y ponderación por familia alcanzó F1 macro agrupado de 0.713 en las predicciones fuera de muestra del conjunto moderno. Su exactitud agrupada fue 0.749. La exactitud media por familia fue 0.636, lo que muestra que el desempeño no es uniforme entre algoritmos. El umbral de decisión se mantuvo en 0.5. Los umbrales ajustados con el mismo conjunto produjeron cifras aparentes mayores, pero no se conservaron porque no superaron una selección anidada.

Las variables temporales merecieron una prueba separada. Un árbol de profundidad uno con la representación temporal obtuvo F1 macro de 0.755 dentro de la campaña moderna. Al aplicarlo a una campaña anterior, sin modificar sus parámetros, el F1 macro descendió a 0.294. La diferencia de cadencia y de condiciones de adquisición entre campañas domina esas variables. Por este motivo, la dinámica temporal se conserva como ablación diagnóstica y no se utiliza en el candidato final.

La evaluación externa se reservó para el conjunto B. DGEMM se excluyó por equivalencia algorítmica y Gaussian, Heartwall y LUD se excluyeron porque ya estaban representadas en A. Las cuatro familias inéditas, Rodinia Backprop, DWT2D, LavaMD y Myocyte, aportaron 140 corridas, 36 *compute_bound* y 104 *memory_bound*. Sin ajustar nuevamente el modelo ni el umbral, el candidato estático obtuvo F1 macro de 0.663 y exactitud de 0.743. La regla constante que predice siempre *memory_bound* alcanzó la misma exactitud por el desbalance del conjunto, pero solo F1 macro de 0.426. La matriz de confusión del modelo reportó 18 de 36 corridas *compute_bound* y 86 de 104 corridas *memory_bound* correctamente clasificadas. La mejora de F1 macro representa discriminación de ambas clases, no un aumento de la exactitud favorecido por la clase mayoritaria.

Se evaluó además una decisión selectiva para uso operativo. La clase se entrega cuando la probabilidad mayor del modelo supera un umbral de confianza elegido exclusivamente mediante particiones internas por familia. En caso contrario, la salida es *revisar*. Con una cobertura mínima interna de 60 %, esta política cubrió 93.9 % de las corridas de A y alcanzó F1 macro agrupado de 0.734 sobre las predicciones cubiertas. El 6.1 % restante no se contabiliza como acierto ni como error de clasificación. La mejora surge porque la política no fuerza una clase en la región donde las señales NVML agregadas son ambiguas. Esta cifra describe una opción de abstención dentro de A y no modifica el resultado externo del conjunto B.

Tabla 3.24: Resultados del candidato GPU congelado y de sus referencias.

Evaluación	Exactitud	F1 macro
Predicciones agrupadas por familia del conjunto moderno	0.749	0.713
Validación externa estricta en el conjunto B	0.743	0.663
Regla constante <i>memory_bound</i> en B	0.743	0.426
Representación temporal en el conjunto B	0.314	0.294
Decisión selectiva en el conjunto moderno	0.764	0.734

La Figura 3.13 permite interpretar estas métricas. La proyección PCA conserva la estructura global y t-SNE aproxima los vecindarios locales de las ocho variables estáticas del candidato. Ambas vistas contienen regiones dominadas por una clase, pero también regiones con etiquetas mezcladas. La separación por clase, calculada en el espacio original estandarizado, tuvo un coeficiente de silueta de 0.092. Este valor describe separación débil. Un clasificador de tres vecinos que nunca observa ejemplos de la familia evaluada alcanzó F1 macro de 0.616, coherente con una señal transferible parcial.

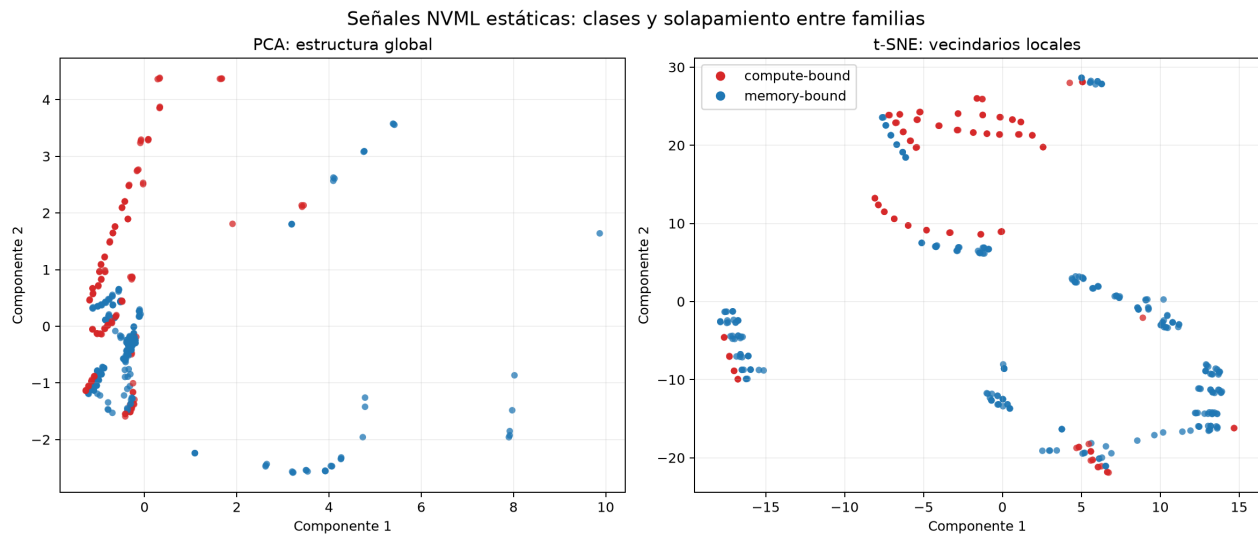


Figura 3.13: Proyecciones PCA y t-SNE de las medianas e IQR de utilización GPU, utilización de memoria, potencia y reloj SM. La proyección t-SNE es diagnóstica y no se utiliza para medir generalización. Nota. Elaboración propia.

3.2.4. Continuidad de la ruta CUPTI

La limitación observada no invalida el uso de NVML como proxy ligero. NVML mide efectos agregados de toda la GPU y no expone FLOPs, bytes movidos, ocupación o causas de espera por kernel. Sus lecturas tampoco contienen la identidad del kernel activo. Un sondeo más frecuente solo produce más lecturas del mismo estado agregado y no reconstruye los límites de los lanzamientos ya ausentes. Por ello, cargas con regímenes Roofline distintos pueden presentar telemetría similar. El resultado externo demuestra que existe señal útil para inferir parcialmente la clase, pero también delimita la capacidad de generalización que puede esperarse de estos agregados.

CUPTI mantiene dos funciones futuras diferenciadas. Su perfilado de contadores puede construir una intensidad operacional y una etiqueta Roofline por kernel en una campaña controlada. Su API de actividades puede registrar los límites temporales de lanzamientos y transferencias, necesarios para asociar esa etiqueta con el segmento NVML de la misma fase [49]. La granularidad fina requiere ambas fuentes en la misma campaña. Sin el perfilado, no se conoce la clase física de cada fase; sin la traza, no se sabe qué parte de la telemetría corresponde a esa fase. Asignar una etiqueta más fina después de capturar solo NVML inventaría esa correspondencia. Ninguna de estas funciones convierte automáticamente a CUPTI en una entrada del clasificador en producción. El perfilado de métricas puede requerir pasadas adicionales o repetición de kernels, mientras que el modelo desplegable debe conservar únicamente señales ligeras de NVML. La nueva campaña y la validación de intrusión de CUPTI quedan pendientes de resolver el bloqueo de frecuencia GPU.

Capítulo 4

Discusión

Los resultados sostienen dos tipos de contribución. La contribución metodológica corresponde a la construcción y verificación del instrumento antes de utilizar sus mediciones para entrenar. La contribución empírica corresponde al comportamiento real de la plataforma y a la capacidad de transferencia de los clasificadores CPU y GPU entre familias algorítmicas.

Buena parte de la literatura revisada en el estado del arte (los métodos de selección de frecuencia óptima de Ali et al. [2] y de clasificación consciente de DVFS de Guerreiro et al. [25], o el marco de control por refuerzo de Wang et al. [58]) asume, de forma razonable dado su alcance, que la escritura de frecuencia sobre el hardware subyacente funciona como el fabricante la documenta, y concentra su aporte en la capa de decisión: qué frecuencia elegir, o cómo aprenderla. Los resultados de actuación muestran que esa suposición debe verificarse empíricamente bajo carga real: en GPU, la prueba positiva sostuvo dos objetivos distintos durante cientos de lecturas activas y confirmó el mecanismo; en CPU, en cambio, la relectura correcta de los límites escritos quedó refutada como prueba de actuación suficiente cuando el nuevo muestreo durante la carga mostró que el turbo global mantenía una frecuencia distinta. Para cualquier política construida sobre estos actuadores, la diferencia entre éxito administrativo, estado solicitado y efecto físico solo aparece al contrastar directamente el hardware durante el trabajo real. La medición de bytes de *uncore* (sección 3.1.3) confirma el mismo principio por una vía independiente: un permiso comunicado como concedido no coincidió inicialmente con el estado verificable del sistema, y solo la comprobación directa contra el binario real dejó en evidencia la brecha y permitió confirmar su corrección posterior.

En la misma línea, la corrección del chequeo de carga externa (sección 3.1.3) ilustra un riesgo metodológico general en la validación de instrumentos de medición sobre cargas propias: una verificación de seguridad diseñada correctamente para un escenario de prueba

corto puede fallar sistemáticamente a escala real, precisamente porque a esa escala dispone de tiempo suficiente para que la señal que observa converja hacia un valor indistinguible de una condición de alarma legítima. Este tipo de falla, a diferencia de un error de sintaxis o de una condición de contorno mal definida, no se manifiesta en pruebas unitarias herméticas ni en una campaña breve; solo aparece al ejercitar el instrumento en las condiciones reales para las que fue diseñado, lo que refuerza la elección metodológica de este trabajo de tratar la campaña real como parte del proceso de validación del instrumento, y no solo como su producto final.

El hallazgo de la sección 3.1.3 extiende ese mismo principio en una dirección que las secciones anteriores no habían cubierto todavía: no basta con verificar que cada mecanismo del instrumento funciona por separado (la escritura de frecuencia, la lectura de *uncore*, la medición directa de FLOPs), porque dos mecanismos correctos en aislamiento pueden interferir entre sí de una forma que ninguna prueba unitaria, centrada por construcción en un solo mecanismo a la vez, puede exponer. El presupuesto de contadores simultáneos verificado como sostenible en la sección 2.3.11 no era, en sentido estricto, incorrecto en el momento en que se verificó; dejó de sostenerse cuando cambió una condición del entorno, una reserva de contador realizada por un mecanismo del propio sistema operativo, ajeno tanto al instrumento como al proyecto, que ninguna verificación anterior tenía motivo para volver a comprobar. La lección metodológica no es que el presupuesto físico de contadores deba tratarse como una constante de la plataforma, sino que ninguna capacidad de hardware nominal, por verificada que haya estado en un momento dado, puede asumirse estable en el tiempo sin volver a contrastarla contra el estado real del sistema; es la misma disciplina de verificación de las secciones 1.1.5 y 1.1.3, aplicada aquí no a la existencia de un permiso, sino a su persistencia.

Esa misma sección ilustra, además, el valor de una práctica adoptada explícitamente en la segunda mitad de este trabajo: antes de comprometer una campaña real de varias horas, ejecutar una réplica reducida, de minutos y sobre una fracción del catálogo o de la matriz de frecuencias, que ejercite el mismo camino de código de principio a fin. El defecto que esa práctica expuso no era detectable por inspección de código ni por las pruebas unitarias ya existentes, que verifican el formato de los datos y la degradación controlada del instrumento, pero no su interacción con el estado dinámico del sistema operativo bajo carga real; y de no haberse expuesto en una prueba de minutos, se habría descubierto, en el mejor de los casos, al final de una campaña de horas, con el consiguiente costo de cómputo perdido, o, en el peor, habría contaminado silenciosamente el conjunto de datos resultante, dado que una ventana con contadores degradados se descarta de la clasificación pero no impide que la corrida se acepte si el resto de sus ventanas son suficientes. Generalizar esta práctica, es decir, exigir una

prueba de humo a escala reducida antes de cualquier campaña de producción y no solo de la primera, es, junto con la verificación empírica ya discutida, un segundo aporte metodológico de esta fase.

La validación cruzada de los techos Roofline (sección 3.1.3) constituye una tercera instancia del mismo principio, aplicada esta vez a la integridad de la propia calibración y no a un permiso o a un mecanismo de actuación. Su hallazgo comparte una estructura común con los dos anteriores: un componente que producía un resultado plausible, un microbenchmark de calibración que compilaba, corría y reportaba una cifra de rendimiento sin ningún error visible, resultó, contrastado contra un mecanismo de medición independiente, sistemáticamente incorrecto por casi un orden de magnitud, sin que ninguna verificación interna del propio instrumento pudiera haberlo detectado: la ausencia de error no es evidencia de corrección cuando el propio mecanismo de verificación comparte la misma fuente de sesgo que el componente que pretende verificar. La elección deliberada de aplicar la corrección resultante, el ancho vectorial explícito de compilación, de forma selectiva y no uniforme entre microbenchmarks, tras encontrar que beneficiaba a uno y perjudicaba a otro, refuerza además un punto ya presente en la discusión de la sección 3.2.1: una corrección que soluciona un defecto real en un caso concreto no debe generalizarse sin verificar, caso por caso, que el mecanismo físico que la justifica aplica igual en cada uno.

El hallazgo de precisión mixta en el catálogo de GPU (sección 3.2.1) es relevante más allá de este trabajo concreto. La práctica de derivar la precisión de caracterización de un kernel a partir de su tipo de dato declarado en el código fuente, en lugar de la precisión efectivamente ejercitada por el hardware durante su ejecución, es razonable como primera aproximación, pero este trabajo encontró que puede diferir de lo medido en más del 70 % del trabajo aritmético de un kernel, y que esa diferencia puede situarse exactamente en el margen que decide una clasificación Roofline. Ningún trabajo de los revisados en el estado del arte reporta explícitamente esta verificación para GPU, lo que sugiere que la práctica de asumir la precisión declarada, sin contrastarla, podría no ser inusual en caracterizaciones similares.

Finalmente, el resultado sobre precisión de `gpu_dgemv_n4096` (sección 3.2.1) matiza, sin contradecir, el hallazgo de Guerreiro et al. sobre la sensibilidad de las cargas GPGPU a la ruta de ejecución elegida por bibliotecas de terceros [25]: una ruta acelerada por hardware especializado invalida efectivamente un método de caracterización diseñado para instrucciones aritméticas convencionales, pero no invalida necesariamente la magnitud medida, siempre que exista una vía de verificación independiente, en este caso el conteo analítico del propio algoritmo, contra la cual contrastarla.

El resultado CPU confirma que la abundancia de intervalos no equivale a disponer de muchas unidades independientes de generalización. Los 1.31 millones de intervalos elegibles proceden de 30 familias algorítmicas y más de la mitad de ellas es homogénea respecto de la clase Roofline. La magnitud de esa distinción se pudo cuantificar con el mismo modelo y los mismos datos: repartir intervalos al azar produce 0.877 de exactitud balanceada, dejar fuera un kernel completo 0.732 y dejar fuera una familia completa 0.728 (sección 3.1.5). La primera cifra no mide clasificación de régimen sino reconocimiento de la carga, y es la que obtendría un protocolo que no separase por familia. Esta comparación es, en sí misma, un argumento metodológico a favor de reportar la validación por familia aun cuando produzca cifras notablemente menores.

El resultado también acota qué palancas quedan disponibles. El algoritmo de aprendizaje, el tamaño del vector de entrada y los hiperparámetros resultaron indiferentes dentro del intervalo de confianza, y un clasificador de vecinos más cercanos sin ajuste alcanza 0.683 con el mismo protocolo. La literatura revisada concentra su aporte en esa capa de decisión, de modo que conviene precisar el alcance del hallazgo: no contradice esos métodos, pero muestra que, en este catálogo y con estas señales, la frontera entre regímenes no se mueve al cambiar de modelo. Lo que sí mueve el resultado es la diversidad algorítmica del catálogo, con una pendiente estimada de 0.034 por duplicación del número de familias. Esa relación convierte la construcción del catálogo, y no la selección del modelo, en la actividad determinante del desempeño alcanzable.

La política selectiva ofrece una respuesta operativa acotada a esa incertidumbre, aunque su ganancia debe dimensionarse con cuidado. Con el umbral congelado en 0.85 el modelo decide sobre el 73.3 % de los intervalos y su exactitud balanceada por celda pasa de 0.727 a 0.750. Abstenerse al azar en la misma proporción no cambia el resultado, de modo que la mejora proviene efectivamente de que la confianza señala casos difíciles. Aun así, la diferencia entre la F1 macro agrupada, de 0.858, y el desempeño de las familias individuales muestra que el buen resultado se concentra en familias transferibles y no constituye una garantía uniforme para una carga inédita.

Un resultado independiente del clasificador, pero decisivo para el objetivo de eficiencia, es que la clase Roofline por sí sola no determina la frecuencia de menor producto energía-retardo. En el procesador estudiado, unos 85 W de potencia de paquete no dependen del reloj de los núcleos, de modo que reducirlo de 3.2 a 0.8 GHz ahorra solo 15 % de potencia mientras alarga el tiempo en proporción a $f^{-\alpha}$. La política resultante es no actuar en ambas clases, con ganancias de 4 a 12 % únicamente en los kernels de exponente $\alpha \lesssim 0.22$ y un techo medio de 2.5 % aun con clasificación perfecta. La utilidad del clasificador reside, por

tanto, en decidir cuándo abstenerse de actuar y en identificar la región de α bajo, más que en repartir frecuencias por clase.

La abstención tampoco corrige los errores de alta confianza, porque la confianza del modelo está mal calibrada fuera de su familia de entrenamiento: declara 0.897 en promedio frente a una exactitud observada de 0.736. Entre los intervalos con confianza mayor o igual a 0.9, *nfb_ft* acierta el 32 % y *phasic* el 35 %. Por ello la probabilidad estimada por XGBoost no debe interpretarse como una probabilidad calibrada de acierto para cualquier familia inédita; su función actual es delimitar una región de operación bajo el conjunto disponible. Para la salida *revisar*, la política conservadora evita convertir la incertidumbre en una actuación DVFS potencialmente perjudicial, y la calibración en línea de la sección 3.1.5 indica que esa espera podría reemplazarse por una medición física breve con una ganancia sustancialmente mayor que la de cualquier ajuste del clasificador.

El reanálisis histórico de GPU delimita el alcance de la clasificación ligera. El conjunto A se utilizó para seleccionar el candidato y el conjunto B, una campaña histórica independiente de agosto, se mantuvo para la prueba externa. La mejora frente al diagnóstico inicial no surgió de seleccionar retrospectivamente el mejor resultado de B, sino de excluir la familia miniBUDE con control de reloj no verificable, agregar las lecturas NVML por corrida, comparar representaciones robustas y ponderar cada familia de forma equivalente durante el entrenamiento. Las razones físicas derivadas, los percentiles, las medias recortadas y la dinámica temporal se evaluaron como alternativas. La representación estática basada en medianas e IQR fue la única que conservó utilidad fuera de su campaña de origen. El F1 macro externo de 0.663 supera ampliamente la regla que siempre predice la clase mayoritaria, cuyo F1 macro fue 0.426, aunque ambas tienen la misma exactitud de 0.743. La igualdad en exactitud no es contradictoria. El modelo recupera 18 corridas *compute_bound* que la regla mayoritaria nunca reconoce y, a cambio, confunde 18 corridas *memory_bound*.

La abstención calibrada ofrece una alternativa operativa coherente con esta limitación. La política no altera la etiqueta ni elimina retrospectivamente casos cercanos al *ridge*. Solo utiliza la confianza calculada a partir de las señales NVML disponibles en producción y devuelve *revisar* cuando esa confianza no supera el umbral seleccionado dentro del entrenamiento. Al retirar de la decisión automática la región en que ambas clases presentan señales agregadas similares, la F1 macro de las decisiones cubiertas aumenta a 0.734. Esta cifra se informa junto con la cobertura de 93.9 %, por lo que no se interpreta como una mejora de cobertura total ni como sustituto de la evaluación externa en B. Esta salida permite diferir una proporción pequeña de decisiones inciertas a una caracterización posterior, en lugar de forzar una clase con evidencia insuficiente.

La separabilidad parcial de la Figura 3.13 explica esta conducta. En los datos históricos, la etiqueta procede de una intensidad operacional por kernel obtenida fuera de línea, mientras que la entrada reúne las señales de todo el intervalo de la corrida. NVML describe actividad, potencia y estado operativo de toda la GPU, pero no observa directamente instrucciones ejecutadas, tráfico de memoria, ocupación, causas de espera o la identidad del kernel activo. Cuando una ejecución tiene subfases con comportamiento distinto, el agregado conserva su efecto medio y pierde el orden y los límites de esas subfases. Por ello, una familia *compute_bound* puede ocupar una región cercana a una familia *memory_bound*. La visualización t-SNE no se interpreta como prueba de generalización, dado que su objetivo es preservar vecindarios locales y sus distancias globales no son métricas físicas. La silueta por clase de 0.092 y el F1 macro de 0.616 obtenido por vecinos que excluyen la familia evaluada confirman el mismo diagnóstico. El conjunto contiene señal física parcial, pero no una frontera estable para todas las familias.

Esta limitación también precisa el papel de CUPTI. El perfilado de CUPTI o Nsight Compute puede producir una etiqueta Roofline por kernel en una campaña controlada, mientras que la traza de actividades aporta los tiempos de lanzamiento necesarios para alinear esa etiqueta con un segmento NVML. Esa combinación permite etiquetar una fase observada, pero no puede reconstruirse retrospectivamente cuando la campaña solo conserva muestras NVML y una etiqueta global de corrida. Estas capacidades fortalecen la construcción y la auditoría de la verdad de entrenamiento. No justifican incorporar contadores de perfilado al vector de producción, pues la repetición de kernels o las pasadas adicionales necesarias para recolectarlos contradicen el requisito de inferencia ligera. La continuación metodológica consiste en desbloquear la campaña, medir la intrusión de la traza CUPTI y repetir la evaluación externa con nuevas familias, manteniendo NVML como fuente de entrada del clasificador desplegable.

Capítulo 5

Conclusiones

Este trabajo completó la construcción y validación empírica del instrumento de caracterización CPU–GPU de la Fase 1; el conjunto DVFS de CPU quedó cerrado y el de GPU aún requiere su recorrido experimental completo. Las dos campañas CPU reunieron 1470 corridas aceptadas y 1 309 755 intervalos *uncore* elegibles con FLOPs medidos por hardware, bytes reales de DRAM y frecuencia observada. Con el turbo desactivado, los nueve niveles fijos son físicamente distintos (de 3.2 a 0.8 GHz) y cada ventana se acepta por su frecuencia observada, de modo que estos datos sirven tanto para entrenar la clasificación como para comparar tiempo, energía y EDP entre niveles. En GPU, el catálogo quedó caracterizado y auditado por precisión, y `-lgc` retiene el reloj bajo carga real, aunque falta aceptar la campaña multi-frecuencia completa.

La Fase 2 produjo el clasificador de fase CPU y su evaluación por familia algorítmica retirada. El modelo final es un XGBoost con doce tasas derivadas de contadores de rendimiento y la frecuencia observada, ajustado con peso total igual por celda de familia y clase, y con una salida *revisar* cuando su confianza no alcanza 0.85. Ante una familia algorítmica que no intervino en ninguna etapa del ajuste clasifica correctamente el 72.7 % de los intervalos en la métrica balanceada por celda, frente al 46.4 % de la referencia sin capacidad de discriminar, y alcanza 75.0 % si se le permite abstenerse en el 26.7 % de los casos. La comparación controlada mostró que ni el algoritmo de aprendizaje, ni el número de variables de entrada, ni la búsqueda de hiperparámetros con Optuna modifican ese resultado más allá del intervalo de confianza, mientras que la diversidad del catálogo sí lo hace, con una ganancia estimada de 0.039 por duplicación del número de familias. La tabla de política de frecuencia derivada de las dos campañas resultó *no actuar* en ambas clases: la potencia de piso, de unos 85 W y el 83 % de la potencia a 3.2 GHz, hace que bajar el reloj solo compense en cargas con exponente

de tiempo $\alpha \lesssim 0,22$, que cumplen cinco kernels tipo STREAM con ganancias de 4 a 12 %, y el mejor caso posible con un clasificador perfecto sería de 2.5 % en promedio. El reanálisis retrospectivo de campañas GPU con NVML obtuvo F1 macro externo de 0.663 sobre cuatro familias no vistas, frente a 0.426 de la regla mayoritaria. La implementación completa del agente y la evaluación de EDP corresponden a las Fases 3 y 4.

El aporte metodológico central de esta fase, más allá de los datos concretos recolectados, es haber demostrado, y no solo enunciado, que ningún mecanismo de este instrumento se dio por funcional sin verificación directa contra el estado real del hardware: ni la disponibilidad de un contador de PMU, ni el presupuesto físico de contadores simultáneos, ni su persistencia en el tiempo frente a mecanismos ajenos al instrumento, ni la escritura de frecuencia de CPU, ni la precisión aritmética declarada de un kernel de GPU, ni la calibración de los propios techos del modelo Roofline. En varias ocasiones documentadas en este trabajo, esa verificación expuso una falla real que una confianza no verificada en la documentación del fabricante, en una medición previa ya dada por válida, o en el diseño original del instrumento, habría dejado pasar silenciosamente hacia el conjunto de datos.

5.1. Limitaciones

Este trabajo reconoce las siguientes limitaciones. Primera, la verdad CPU tiene la granularidad del intervalo *uncore*, de aproximadamente 10 ms o más, y no la de las ventanas de núcleo de aproximadamente 1 ms. Segunda, los 1.31 millones de intervalos proceden de 30 familias y no constituyen unidades independientes de generalización. El límite de 1000 intervalos por celda familia-clase puede omitir submodos minoritarios dentro de una celda, aunque preserve todas las familias y clases observadas. Tercera, la abstención mejora la calidad agregada dentro de la cobertura, pero no elimina errores de alta confianza en familias no vistas. Cuarta, el presupuesto simultáneo de PMU depende de mecanismos del sistema operativo y requiere repetir su verificación si cambia el entorno. Quinta, el control de frecuencia CPU por núcleo exige desactivar el turbo global, lo que depende de un permiso específico del nodo. Sexta, el clasificador GPU parte de 394 corridas y 13 familias, con predominio *memory-bound*; su validación externa contiene solo cuatro familias inéditas. Séptima, NVML no mide directamente los contadores microarquitectónicos que definen el régimen Roofline y las variables temporales no transfirieron entre campañas. Octava, todavía no existe una campaña GPU multi-frecuencia completa aceptada.

5.2. Trabajo Futuro

En el corto plazo, tres tareas siguen directamente habilitadas. Primera, en CPU, evaluar el modelo congelado sobre familias compute inéditas reservadas como prueba externa y llevar la tabla de política (hoy *no actuar* en ambas clases) a las aplicaciones compuestas de la Fase 3. Segunda, desbloquear la campaña GPU y medir la intrusión de CUPTI Activity antes de utilizar sus trazas para alinear etiquetas por fase o ventana. El perfilado de CUPTI debe conservarse para construir y auditar etiquetas, mientras que NVML debe permanecer como entrada del clasificador ligero. Tercera, ejecutar una nueva campaña GPU con familias inéditas y evaluar una única vez el modelo congelado en esa campaña, sin reajustar umbrales ni seleccionar características con sus resultados.

Bibliografía

- [1] Takuya Akiba, Shotaro Sano, Toshihiko Yanase, Takeru Ohta, and Masanori Koyama. Optuna: A next-generation hyperparameter optimization framework. In *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*, pages 2623–2631, 2019. doi: 10.1145/3292500.3330701.
- [2] G. Ali, M. Side, S. Bhalachandra, N. J. Wright, and Y. Chen. An automated and portable method for selecting an optimal gpu frequency. *Future Generation Computer Systems*, 149:71–88, 2023. doi: 10.1016/j.future.2023.07.011.
- [3] Oscar Antepara, Leonid Oliker, and Samuel Williams. Roofline analysis of tightly-coupled CPU-GPU superchips: A study on MI300A and GH200. In *Proceedings of the SC ’25 Workshops of the International Conference for High Performance Computing, Networking, Storage and Analysis*, pages 1205–1216, 2025. doi: 10.1145/3731599.3767497.
- [4] F. Antici, A. Bartolini, Z. Kiziltan, O. Babaoglu, and Y. Kodama. Mcbound: An online framework to characterize and classify memory/compute-bound hpc jobs. In *SC24: International Conference for High Performance Computing, Networking, Storage and Analysis*, pages 1–15, Atlanta, GA, USA, 2024. doi: 10.1109/SC41406.2024.00062.
- [5] D. H. Bailey, E. Barszcz, J. T. Barton, D. S. Browning, R. L. Carter, L. Dagum, R. A. Fatoohi, P. O. Frederickson, T. A. Lasinski, R. S. Schreiber, H. D. Simon, V. Venkatakrisnan, and S. K. Weeratunga. The NAS parallel benchmarks. *The International Journal of Supercomputer Applications*, 5(3):63–73, 1991. doi: 10.1177/109434209100500306.
- [6] E. Barrett, C. F. Bolz-Tereick, R. Killick, S. Mount, and L. Tratt. Virtual machine warmup blows hot and cold. *Proceedings of the ACM on Programming Languages*, 1 (OOPSLA):1–27, 2017. doi: 10.1145/3133876.
- [7] David A. Beckingsale, Jason Burmark, Rich Hornung, Holger Jones, William Killian, Adam J. Kunen, Olga Pearce, Peter Robinson, Brian S. Ryujin, and Thomas R. W. Scogland. RAJA: Portable performance for large-scale scientific applications. In *2019*

- IEEE/ACM International Workshop on Performance, Portability and Productivity in HPC (P3HPC)*, pages 71–81, 2019. doi: 10.1109/P3HPC49587.2019.00012.
- [8] James Bergstra and Yoshua Bengio. Random search for hyper-parameter optimization. *Journal of Machine Learning Research*, 13:281–305, 2012.
 - [9] Leo Breiman. Random forests. *Machine Learning*, 45:5–32, 2001. doi: 10.1023/A:1010933404324.
 - [10] Kay H. Brodersen, Cheng Soon Ong, Klaas E. Stephan, and Joachim M. Buhmann. The balanced accuracy and its posterior distribution. In *2010 20th International Conference on Pattern Recognition*, pages 3121–3124, 2010. doi: 10.1109/ICPR.2010.764.
 - [11] E. Calore, A. Gabbana, S. F. Schifano, and R. Tripiccone. Evaluation of dvfs techniques on modern hpc processors and accelerators for energy-aware applications. *Concurrency and Computation: Practice and Experience*, 29(12):e4143, 2017. doi: 10.1002/cpe.4143.
 - [12] Gavin C. Cawley and Nicola L. C. Talbot. On over-fitting in model selection and subsequent selection bias in performance evaluation. *Journal of Machine Learning Research*, 11:2079–2107, 2010.
 - [13] Shuai Che, Michael Boyer, Jiayuan Meng, David Tarjan, Jeremy W. Sheaffer, Sang-Ha Lee, and Kevin Skadron. Rodinia: A benchmark suite for heterogeneous computing. In *2009 IEEE International Symposium on Workload Characterization (IISWC)*, pages 44–54, 2009. doi: 10.1109/IISWC.2009.5306797.
 - [14] Tianqi Chen and Carlos Guestrin. XGBoost: A scalable tree boosting system. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pages 785–794, 2016. doi: 10.1145/2939672.2939785.
 - [15] Davide Chicco and Giuseppe Jurman. The advantages of the matthews correlation coefficient (mcc) over f1 score and accuracy in binary classification evaluation. *BMC Genomics*, 21(1):6, 2020. doi: 10.1186/s12864-019-6413-7.
 - [16] M. T. Costa et al. Characterizing the impact of gpu power management on an exascale system. In *Proc. SC ’25 Workshops of the International Conference for High Performance Computing, Networking, Storage and Analysis (SC Workshops ’25)*, pages 1524–1533, New York, NY, USA, 2025. doi: 10.1145/3731599.3767702.
 - [17] H. David, E. Gorbato, U. R. Hanebutte, R. Khanna, and C. Le. RAPL: Memory power estimation and capping. In *Proc. 16th ACM/IEEE International Symposium on Low*

- Power Electronics and Design (ISLPED)*, pages 189–194, 2010. doi: 10.1145/1840845.1840883.
- [18] Kalyanmoy Deb, Amrit Pratap, Sameer Agarwal, and T. Meyarivan. A fast and elitist multiobjective genetic algorithm: NSGA-II. *IEEE Transactions on Evolutionary Computation*, 6(2):182–197, 2002. doi: 10.1109/4235.996017.
 - [19] Bradley Efron and Robert J. Tibshirani. *An Introduction to the Bootstrap*. Chapman & Hall, New York, 1993.
 - [20] A. Georges, D. Buytaert, and L. Eeckhout. Statistically rigorous java performance evaluation. In *Proc. 22nd Annual ACM SIGPLAN Conference on Object-Oriented Programming Systems and Applications (OOPSLA '07)*, pages 57–76, Montreal, QC, Canada, 2007. doi: 10.1145/1297027.1297033.
 - [21] Pierre Geurts, Damien Ernst, and Louis Wehenkel. Extremely randomized trees. *Machine Learning*, 63(1):3–42, 2006. doi: 10.1007/s10994-006-6226-1.
 - [22] R. Gonzalez, B. M. Gordon, and M. A. Horowitz. Supply and threshold voltage scaling for low power CMOS. *IEEE Journal of Solid-State Circuits*, 32(8):1210–1216, 1997. doi: 10.1109/4.604018.
 - [23] Kazushige Goto and Robert A. van de Geijn. Anatomy of high-performance matrix multiplication. *ACM Transactions on Mathematical Software*, 34(3):1–25, 2008. doi: 10.1145/1356052.1356053. Article 12.
 - [24] B. Gregg. *Systems Performance: Enterprise and the Cloud*. Prentice Hall, Upper Saddle River, NJ, USA, 2nd edition, 2020.
 - [25] J. Guerreiro, A. Ilic, N. Roma, and P. Tomás. DVFS-aware application classification to improve GPGPUs energy efficiency. *Parallel Computing*, 83:93–117, 2019. doi: 10.1016/j.parco.2018.02.001.
 - [26] R. Hebbbar and A. Milenković. PMU-events-driven DVFS techniques for improving energy efficiency of modern processors. *ACM Transactions on Modeling and Performance Evaluation of Computing Systems*, 7(1):1–31, 2022. doi: 10.1145/3538645.
 - [27] Myles Hollander, Douglas A. Wolfe, and Eric Chicken. *Nonparametric Statistical Methods*. John Wiley & Sons, 3rd edition, 2014. doi: 10.1002/9781119196037.
 - [28] IBM. What is random forest? IBM Think Topics, 2026. URL <https://www.ibm.com/think/topics/random-forest>. Accessed: Apr. 3, 2026.

- [29] Intel. Cpu metrics reference. Intel VTune Profiler User Guide, 2026. URL <https://www.intel.com/content/www/us/en/docs/vtune-profiler/user-guide/2026-1/cpu-metrics-reference.html>. Accessed: Aug. 7, 2026.
- [30] Intel Corporation. *3rd Gen Intel Xeon Processor Scalable Family, Codename Ice Lake, Uncore Performance Monitoring Reference Manual*, May 2021. URL <https://cdrdv2-public.intel.com/679093/639778%20ICX%20UPG%20v1.pdf>. Revision 1.0.
- [31] Intel Corporation. Roofline analysis with intel advisor. Intel Advisor User Guide, 2026. URL <https://www.intel.com/content/www/us/en/docs/advisor/user-guide/2026-0/roofline-perspective-in-intel-advisor.html>. Accessed: Aug. 12, 2026.
- [32] Intel Corporation. perfmon: Performance monitoring events for intel architecture processors (evento fp_arith_inst_retired, familia ice lake), 2026. URL <https://github.com/intel/perfmon>. Accessed: Aug. 10, 2026.
- [33] Intel Corporation. Intel 64 and ia-32 architectures software developer’s manual, volume 3b: System programming guide, part 2 (performance monitoring, nmi watchdog interaction with fixed-function and general-purpose counters), 2026. URL <https://www.intel.com/content/www/us/en/developer/articles/technical/intel-sdm.html>. Accessed: Aug. 12, 2026.
- [34] Gareth James, Daniela Witten, Trevor Hastie, and Robert Tibshirani. *An Introduction to Statistical Learning: with Applications in R*. Springer Texts in Statistics. Springer, 2nd edition, 2021. doi: 10.1007/978-1-0716-1418-1.
- [35] Shachar Kaufman, Saharon Rosset, Claudia Perlich, and Ori Stitelman. Leakage in data mining: Formulation, detection, and avoidance. *ACM Transactions on Knowledge Discovery from Data*, 6(4):1–21, 2012. doi: 10.1145/2382577.2382579.
- [36] M. Kerrisk. *The Linux Programming Interface: A Linux and UNIX System Programming Handbook*. No Starch Press, San Francisco, CA, USA, 2010.
- [37] J. H. Laros III, K. Pedretti, S. M. Kelly, W. Shu, K. Ferreira, J. Van Dyke, and C. Vaughan. Energy delay product. In *Energy-Efficient High Performance Computing: Measurement and Tuning*, SpringerBriefs in Computer Science, chapter 6, pages 51–55. Springer, London, UK, 2013. doi: 10.1007/978-1-4471-4492-2_9.
- [38] Linux Kernel Documentation. CPU Performance Scaling, 2026. URL <https://docs.kernel.org/admin-guide/pm/cpufreq.html>. Accessed: Sep. 5, 2026.

- [39] Linux Kernel Documentation. `intel_pstate` cpu performance scaling driver, 2026. URL https://docs.kernel.org/admin-guide/pm/intel_pstate.html. Accessed: Sep. 5, 2026.
- [40] Linux man-pages project. `clock_gettime(3)` — linux manual page, 2026. URL https://man7.org/linux/man-pages/man3/clock_gettime.3.html. Accessed: Aug. 29, 2026.
- [41] Linux man-pages project. `perf_event_open(2)` — linux manual page, 2026. URL https://man7.org/linux/man-pages/man2/perf_event_open.2.html. Accessed: Aug. 29, 2026.
- [42] Linux perf project. `perf-stat(1)` — linux manual page, 2026. URL <https://man7.org/linux/man-pages/man1/perf-stat.1.html>. Accessed: Aug. 29, 2026.
- [43] T.-Y. Liu, J. Guo, and B. Huang. Efficient cross-platform multiplexing of hardware performance counters via adaptive grouping. *ACM Transactions on Architecture and Code Optimization*, 21(1):1–26, 2024. doi: 10.1145/3629525.
- [44] Yu Jung Lo, Samuel Williams, Brian Van Straalen, Terry J. Ligocki, Matthew J. Cordery, Nicholas J. Wright, Mary W. Hall, and Leonid Oliker. Roofline model toolkit: A practical tool for architectural and program analysis. In *High Performance Computing Systems. Performance Modeling, Benchmarking, and Simulation (PMBS 2014)*, volume 8966 of *Lecture Notes in Computer Science*, pages 129–148. Springer, 2015. doi: 10.1007/978-3-319-17248-4_7.
- [45] John D. McCalpin. Memory bandwidth and machine balance in current high performance computers. Technical report, IEEE Computer Society Technical Committee on Computer Architecture (TCCA) Newsletter, December 1995.
- [46] S. McCarty. Architecting containers part 1: Why understanding user space vs. kernel space matters. Red Hat Blog, 2015. URL <https://www.redhat.com/en/blog/architecting-containers-part-1-why-understanding-user-space-vs-kernel-space-matters>. Accessed: Apr. 3, 2025.
- [47] Douglas C. Montgomery. *Design and Analysis of Experiments*. Wiley, 9th edition, 2017. ISBN 9781119299363.
- [48] National Energy Research Scientific Computing Center (NERSC). Roofline performance model. NERSC Documentation, 2026. URL <https://docs.nersc.gov/tools/performance/roofline/>. Accessed: Aug. 7, 2026.

- [49] NVIDIA. CUDA profiling tools interface (CUPTI). NVIDIA Developer Documentation, 2026. URL <https://docs.nvidia.com/cupti/>. Accessed: Sep. 18, 2026.
- [50] NVIDIA. Nvidia management library (nvml). NVIDIA Developer Documentation, 2026. URL <https://docs.nvidia.com/deploy/nvml-api/index.html>. Accessed: Apr. 3, 2026.
- [51] NVIDIA. Nsight compute. NVIDIA Developer Documentation, 2026. URL <https://docs.nvidia.com/nsight-compute/>. Accessed: Aug. 8, 2026.
- [52] David R. Roberts, Volker Bahn, Simone Ciuti, Mark S. Boyce, Jane Elith, Gurutzeta Guillera-Arroita, Severin Hauenstein, José J. Lahoz-Monfort, Boris Schröder, Wilfried Thuiller, David I. Warton, Brendan A. Wintle, Florian Hartig, and Carsten F. Dormann. Cross-validation strategies for data with temporal, spatial, hierarchical, or phylogenetic structure. *Ecography*, 40(8):913–929, 2017. doi: 10.1111/ecog.02881.
- [53] O. S. Simsek, J.-G. Piccinalli, and F. M. Ciorba. Increasing energy efficiency of astrophysics simulations through gpu frequency scaling. In *SC24-W: Workshops of the International Conference for High Performance Computing, Networking, Storage and Analysis*, pages 1826–1834, Atlanta, GA, USA, 2024. doi: 10.1109/SCW63240.2024.00229.
- [54] Marina Sokolova and Guy Lapalme. A systematic analysis of performance measures for classification tasks. *Information Processing & Management*, 45(4):427–437, 2009. doi: 10.1016/j.ipm.2009.03.002.
- [55] P. Thamm and U. Leser. Strategies to measure energy consumption using RAPL during workflow execution on commodity clusters. *arXiv preprint arXiv:2505.09375*, 2025.
- [56] J. Treibig, G. Hager, and G. Wellein. LIKWID: A lightweight performance-oriented tool suite for x86 multicore environments. In *Proc. 39th International Conference on Parallel Processing Workshops (ICPPW)*, pages 207–216, San Diego, CA, USA, 2010. doi: 10.1109/ICPPW.2010.38.
- [57] D. Velicka, O. Vysocky, and L. Riha. Methodology for GPU frequency switching latency measurement. In *Proc. 2025 IEEE International Parallel and Distributed Processing Symposium Workshops (IPDPSW)*, pages 830–839, 2025. doi: 10.1109/IPDPSW66978.2025.00133.
- [58] Y. Wang, M. Hao, H. He, W. Zhang, Q. Tang, X. Sun, and Z. Wang. DRLCap: Runtime GPU frequency capping with deep reinforcement learning. *IEEE Transactions on Sustainable Computing*, 9(5):712–726, 2024. doi: 10.1109/TSUSC.2024.3361596.

- [59] V. M. Weaver. Self-monitoring overhead of the linux perf event performance counter interface. In *Proc. IEEE International Symposium on Performance Analysis of Systems and Software (ISPASS)*, pages 102–111, 2015.
- [60] V. M. Weaver and S. A. McKee. Can hardware performance counters be trusted? In *Proc. IEEE International Symposium on Workload Characterization (IISWC)*, pages 141–150, Seattle, WA, USA, 2008. doi: 10.1109/IISWC.2008.4636099.
- [61] S. Williams, A. Waterman, and D. Patterson. Roofline: An insightful visual performance model for multicore architectures. *Communications of the ACM*, 52(4):65–76, 2009. doi: 10.1145/1498765.1498785.

Repositorio y recursos complementarios

Repositorio público del proyecto:

<https://github.com/AdrianCCRS/hyperion>